

SAW User Manual

The SAW Development Team

Oct 28, 2026

Contents

1 Introduction **1**
- 1.1 Notation . . . 2
- 1.2 Outline . . . 2

2 Installing SAW **3**
- 2.1 Binary Distributions . . . 3
- 2.2 Building from Source . . . 4
- 2.3 Installing Solvers . . . 4
- 2.4 Installing the Python Bindings . . . 5
- 2.5 Using the Docker Images . . . 5

3 Getting Started **7**
- 3.1 Proofs In Files . . . 8
- 3.2 Proofs About Code . . . 9

4 Getting Started with the Python Bindings **15**

5 Overview of SAW and SAW's Structure **17**
- 5.1 Key Subsystems . . . 17
- 5.2 Typical / Common Use Cases . . . 18
- 5.3 Terminology . . . 19
- 5.4 Running Example . . . 19

6 Running SAW **21**
- 6.1 Invoking `saw` . . . 21
- 6.2 Invoking `saw-remote-api` . . . 24

7 Quick Introduction to the SAWScript Language **27**
- 7.1 Types . . . 28
- 7.2 Looking up Builtins . . . 28

7.3 Monads . . . 29
7.4 `return` . . . 30
7.5 `do` Blocks . . . 31
7.6 Expressions . . . 31
7.7 Common Pitfalls . . . 32
7.8 The REPL . . . 32
7.9 Experimental and Deprecated Builtins . . . 32
7.10 Haskell-Relative Notes . . . 33

8 Proofs about Cryptol Models **35**
8.1 Importing Cryptol . . . 35
8.2 Writing Cryptol Inline in SAWScript . . . 36
8.3 Semantics of Imports . . . 36
8.4 Semantics of Quasiquotation Blocks . . . 37
8.5 Stating Proof Goals . . . 37
8.6 Proving Things . . . 38
8.7 Using Solvers . . . 38
8.8 Uninterpreted Functions . . . 40
8.9 Satisfiability . . . 40

9 Introduction to Symbolic Execution **41**
9.1 Path Satisfiability . . . 42
9.2 Symbolic Termination . . . 43
9.3 Setting up Symbolic Execution . . . 44

10 Verifying Code Using Symbolic Execution **45**
10.1 Simple Example for Getting Started . . . 45
10.2 Languages for Specifications . . . 45
10.3 Specifications . . . 46
10.4 The LLVM Backend and LLVM Specifications . . . 51
10.5 The JVM Backend and JVM Specifications . . . 62
10.6 The MIR backend and Rust/MIR Specifications . . . 66
10.7 Compositional Verification . . . 83
10.8 Using Ghost State . . . 90
10.9 Extracting Code . . . 90
10.10 A Heap-Based Example . . . 91
10.11 An Extended Example . . . 93
10.12 Verifying Cryptol FFI functions . . . 98
10.13 Multi-Step Proofs and Loop Invariants with Cuts . . . 100

11 Verifying Hardware Using Yosys **107**
11.1 Processing VHDL With Yosys . . . 107
11.2 Example: Ripple-Carry Adder . . . 108
11.3 API Reference . . . 112

12 The SAWScript Language **115**
12.1 Lexical Structure and Basic Syntax . . . 116
12.2 Types . . . 119
12.3 Statements . . . 121
12.4 Expressions . . . 125
12.5 The SAWScript REPL . . . 129

13 The Python Bindings **131**

14 Interactive Proofs **133**
14.1 Transforming Term Values . . . 133
14.2 Proofs about Terms . . . 138

15 Extraction to Rocq **147**
15.1 Support Library . . . 147
15.2 Cryptol module extraction . . . 148
15.3 Proofs involving uninterpreted functions . . . 149
15.4 Translation limitations and caveats . . . 149

16 Bisimulation Prover **151**
16.1 Bisimulation Example . . . 152
16.2 Understanding the proof goals . . . 155
16.3 Limitations . . . 156

17 Command Line Reference **159**
17.1 `saw` Command Line . . . 159
17.2 `saw` Environment Variables . . . 161
17.3 `saw-remote-api` Command Line . . . 162
17.4 `saw-remote-api` Environment Variables . . . 164

18 SAWScript Language Reference **165**
18.1 Using Cryptol in SAWScript . . . 165

19 SAWScript Commands Reference **171**

20 SAW REPL Reference **173**
20.1 REPL Commands . . . 173

21 Python API Reference **177**

22 SAWCore Language Reference **179**

23 Mapping Cryptol to SAWCore **181**

24 Appendices **183**
24.1 Formal Deprecation Process . . . 183
24.2 Deprecated Items . . . 184

24.3 Glossary 186

CHAPTER 1

Introduction

SAW, the Software Analysis Workbench, is a tool for working with formal models of the computational behavior of software. It supports numerous input languages and formats and can establish proofs about many kinds of properties and relationships. It also has some ability to transform these inputs into other forms for output.

If you have C or C++ source code, you can run it through the Clang compiler and load the resulting LLVM bitcode into SAW. If you have Rust source, you can run it through a `rustc`/`cargo` plugin called `mir-json`; it dumps out the MIR intermediate representation in a form that SAW can load. If you have Java source, you can compile it and load the resulting Java bytecode into SAW. There has also been some success verifying code in other languages whose compilers produce LLVM or Java bytecode. Beyond this there is also (experimental but rapidly maturing) support for verifying hardware designs processed by Yosys.

SAW is also designed to work with Cryptol; you can load Cryptol code or Cryptol models into SAW and prove things about them. The real strength, however, comes from loading code on the one hand and a Cryptol model on the other, and proving equivalence.

All of this works because of an underlying proof language known as SAWCore. SAWCore serves as a common substrate for importing all these disparate forms of input and reasoning about them.

SAW falls (mostly) on the solver side of the solver vs. interactive proving divide. SAW supports a range of SMT and SAT solvers that can discharge most proofs. It also supports two different solver interface libraries, `what4` and `SBV`, that have somewhat different characteristics. Proofs that are too large, too complex, or too deep for solvers can be exported to external proof tools, including the Rocq proof assistant. (SAW does also have its own experimental interactive proving interface, but for the time being it is not generally recommended.)

There are some restrictions and limitations, of course. The biggest limitation is that the symbolic

execution SAW uses to construct models from code needs to be bounded. This means that the input programs must have fixed-size inputs and outputs, and all loops (including recursion) must terminate after a fixed number of iterations. There are also some unsupported constructs in the LLVM, JVM, and particularly MIR backends.

1.1 Notation

> **Warning**
>
> This section is under construction!

Inline text from programming languages (thus names of functions, keywords, etc.) appears in `typewriter font`.

Metavariables in such text appear in *`typewriter italics`*. (Thus for example a function to retrieve the nth character from a string might be described as `string_get` *`n str`*.)

1.2 Outline

This manual is divided into two major parts: a user guide, introducing SAW and the things one can do with SAW; and a reference manual, which gives a complete breakdown of all the bells and whistles.

The user guide is intended to be read in order, to the extent feasible; later sections build on earlier sections and we have attempted to avoid forward references. The reference manual is intended to be opened to the point that talks about the thing you need to know about; it makes no attempt to introduce anything, but also is intended to contain all the details you might need to know.

A series of appendices provides additional supplementary information.

CHAPTER 2

Installing SAW

There are three ways to install SAW: you can download the binary distribution of a release, compile it from source yourself, or download either or both of the Docker images.

If you are using the Python scripting interface you will also need to install the Python bindings.

2.1 Binary Distributions

SAW releases include binary builds for several platforms. These can be downloaded from the releases (https://github.com/GaloisInc/saw-script/releases) page of the GitHub repository.

There are two sets of binary builds: one that includes solver executables (from a what4-solvers (https://github.com/GaloisInc/what4-solvers) build) and one that does not. If you use one of the non-solver builds, you will need to install your own solver executables. SAW requires at least the Z3 solver and many proofs use others. See below.

A SAW binary distribution includes:

- executables for SAW and Cryptol;
- as noted, optionally executables for a range of SMT solvers;
- PDFs for the SAW manual and tutorials;
- the SAW examples;
- the Cryptol standard library;
- a copy of the `cryptol-specs` library of cryptographic algorithm specifications;

- top-level `README` and `LICENSE` files.

Note that the structure of the release is not suitable for unpacking directly into `/usr/local/bin` or any other Unix installation prefix. The recommended procedure is to unpack it somewhere in your home directory and include its `bin` directory in your `$PATH`.

2.2 Building from Source

To build from source you must first get the source. The recommended way to do that is to clone the git repository from GitHub (https://github.com/GaloisInc/saw-script).

If you want to build a release rather than the current development version, check out the tag for the release you want. For example, use `git checkout v1.3` to get the 1.3 release.

Then, you will need to also fetch a number of git submodules. This can be done with `git submodule update --init`. Note that the often-used `--recursive` flag to `git submodule update` is not needed, and is not recommended either as it will unnecessarily download substantially more material, including duplicate copies of some repositories. The SAW repository and others related to it are specifically set up to only require one layer of submodules.

Now follow the build instructions in the repository `README`. (Those instructions are not duplicated here in the interests of having one canonical copy.)

The build process places the SAW and Cryptol executables in a `bin` subdirectory of the source tree. Typical usage is to add this directory to your `$PATH` rather than installing them anywhere.

You will need to install Z3 and possibly other solvers, as described below.

2.2.1 Release Sources

In theory you can download the sources from a release (https://github.com/GaloisInc/saw-script/releases); however, the source downloads as generated by GitHub are missing their submodule dependencies as well as the proper versions of those submodules to use. This makes them unbuildable and useless. See issue #2532 (https://github.com/GaloisInc/saw-script/issues/2532).

Starting with release 1.6 there is a separate properly-built source distribution in addition to the broken GitHub-generated one. For example, for release 1.6 you want `saw-1.6-sources.tar.gz` rather than the link that says "Source code" and downloads `v1.6.tar.gz`.

2.3 Installing Solvers

Unless you use one of the binary snapshots that comes with solvers, you will need to install your own. One easy way to do this is to download a what4-solvers (https://github.com/GaloisInc/what4-solvers) binary snapshot. This includes all the external solvers SAW can use, and provides the versions that we are using upstream.

However, you can install your own copies in whatever way is convenient.

Be aware though that using either a newer or older version of any given solver can lead to regressions where given proofs stop working. This unfortunately happens more or less randomly. If you discover regressions with newer versions of solvers than we ship with `what4-solvers`, it will often make sense to report them to us and/or the upstream for the solver in question.

SAW requires the Z3 solver from Microsoft Research (https://github.com/Z3Prover/z3). In addition, it is capable of using the following other external solvers:

- ABC (https://github.com/berkeley-abc/abc)
- Bitwuzla (https://github.com/bitwuzla/bitwuzla)
- Boolector (https://github.com/Boolector/boolector) (superseded upstream by bitwuzla, deprecated in SAW)
- CVC5 (https://github.com/cvc5/cvc5)
- MathSAT
- Yices (https://github.com/SRI-CSL/yices2)

Note that the full picture of solver usage in SAW is considerably more complex than this (because among other things, there are also *internal* solvers); see the *Using Solvers* section for the full discussion.

Because each solver reacts differently to each problem, there is a strong tendency to eventually end up using all of them.

2.4 Installing the Python Bindings

The Python bindings for SAW are published on PyPI as the `saw-client` package. You can install this, or depend on it in your own project, using your favorite Python package management solution.

You can also install from the source tree using `poetry` (https://python-poetry.org/docs/#installation) as follows:

```
cd saw-python
poetry install
```

2.5 Using the Docker Images

The packages page (https://github.com/orgs/GaloisInc/packages/container/package/saw) of the GitHub repository contains prebuilt Docker images for SAW. These include both release images and a "nightly" image that is built from the current development version. All of these include solver executables and are ready to run.

These provide a simple way to get the current development version without having to build it, and are convenient if you are already using Docker.

Note that there are separate Docker images for the standalone SAW executable `saw` (for developments using SAWScript) and the server executable `saw-remote-api` used with the Python bindings for developments scripted using Python.

(And also note: there is a third image built from the SAW repository that you may see. It is called `crux-mir-comp` and provides an environment for running that tool, which is a version of Crux that uses some of SAW's infrastructure on the back end. See the Crux documentation for further information.)

If you are verifying Rust code you will likely also want the Docker image for mir-json (https://github.com/GaloisInc/mir-json).

2.5.1 Building the Docker Images

You can, if desired, build the Docker images yourself. To do this, clone the repository as described above, including the `git submodule init` step, or download the sources; then run the following shell commands:

```
./build.sh gitrev
docker build -t saw -f saw/Dockerfile .
docker build -t saw-remote-api -f saw-remote-api/Dockerfile .
```

You can of course build either or both images as needed.

CHAPTER 3

Getting Started

For your very first proof, after installing SAW, start the `saw` executable. At the REPL (the `sawscript>` prompt) type the following:

```
prove_print z3 {{ 2 < 3 }};
```

This asks SAW to prove, using the Z3 solver, that 2 is less than 3. This in turn is not a very difficult proof, so SAW will in response immediately print something like the following:

```
Theorem (EqTrue
  (ecLt Integer PCmpInteger
     (ecNumber (TCNum 2) Integer PLiteralInteger)
     (ecNumber (TCNum 3) Integer PLiteralInteger)))
```

The word `Theorem` indicates what you got. The rest is a rendering of the theorem statement `2 < 3` into SAW's internal interchange language called SAWCore.

Note that you do *not* need to be able to read the SAWCore version of the theorem statement except for certain very advanced uses of SAW. (Nonetheless we hope to make it more readable in the future…)

Let's take a closer look at exactly what this operation does. The `prove_print` command asks SAW to prove some proposition (the second argument) using some solver or proof script (the first argument) and print the result.

For the proof script we use the builtin tactic `z3` that runs the Z3 solver. The proposition `{{ 2 < 3 }}` is a Cryptol code block containing a simple Cryptol expression. Cryptol expressions can be embedded in double curly braces like this.

Now try asking for something that isn't true:

```
prove_print z3 {{ 3 < 2 }};
```

This will fail, print a stack trace showing you were in z3 (which is not super helpful in this context, but can be important in a large proof development), as well as `prove: 1 unsolved subgoals`, meaning that it couldn't prove `{{ 3 < 2 }}`, plus also `Invalid: []`, which means that the overall proof attempt failed.

The empty list in the `Invalid` result is, in general, a counterexample for which the proposition isn't true. In this case because everything is a constant, the counterexample is empty.

That isn't very interesting, so let's try something else not true:

```
prove_print z3 {{ \(x: [8]) (y: [8]) -> x < y }};
```

The proposition should be read as "for all `x` and `y` that are 8-bit unsigned bitvectors, `x < y`." That is obviously not true, so we get a failure, and a counterexample with values of `x` and `y` for which it's not true: `Invalid: [x = 0, y = 0]`. (You will probably get zero as the counterexample, but you might not; solvers are not very deterministic and might pick any of the 32,896 possible counterexamples.)

The `[8]` clauses are, as suggested above, type annotations to specify that `x` and `y` should be treated as 8-bit bitvectors. If you leave this off, the type is ambiguous and SAW rejects the proof. Unfortunately, as of the current writing, it does so with the following mysterious and almost entirely unhelpful complaint:

```
prove_print z3 {{ \x y -> x < y }};
```

produces

```
sequentToSATQuery: expected first order type or assertion:
isort 0
```

See issue #1418 (https://github.com/GaloisInc/saw-script/issues/1418).

3.1 Proofs In Files

For all but the simplest experiments you'll want to put things in a file and not type them at the REPL over and over again.

Put the example from above in a file called `myproof.saw`:

```
prove_print z3 {{ 2 < 3 }};
```

and then run

```
saw myproof.saw
```

This will print:

```
Loading file "myproof.saw"
```

Note that it does not print the `Theorem` value that the proof produces; only the REPL does that. If you want your files to print things as they run, you can use `print`. Put the following in `myproof2.saw`:

```
prove_print z3 {{ 2 < 3 }};
print "Two is less than three!";
prove_print z3 {{ 3 < 4 }};
print "Three is less than four!";
```

Running that with `saw myproof2.saw` will produce

```
Loading file "myproof2.saw"
Two is less than three!
Three is less than four!
```

Now see what happens attempting to prove something false. Put this in `myproof3.saw`:

```
prove_print z3 {{ 3 < 2 }};
print "Three is less than two!";
```

Running with `saw myproof3.saw` produces:

```
Loading file "myproof3.saw"
Stack trace:
   (builtin) in z3
   myproof3.saw:1:13-1:15 in (callback)
   (builtin) in prove_print
   myproof3.saw:1:1-1:27 (at top level)
prove: 1 unsolved subgoal(s)
Invalid: []
```

Note that it does *not* print "Three is less than two!"; it bails out instead. The stack trace tells you the line number it failed on, so if you have multiple proofs in the file you can tell which one didn't work.

If you have your shell configured to report exit status you'll see that it exits with nonzero status to show failure. This allows managing larger SAW developments with `make`.

3.2 Proofs About Code

While proving arbitrary Cryptol propositions is an important use case, SAW is perhaps most commonly used to do proofs about code. These proofs have a somewhat different form.

Proofs about code in SAW are generally done via symbolic execution. SAW uses the Crucible symbolic execution engine to run the code against arbitrary ("symbolic") inputs, and checks that this execution matches a specification written in terms of preconditions and postconditions.

In SAW these specifications are assembled programmatically; then one fires off the proof itself as a separate step.

For a simple example, place the following C code in a file called `example.c`:

```
int clamp(int x) {
   return x > 100 ? 100 : x;
}
```

Then compile it as follows:

```
clang -emit-llvm -c example.c -o example.bc
```

This produces an LLVM bitcode file that SAW can read.

Now place the following in a file `example.saw`:

```
bc <- llvm_load_module "example.bc";

let clamp_spec = do {
   x <- llvm_fresh_var "x" (llvm_int 32);
   llvm_execute_func [llvm_term x];
   let x' = {{ if x >$ 100 then 100 else x }};
   llvm_return (llvm_term x');
};

llvm_verify bc "clamp" [] true clamp_spec z3;
```

Now run `saw` on the script file:

```
saw example.saw
```

It should print `Verifying clamp...`, then `Simulating clamp...` (this is the symbolic execution stage), then `Checking proof obligations clamp...` (this is checking the conditions identified by the symbolic execution that are needed for success), then `Proof succeeded! clamp`.

To see what it does if the code doesn't match the specification, change the `x >$ 100` in `example.saw` to `x >$ 101`. The proof will fail, and provide you with a counterexample: the specification and the code do not match when `x` is 100.

Now, let's unpack what this example does. The C code contains a simple function `clamp` that takes an integer argument, and returns the argument value, clamping it to no greater than 100.

The first step in the SAW file loads the LLVM bitcode we generated with `clang`. This is done with the command `llvm_load_module`; that returns a handle that we remember as `bc` for "bitcode".

The next part is the LLVM-level specification:

```
let clamp_spec = do {
   x <- llvm_fresh_var "x" (llvm_int 32);
   llvm_execute_func [llvm_term x];
   let x' = {{ if x >$ 100 then 100 else x }};
   llvm_return (llvm_term x');
};
```

The name `clamp_spec` is arbitrary, although it's usually helpful to name specifications after the code they specify. The `do` block is a series of actions that we're going to attach to this name. This does not perform these actions right away; rather, it collects them up to be executed later. You will notice that some of the steps (both in the spec and at the top level of our proof development) bind variables with `let` and some use `<-` instead. The short explanation is that `<-` executes do-blocks and `let` doesn't. We'll return to this point in more detail later. There is a quick discussion in *the introduction to SAWScript*. More about `<-` can be found in *Programming in SAWScript*.

Now, the spec in the `do` block contains four steps. First, we call `llvm_fresh_var` to allocate a variable for use in the specification. Variables allocated with `llvm_fresh_var` like this become inputs to the verification. They are forall-quantified in the resultant theorem. Here we pass it the string `"x"` as a name for SAW to use when talking to us; `x` is arbitrary but as with the specification name it's usually helpful to name specification objects after their corresponding code objects. Meanwhile, the unquoted `x` on the left is the proof-level metavariable we store the result in. The argument `llvm_int 32` is the LLVM-level type for the variable, in this case a 32-bit bitvector. This corresponds to the C type `int` used in the C code.

The second step is that we set up the calling sequence for the function we're verifying. This specifies the arguments in terms of the specification variables we created. In this case we have one argument, `llvm_term x`. The use of `llvm_term` promotes `x`, which is a Cryptol-level value, to an LLVM-level value. See *Languages for Specifications*.

The third step is to figure the return value we want to specify. We bind it to the name `x'`.

The final step of the specification is to specify the return value for the C function. Again we use `llvm_term` to lift the Cryptol-level value to an LLVM-level value.

Having written the specification, we now can now use it by verifying the function `clamp` in the bitcode module `bc`.

llvm_verify bc "clamp" [] true clamp_spec z3;

```
Here `bc` is the LLVM bitcode module and `"clamp"` is the function
within it we want to verify.
The empty list `[]` is a list of other previously verified functions
we want to use as lemmas for verifying this function.
Our simple example function does not call any other functions, so
we provide nothing here.
In more complex verifications there will often be entries here.
```

(continues on next page)

(continued from previous page)

```
See
[Overrides and Compositional Verification](overrides-and-compositional-
↪verification).

The `true` enables _path satisfiability checking_.
In general you should leave this set to `true`, though in some cases
turning it off with `false` can improve proof performance.
<!--
   XXX: when we have a discussion of this in some advanced topics␣
↪chapter
   that doesn't currently exist, make a forward reference to it.
-->

<!--- XXX move this text
This prevents SAW from exploring certain cases of impossible execution.
In simple code like this, it has no effect; in other cases it avoids
spurious verification failures.
In some cases with complex code path satisfiability checking can
become combinatorially expensive, and in some of those cases disabling
it avoids the consequent performance problems and still successfully
verifies the code without generating spurious failures.
In general best practice is to enable path satisfiability checking,
and only turn it off if complications ensue.
-->

Finally, the last argument is a proof script or solver to use to
solve the proof goals that result from symbolic execution.
(Solver usage will be discussed further later on; see
[Using Solvers](#using-solvers).)

## Further Examples

For further worked examples, please consult one or both of the
SAW tutorials:
_LLVM/Java Verification with SAW_
and
_Rust Verification with SAW_
that you can find alongside this manual.

There is also a collection of examples in the `examples` subtree of the
source repository.

<!--
```

(continues on next page)

(continued from previous page)

```
XXX: Go through this and sprinkle forward references, certainly
XXX: all the XXX ones but probably quite a few more.
XXX: I'm putting this off because a lot of the references we ought
XXX: to have don't have targets yet, or the targets are misnamed or
XXX: in the wrong place.
-->

<!--
XXX: Also, some people are going to not really want to read any more
XXX: than this and would rather just jump in, and what they'll want at
XXX: this point is a quick overview of how to discover things (:h, :t,
XXX: :search etc.) But as things stand, I don't think that's going to
XXX: be very successful; at a minimum they'll need to read about
XXX: memory allocation, but in general the system is still too tetchy
XXX: and too opaque for that approach to work very well. We ought to
XXX: try to figure out how to cater to this kind of reader better; I
XXX: think we should revisit the question once we've condensed the next
XXX: few chapters by moving stuff inappropriate for the user guide to
XXX: the reference manual. At that point it should be a lot clearer
XXX: (and reading on a bit will likely look more appealing, too...)
-->
```

CHAPTER 4

Getting Started with the Python Bindings

Warning

This section is under construction!

CHAPTER 5

Overview of SAW and SAW's Structure

Now that you've seen SAW and run a couple simple proofs, it's time to provide some context for going deeper. Like in any other proof system, a SAW development is structured as a collection of theorems and proofs. SAW is a little different, however, because it makes the metalanguage (called SAWScript) more prominent than usual; in some cases perhaps too prominent. This emphasis on scripting proofs has led to SAW having a whole second front end based on Python. In the Python interface, Python replaces SAWScript; however, both manipulate the same internal objects and proof states.

Thus, in the same way that a web browser can be seen as an underlying system of objects that can be manipulated with JavaScript or with Web Assembly, SAW can be seen as an underlying system of objects that can be manipulated with either SAWScript or Python. This manual, accordingly, concentrates on documenting the underlying concepts and subsystems with less emphasis on explaining the concrete syntax. Examples are given in (to the extent currently practical) both SAWScript and Python. The SAWScript language itself is not introduced in detail until *The SAWScript Language*; it is sometimes necessary to program in it, but for routine proofs and developments it is usually sufficient to use the stylized forms suggested in the next few sections.

5.1 Key Subsystems

The most fundamental component of SAW is a language called SAWCore. SAWCore is a dependently-typed lambda calculus used as a common interchange language for higher-level material and for proof obligations and proof terms. The underlying SAWCore library provides an internal representation of SAWCore and an encapsulated interface to the rest of the system intended to prevent unsound manipulations.

Atop SAWCore is a proof system that keeps track of both proved theorems and also the goals and hypotheses of any proof in progress. This is, eventually, intended to provide an encapsulated interface to the rest of the system intended to prevent unsound manipulations. The proof system also includes the solver interface; SAWCore is lowered to SMTlib and sent to any of a range of external solvers for analysis.

A range of additional functional modules rest on top of the proof system and SAWCore. The largest of these is the Crucible interface. Crucible is a symbolic execution library; it is used to handle impure/stateful external code that is difficult to represent directly in SAWCore. The LLVM, JVM, and MIR (Rust) verification support, as well as the experimental x86_64 binary verification support, is based on symbolic execution using Crucible.

Other substantial chunks of functionality include the Cryptol importer (that translates Cryptol to SAWCore), the Rocq exporter (that translates SAWCore to Gallina for Rocq) and the Yosys importer (that translates hardware designs from Yosys's JSON representation to SAWCore). There is also code for handling and-inverter graphs (AIGs), support for boolean circuits and formulae in conjunctive normal form (CNF), and code for exporting to Verilog.

5.2 Typical / Common Use Cases

The most common use case for SAW is importing some code on the one hand, and a Cryptol model on the other, and proving equivalence between the two. As noted, this can be done for C code, Rust code, Java code, and experimentally for arbitrary x86_64 binaries. There has also been some success using the LLVM interface for Ada code; in principle it can be used for any compiler with an LLVM backend.

Probably the next most common use case is to prove things about Cryptol models, either properties of a single model or refinement relationships between increasingly detailed models. While Cryptol has its own solver interface and many things can be proven directly from Cryptol, it often proves helpful (or necessary) to use SAW's more sophisticated proof interface. This does not necessarily involve resorting to SAW's manual proof interface; SAW also provides more facilities for working with solvers.

One can also import a hardware design and prove it equivalent to a Cryptol model. This is still experimental, but maturing rapidly, and modulo some restrictions on what Yosys features it handles will work for anything Yosys can import. That includes both Verilog and VHDL.

There is a range of other more exotic functionality available; for years a wide variety of research ideas were bolted onto SAW because it was convenient to do so.

Also note that, like Cryptol, SAW was built first to verify cryptographic code. It is not intended to be *limited* to cryptographic code; however, its assumptions, limitations, design structure, areas of unimplemented functionality, and so forth reflect this fact. You are unlikely to run into significant problems with SAW itself when verifying classical cryptographic code (post-quantum cryptography can be a bit more difficult); however, the further afield from that core domain you venture the more likely you are to encounter complications.

5.3 Terminology

> **Warning**
>
> This section is under construction!

5.4 Running Example

> **Warning**
>
> This section is under construction!

CHAPTER 6

Running SAW

There are two almost completely different top-level ways to run SAW, and as hinted at in the previous sections they are even separate executables.

The default way to use SAW is with its native setup and tactic scripting metalanguage called SAWScript. For this approach you use the `saw` executable.

The other way is to use Python as the scripting language. This involves more moving parts; the Python bindings connect over a JSON-based RPC protocol to a server process. This requires first starting the `saw-remote-api` executable. Then your (separate) Python script talks to the running `saw-remote-api` process. SAW can then be used from your choice of Python environments, allowing for example the use of notebooks and other similar interfaces.

6.1 Invoking `saw`

The most common way to run `saw` is by running it on a file of SAWScript code:

```
saw myproofs.saw
```

If you leave the file name off, or give the `-I` option, SAW will instead start an interactive read-eval-print loop ("REPL") that you can use for experimentation.

Like many such interfaces, the SAW REPL contains, in addition to the ordinary SAWScript operations, REPL-specific commands that are written beginning with a colon (`:`). If you have a file of REPL commands that you want to run, you can do that with the `-B` ("batch") option.

The most common colon-commands are `:help` (or `:h` or `:?`) which you can use to retrieve the type and help text for a SAWScript command, and `:search`, which lets you look for SAWScript

commands by their SAWScript types.

6.1.1 Invoking `saw` with Docker

The Docker container for `saw` is a wrapper around the `saw` executable, so `saw` is the default entry point.

The simplest way to run the docker container (on a file called `myproofs.saw`) is like this:

```
docker run --rm -v .:/work -w /work \
   ghcr.io/galoisinc/saw:nightly myproofs.saw
```

This fetches the container image from GitHub, and runs it transiently (with the `--rm` argument). The `-v` and `-w` arguments bind your current working directory to the directory `/work` within the container, and makes that the current working directory inside. This allows SAW to find your file `myproofs.saw`.

If you built the container locally and called it `saw`, you can run that instead like this:

```
docker run --rm -v .:/work -w /work saw myproofs.saw
```

You can pass options to the `saw` executable by including them before the filename, and you can pass environment variables by using the `-e` option of `docker run`.

Like with the standalone `saw` executable, if you don't pass a filename it will start the REPL. If you want to do that, you'll also want the `-i` and `-t` flags to `docker run` so you can type into it and so the line editing works. Thus:

```
docker run -i -t --rm -v .:/work -w /work ghcr.io/galoisinc/saw:nightly
```

or

```
docker run -i -t --rm -v .:/work -w /work saw
```

If you want to start a shell inside the container to look around, instead of running the SAW executable, you can use `--entrypoint /bin/sh`.

You can also create and start a non-transient container image with `docker run` and then run SAW from it using `docker exec`. For more information about using Docker, consult its documentation or one of the many tutorials available on the internet.

6.1.2 `saw` Options

Common options include:

`-h` or `--help`
: Show the command-line options summary.

`-V` or `--version`
Print the SAW version and exit.

`-s` *file* or `--summary=`*file*
Write a verification summary to the given file.

`-f json` or `--summary-format=json`
Generate a machine-readable verification summary. (The default is "pretty", which is meant to be human-readable.)

`--detect-vacuity`
Check for specifications whose precondition is unsatisfiable. This defaults to off because it can be expensive and is rarely needed, but can be very helpful if one becomes suspicious that some of one's specifications are wrong.

`--no-color`
Disable terminal colors, non-ASCII output, and other terminal control codes.

`-i` *dirs*
Add *dirs* to the search path for SAWScript includes. *dirs* may be multiple directory names.

Commonly used environment variables include:

`CRYPTOLPATH=`*dirs*
Specify the search path used for Cryptol imports, as with Cryptol.

`SAW_IMPORT_PATH=`*dirs*
Specify the search path used for SAWScript includes.

`SAW_SOLVER_CACHE_PATH=`*dir*
Tells SAW to keep a cache of solver results in the specified directory.

Multiple directory names should be separated by colons on Unix and semicolons on Windows, per standard usage on those platforms.

These options are used with Java verification:

`-b` *dirs* or `--java-bin-dirs=`*dirs*
Add *dirs* to the search path used to find a Java executable. *dirs* may be multiple directory names.

`-c` *dirs* or `--classpath=`*dirs*
Add *dirs* to the Java class path. *dirs* may be multiple directory names.

`-j` *dirs* or `--jars=`*dirs*
Add *dirs* to the Java JAR list. *dirs* may be multiple directory names.

These environment variables are used with Java verification:

`SAW_JDK_JAR`
Specify the path of the `.jar` file containing the core Java libraries. Only needed if SAW cannot discover the location by finding a Java executable.

CLASSPATH
: Specify the search path for Java class files. Can include `.jar` files as well as directories.

See *the REPL reference* for the complete list of supported options and environment variable settings.

6.2 Invoking `saw-remote-api`

There are several ways to run the remote API server.

The simplest way is to just make sure `saw-remote-api` is on your `$PATH` and run your Python scripts. The Python bindings code will find and run it as a subprocess in each script. You can also set the environment variable `$SAW_SERVER` to point to the executable.

You can, however, also run it as a standalone server. Run the following:

```
saw-remote-api http --host 127.0.0.1 --port 8080 saw
```

then before running your Python run the following:

```
export SAW_SERVER_URL=http://localhost:8080/saw/
```

where you can set the port number (here 8080) and the endpoint name (here `saw`) on both ends to anything you like within reason.

If you are using Docker, you might do this:

```
docker run --name=saw-remote-api -v somedir/myfiles:/home/saw/myfiles -
↪p 8080:8080 galoisinc/saw-remote-api:nightly
```

and

```
export SAW_SERVER_URL=http://localhost:8080/
```

This runs the server in its Docker container, exports its socket to the host machine, and shares the `myfiles` directory.

Note: the Docker container's port is wired to 8080, and the endpoint is `/`.

While in principle you can run either or both of these in the background (with `&` in the shell, or the `-d` detach option in Docker), currently there is a strong tendency for SAW errors and messages to accidentally print to the `saw-remote-api` terminal instead of being sent across the RPC connection. (Such cases are bugs; feel free to report any you encounter.)

Also, occasionally it has been known to abort on error, in which case being able to restart it easily is helpful. (Such cases are serious bugs; please report any you encounter.)

6.2.1 Common `saw-remote-api` Options

`-h` or `--help`
Show the command-line options summary.

`-v` or `--version`
Print the SAW version and exit.

`--max-occupancy` *num*
Set the maximum number of sessions allowed at once. The default is 1.

`--no-evict`
Don't evict sessions if the server is full.

The following option can be given after the `http` verb on the command line when running in HTTP mode.

`--tls`
Enable HTTPS mode, that is, encrypt the network connection.

CHAPTER 7

Quick Introduction to the SAWScript Language

SAWScript is the metalanguage for the SAW system: it is the framework used to load specifications, state theorems, set up proofs, and solve proof goals. SAW's functionality is, in general, accessed by executing SAWScript builtins from SAWScript scripts.

One can program in SAWScript; however, ordinary usage does not really require it; it's generally sufficient to write sequences of builtin invocations according to the manual and sometimes group them into blocks. However, there's enough language involved that we should stop briefly for a quick introduction and tour.

As discussed in the previous chapter, the most common way to run SAW (when using SAWScript at all) is on a file of SAWScript code, although there's also a REPL. In this chapter we'll outline what goes in those files and just enough of the syntax and semantics to get started. A fuller treatment can be found in *a later chapter*.

A SAWScript file is a sequence of SAWScript statements executed in order, top to bottom, each terminated with a semicolon. Most of the statements you will use are calls to SAWScript builtin functions, so the first topic is how to call a function.

This script:

```
print 3;
```

calls the `print` builtin with the argument `3`, and will print the following when run:

```
Loading file "print.saw"
3
```

Note that there are no parentheses around argument lists like there are in C and Rust. The syntax for function calls is like ML and Haskell: you just put the argument next to the function name.

Some SAWScript functions take optional named arguments; for example, `str_concats`, which concatenates a list of strings, takes an optional argument `sep` that, if given, gets inserted as a separator between each pair of strings in the list. These arguments show up with the name and a `?` in the type of the function: the optional argument to `str_concats` appears in its type signature as `sep?String`.

Optional arguments can, obviously, be left off. To pass one, use its name, an equals sign, and an expression giving the value you want: `str_concats sep=" - " ["x", "y", "z"]` returns the string `"x - y - z"`, and `str_concats ["x", "y", "z"]` returns `"xyz"`.

7.1 Types

The basic types in SAWScript are:

Name	Description	Example values
`Bool`	truth values	`true`, `false`
`Int`	integers	`3`
`String`	strings	`"abc"`
`[`*a*`]`	lists of *a*	`[1, 2, 3]`
`()`	unit	`()`
`(`*a*`, `*b*`, ...)`	tuples	`(1, "abc")`
a `->` *b*	functions	`print`

There are many other types, most of them abstract builtin types for working with different kinds of verification tasks. These will be introduced in later chapters.

Functions are curried; that means a function of two arguments is the same type as a function of one argument that returns a function taking the other argument.

7.2 Looking up Builtins

To look up a builtin, start the REPL and use the `:h` (help) command with the builtin name:

```
sawscript> :h print
Description
-----------

    print : {a} a -> TopLevel ()

Print the value of the given expression.
sawscript>
```

This gives both the type and the builtin's documentation. You can use `:t` instead to get just the type.

Here the type is polymorphic: print will take a value of any type (the syntax `{a}` is a forall-binding, like in Cryptol).

The `TopLevel` annotation in the return type is a monad.

7.3 Monads

SAWScript is a monadic language. If you have seen programming-language monads (from Haskell or elsewhere) before, this will be familiar. If not, don't worry: there's no need to understand how it works under the covers or at any sort of deep level, and no experience with Haskell (or for that matter category theory) is needed.

You can think of SAWScript's monads as contexts for code to run in. There are five of them, all hard-wired, and no facilities for creating more or doing anything fancy with them.

There are, accordingly, two kinds of builtin.

Builtins that belong to a monad have a return type of the form *M ty*, where *M* is one of the monads, and the type *ty* is the function's actual return type. Thus the `print` builtin shown above executes in the `TopLevel` monad and returns the unit value `()`.

Builtins that have an ordinary return type are not in any monad ("pure") and can be used anywhere, but are invoked in a slightly different way, as we'll see in a moment.

For example, the `length` builtin that returns the length of a list:

```
sawscript> :h length
Description
-----------

    length : {a} [a] -> Int

Compute the length of a list.
sawscript>
```

The five SAWScript monads are:

- `TopLevel` is the main monad. The syntactic top level, that is, the statements in SAWScript files that aren't nested in something else, as well as statements typed into the REPL, run in `TopLevel`.
- `ProofScript` is a context for proof tactics. Writing your own proof scripts is an advanced topic; see the *Interactive Proofs* chapter.
- `LLVMSetup` is a context for assembling specifications for LLVM code.
- `JVMSetup` is a context for assembling specifications for JVM code.

- `MIRSetup` is a context for assembling specifications for MIR (Rust) code.

The tools available in the latter three monads are discussed in the *Verifying Code* chapter. For now, we'll concentrate on `TopLevel`. The other monads use the same syntax and work in the same way.

Because of the monads there are two ways to call a function.

A pure function like `length` that isn't in a monad is called by let-binding a variable to hold its return value, then passing it the right number of arguments:

```
let x = length [2, 4, 6];
```

This will bind a new variable `x` and store 3 in it.

A pure "function" that doesn't take any arguments is just a value. The boolean constants `true` and `false` are examples.

Meanwhile, a function in a monad is called by using `<-` instead of `let`:

```
thm <- prove_print z3 {{ 1 < 2 }};
```

This calls the solver z3 to prove that 1 is less than 2 (more on this soon) and returns a theorem value, which we save in a new variable `thm`.

If you don't want the return value, because you don't need it or because it's `()`, you can throw it away by binding it to _. You can also leave off the `<-` entirely and just call the function:

```
_ <- print 3;
print 3;
```

You can think of `<-` as executing the monad function on its right hand side. (What it actually does is give it the current context to run in. Thus, you can't use `<-` with functions in other monads, only the one you're currently in. So you can only use `TopLevel` functions at the top level, and you can't use `TopLevel` functions when assembling an LLVM specification in `LLVMSetup`.)

A monad function that takes no arguments does nothing until it's executed with `<-`.

7.4 `return`

The `return` builtin is a monad function that takes an argument and hands it back when executed. Thus these statements are equivalent:

```
let x = 3;
x <- return 3;
```

Note though that, confusingly, `return` is *not* a control-flow operator; it just creates a monad function out of an ordinary value. Blame Haskell (where it originated) for the confusion.

7.5 do Blocks

One can group a series of statements into a nested block. The syntax for this is `do { statement... }`; hence these are called `do` blocks. A do-block is a monad function that takes no arguments. The last entry in a `do` block must be an expression without a variable binding; it produces a result value for the block. It will often be the `return` builtin; this is how the name originated.

The statements in a `do` block must all be in the same monad, but it can be any monad. To write an LLVM specification using the `LLVMSetup` monad, write a `do` block using the `LLVMSetup` builtins (as noted before, these are introduced in the *Verifying Code* chapter) and, to avoid trying to execute it in `TopLevel`, let-bind it rather than using `<-`.

Thus for example:

```
let spec = do {
   llvm_execute_func [];
   return ();
};
```

The last statement produces `()` as the result value for the block, which is what's expected for these specifications.

Here `spec` gets bound to a no-arguments monad function of type `LLVMSetup ()`, which can then be passed in for verification. The verification backend creates and applies the `LLVMSetup` context needed to execute it.

This works the same way for the other setup monads.

7.6 Expressions

The arguments to functions are expressions, not statements. The right-hand side of a `let` statement is also an expression. Thus calls to pure functions can appear as arguments to functions (you'll need parentheses to group them). Calls to monadic functions cannot. You must execute the monadic function with `<-` and bind the result to a new variable, then pass the variable as the argument expression.

There is one exception to this: when the argument is itself supposed to be a monadic function. In this case you should pass the monadic function itself without executing it with `<-`. This comes up with the `ProofScript` arguments to verification functions as well as the `LLVMSetup`/`JVMSetup`/`MIRSetup` specification blocks. Builtins like these that take monadic functions directly execute them themselves at the proper point.

SAWScript also has no infix operators; expressions are basically either literals of various kinds or function calls. Computations are done in Cryptol.

7.7 Common Pitfalls

Because functions are curried, missing arguments are not an error; instead you get a function value back. If you didn't intend this it will cause potentially confusing type errors downstream.

Another common problem is to accidentally leave off the semicolon at the end of a statement, especially one that's a function call. If the next line is also a function call, it and its arguments will be interpreted as excess arguments to the first function, also producing potentially confusing type errors.

7.8 The REPL

The REPL accepts statements in `TopLevel`. (There is also a `ProofScript` REPL, but that's an advanced topic.) You may leave off the semicolons at the ends of statements.

It will also accept pure (non-monadic) expressions. This can sometimes lead to confusion because code that will be accepted at the REPL causes type errors if pasted into a SAWScript file. The simplest way to replicate the REPL behavior in a SAWScript file is to wrap pure expressions in the `print` builtin; in addition to fixing the types, it will print the results like the REPL does. (If you just want to compute the value and not print the result, you can use the `return` builtin instead.)

```
print "Does not print:";
return (length [2, 4, 6]);
print "Prints:";
print (length [2, 4, 6]);
```

produces

```
Loading file "test.saw"
Does not print:
Prints:
3
```

7.9 Experimental and Deprecated Builtins

Some of the primitives available in SAW at any given time are experimental. These may be incomplete, unfinished, use at your own risk, etc. They may also be unstable and change type or behavior in future releases. The functions and commands associated with these are unavailable by default; they can be made visible with the `enable_experimental` command.

Other primitives are considered deprecated. Some of these, as the *deprecation process* proceeds, are unavailable by default.

They can be made visible with the `enable_deprecated` command.

7.10 Haskell-Relative Notes

Haskell veterans will want to know that SAWScript is eagerly evaluated, not lazily, even though it's monadic. For non-Haskell folks, this basically means that execution works the way you expect and things never happen out of order.

CHAPTER 8

Proofs about Cryptol Models

Cryptol is a modeling and specification language for cryptographic algorithms. It is integrated into SAW and used for a broad range of tasks. Use of SAW for essentially any purpose requires use of Cryptol. Thus, the Cryptol manual (https://galoisinc.github.io/cryptol/master/RefMan.html) is an important additional resource for SAW users.

In this chapter we discuss how to import Cryptol models and prove properties about them.

8.1 Importing Cryptol

Cryptol modules can be imported with `import`. For example:

```
import "Foo.cry";
```

The name to import can be a filename in double quotes, or a Cryptol module name, possibly qualified with `::`, *without* quotes. The environment variable `CRYPTOLPATH` is used to search for Cryptol modules. SAW uses the same code to do this as Cryptol itself, so the behavior should be the same, including in complicated cases.

To import a submodule (Cryptol submodules are ML-style modules nested within source files), use the keyword `submodule` before the module path.

By default the contents of the imported module are placed in scope. If this isn't what you want, you can use the keyword `as` to introduce a qualified name.

After

```
import Foo::Bar as Baz;
```

the contents of the module `Foo::Bar` are available under the name `Baz`; e.g. `Foo::Bar::myfunc` appears as `Baz::myfunc`.

It is also possible to restrict the set of symbols imported. A list of symbols in parentheses after the rest of the import restricts the import to the symbols named; others are hidden. Alternatively, a list with the keyword `hiding` hides the listed symbols and imports the rest.

For example,

```
import Foo::Bar as Baz hiding (myfunc);
```

prevents referring to `Baz::myfunc`.

Partial imports are often useful to reduce namespace pollution or avoid name conflicts.

There is an additional mechanism `cryptol_load`, which is almost but not quite equivalent to `import ... as`.

The syntax `Baz <- cryptol_load "Foo/Bar.cry";` is comparable to `import "Foo/Bar.cry" as Baz;` with one important difference: the returned handle `Baz` is a first-class value and can be passed to SAWScript functions if doing complicated things. In the `import` form that does not work; it can be used as a module qualifier only. Eventually the `import` form will become equivalent, at which point `cryptol_load` will be deprecated.

Note that `cryptol_load` does not support qualified module names or partial imports.

8.2 Writing Cryptol Inline in SAWScript

Typically, complex or nontrivial models will be written as one or more external Cryptol modules and imported into SAW. However, it isn't necessary to create a whole Cryptol module for a few simple definitions. The quasiquotation syntax `{{ expr }}` allows embedding a Cryptol expression directly into SAWScript. We have already seen this in the introductory examples.

If you want to refer to a Cryptol type, the syntax for that is `{| Ty |}`.

You can also use the syntax `let {{ decls }};` anywhere the SAWScript `let decl;` form can appear, which lets you write a Cryptol-level declaration, or possibly several of them.

8.3 Semantics of Imports

All Cryptol imported, or introduced by quasiquotation, is translated into SAWCore. (Recall that SAWCore is SAW's internal interchange and proof language.)

This is done at the point in SAWScript execution where the import or quasiquotation block occurs. Unless doing complex things in SAWScript (discussed *later*), the primary consequence of this is that

type errors and even syntax errors in embedded Cryptol are not detected until "runtime". This can be annoying. However, it allows, within certain restrictions, Cryptol numeric types to be chosen at "runtime" rather than being fully constant; this in turn allows constructing more flexible proofs than would otherwise be possible.

8.4 Semantics of Quasiquotation Blocks

Embedded Cryptol blocks can refer to elements from imported Cryptol modules (complete with module qualifiers where needed, etc.) However, they can *also* refer to the following other things from the surrounding SAWScript:

- Values whose SAWScript type is `Term`, regardless of where they came from, as long as they have an underlying SAWCore type that can be represented in Cryptol; this includes e.g. values from non-Cryptol importers.
- Values whose SAWScript type is `Type`, regardless of where they came from, as long as they have a kind (type of type) that can be represented in Cryptol. These appear as Cryptol types.
- SAWScript integers. These appear as Cryptol numeric types. If you want to use one as a Cryptol numeric value, you can lower it with ` in your Cryptol expression, as usual in Cryptol.
- SAWScript boolean and string values appear as Cryptol boolean and string values.

Currently because SAW strings are Unicode and Cryptol strings are bytes, the translation of strings is unsatisfactory. This will almost certainly change in the future.

Unfortunately, due to internal restrictions, when an element cannot be made available in the Cryptol environment because its type is unsuitable, its name just disappears and references to it fail without any direct explanation. If you think something should be visible from Cryptol and it isn't, check its type.

8.5 Stating Proof Goals

A proof goal can be any well-typed Cryptol expression that produces a boolean. (It can, but need not be, something specifically declared as a property in Cryptol.) The convention in Cryptol is that quantified proof goals are expressed as lambdas (that is, functions) rather than having forall-quantified types in Cryptol syntax. The operator for implication is ==>. Thus, for example, to prove that addition of integers respects less-than, the proof goal would be

```
{{ \(x : Integer) -> \y -> x < y ==> x + 1 < y + 1 }}
```

or equivalently

```
{{ \(x : Integer) y -> x < y ==> x + 1 < y + 1 }}
```

Beware that if you don't specify the types when writing such a goal, the resulting goal may be polymorphic (for example, in this case it will range over all types in Cryptol `Num`) and then solvers will refuse to handle it.

Also beware that if you misspell the type name, Cryptol will assume you intended to introduce a type variable, with the same result.

8.6 Proving Things

The basic mechanism for proving Cryptol proof goals is the SAWScript function `prove_print`:

```
prove_print z3 {{ \(x : Integer) -> \y -> x < y ==> x + 1 < y + 1 }};
```

It takes two arguments; the second is the goal, which will usually be a Cryptol expression in double-braces as shown here, and the first is a proof script.

Unless using the manual proof interface, which is still largely experimental and not really recommended for other than advanced users (see *Interactive Proofs*), the proof scripts you'll be using will be solver invocations.

The `prove_print` function fails (returning to the REPL if interactive and exiting with failure if not) if the proof does not succeed. Beware: a SAWScript function `prove` also exists. It returns a success/failure value, rather than failing; accidental use of `prove` instead of `prove_print` can result in proofs failing more or less silently. (At some point `prove` will be deprecated to remove this footgun; in the meantime, be careful.)

8.7 Using Solvers

The basic proof scripts are just solver names. For example, the `z3` proof script runs the Z3 theorem prover, the `abc` proof script runs the ABC prover, and the `cvc5` script runs the CVC5 solver. All of these are reasonable choices.

Others supported by SAW include Bitwuzla (`bitwuzla`), Boolector (`boolector`, deprecated because it's been replaced upstream by Bitwuzla), MathSAT (`mathsat`), and Yices (`yices`). There is also an internal solver/decision procedure called `rme` that uses Reed-Muller expansions.

The question of which solver to use is fundamentally unanswerable: they are all different internally, and handle different sets of things well, and it's extremely difficult to characterize those sets from outside.

The best general guidance we can offer is: if one doesn't work, or takes excessively long to run or runs out of memory, try another.

There are a couple specific points worth mentioning:

- A lot of people reach for `z3` by default, which is as much because it's better known than for any particular other reason. At one point it had broader coverage of theories than others; that

is not true so much any more. This does not make it a *bad* choice; but sometimes, others are faster.

- Bitwuzla (and Boolector before it) are specialized to bitvectors and don't support integers (integers in the "math integer" and Cryptol `Integer` type sense) or many other theories; however, proofs near code tend to involve a lot of bitvectors and sometimes this is what you want.
- The internal `rme` solver is based on an algorithm that specifically handles chains of exclusive-or efficiently, rather than (as often happens) expanding it in terms of plain and/or. This makes it work particularly well on the Galois field operations that show up, for example, in AES.

Also be aware that different versions of the same solver often also perform differently on any given proof, and newer versions are not always better.

Ultimately, the reason SAW supports so many solvers is that the only real way to deal with the question of which solver to use is to punt it to the user.

8.7.1 Further Solver Scripts

There is also a collection of additional solver scripts that run the solvers in slightly different ways.

The general form of these is

```
[offline_][LIBRARY_][unint_]SOLVER[_VARIANT]
```

The `LIBRARY` part can be either `sbv` for the `SBV` solver interface library or `w4` for the `what4` solver interface library. Neither of these is better nor worse; they're just different. Using one or the other can occasionally unstick a proof that otherwise runs forever, or avoid a particular idiom that causes one of the solvers to complain or fail. Note that the default interface library if not requested specifically can vary; check the help text for each function with the `:h` REPL command if in doubt.

The `unint` fragment, when present, indicates a proof script that takes an additional argument, which is a list of names to be treated as uninterpreted. See the next section for further discussion.

The `offline` fragment, when present, indicates a proof script that takes a filename as an additional argument. Instead of running the solver, SAW will dump the solver query out to the file. This can be used to inspect the solver query as well as to run the solver by hand; the latter can be useful if you want to experiment with solver options or different solver versions or other things SAW does not provide good direct access to. Note that while all solvers theoretically speak SMTlib, each has its own dialect, so the offline output for each solver will be slightly different. Also note that the offline solver tactics always succeed; they assume you will run the solver yourself and the proof will work.

Not all combinations exist; in particular the offline tactics use `what4` and generally also include `unint_`.

Use `:search ProofScript` or `:search (_ -> ProofScript ())` to find everything that exists. (The second form excludes things like `prove_print` that take proof scripts. Both forms will also show you interactive proof tactics, especially if you have enabled experimental features.)

If you find that a combination you want to use is missing, please file a bug report.

8.8 Uninterpreted Functions

The most common technique to make a solver proof go faster is to mark certain functions as uninterpreted. This hides the implementation of a function. In fact, it replaces it with an arbitrary function where the only thing we know about it is that it *is* a function in the math sense: given the same argument, it produces the same result. This prevents the solver from wasting time exploring the definition. Note that this is a generalization of the proof goal: it replaces a goal that asks for something to be true about a specific function with one that asks for it to be true about all functions. So it's a safe transformation.

This is an extremely helpful thing to do in many cases when reasoning about an equivalence that has the same function appearing on both sides. It's very common that the function will also have the same argument on both sides, and that if it doesn't, it's wrong. In this case, preventing the solver from looking inside the function can save a lot of time.

Even though, arguably, converting a function to uninterpreted should be its own interactive proof tactic separate from (and used prior to) running a solver, it is common enough, especially in proofs about cryptographic code, that these compound proof scripts have been set up.

8.9 Satisfiability

SAW can also do satisfiability queries as well as proofs. The command is `sat_print`; it is used essentially the same way as `prove_print`. (There is also a `sat` that is, like `prove`, generally best avoided.) On success it prints the satisfying assignment found.

For example:

```
sawscript> sat_print z3 {{ \(x : Integer) (y : Integer) -> x < y }}
Sat: [
x = 0
y = 1
]
```

In the case of `prove_print` the goal is read "for all `x` and `y`…"; here it's read "exists `x` and `y` such that…".

There is, alas, currently no good way to state goals that require quantifier alternation. Since solvers cannot in general handle them, this is not a huge limitation in practice.

CHAPTER 9

Introduction to Symbolic Execution

Analysis of LLVM, Java, and Rust and in SAW relies heavily on *symbolic execution* (sometimes also called *symbolic simulation* or even just *simulation*), so some `background on how this process works can help with understanding the system and its behavior. If you are already reasonably familiar with symbolic execution and its concepts and limitations, you can safely skip this chapter.

At the most abstract level, symbolic execution works like normal program execution except that program values can be *symbolic*: instead of a single concrete value like `3`, a value can be a symbol that stands for any value of the proper type, or a subset of those values, or an expression in terms of such symbols. Typically we create fresh symbols to serve as the program input, and then as symbolic execution proceeds, new values are computed from old ones and are processed as expressions in terms of those input symbols. Thus a single symbolic execution is equivalent to doing many concrete executions at once. (Ideally, one symbolic execution covers all possible concrete executions, but as we'll see it isn't quite that simple.)

Consider the following C function that returns the maximum of two values:

```
unsigned int max(unsigned int x, unsigned int y) {
    if (y > x) {
        return y;
    } else {
        return x;
    }
}
```

If you call this function with two concrete inputs and assign the result, like this:

```
int r = max(5, 4);
```

then it will assign the value `5` to `r`. Now, however, we can create two fresh symbols `a` and `b` and call the function with them as the arguments:

```
int r = max(a, b);
```

The symbolic value that we store in `r` is some form of the expression `if b > a then b else a`, perhaps the specific C expression `b > a ? b : a` and possibly something slightly different.

(Aside: whether symbolic expressions inside a symbolic execution engine are represented in the input language or some other language is a design decision. The tradeoffs involved are well beyond the scope of this introduction.)

To generate this expression, however, you will note that we have to execute both branches of the `if` statement in the original function. Any concrete execution can evaluate the `if` condition to either 0 or 1 and then execute one branch of the `if` and not the other. However, during symbolic execution the `if` condition is the symbolic expression `b > a`, which could be either true or false. Therefore we need to proceed down both branches of the `if`. This in turn produces (in general) two different program states afterward, which then need to be merged back together. This merge applies the `if` condition to the results of each side, and introduces the `if` that appears in our symbolic result.

You will have noticed that the symbolic expression we get from calling `max` with arbitrary inputs is scarcely different from the original code; this is because `max` is a very simple function and we have used the most simple possible input context.

9.1 Path Satisfiability

Suppose, however, that we already knew something about the inputs before calling the function. Consider this calling context instead:

```
int minmax(unsigned int x, unsigned int y) {
    int r;

    if (x > y) {
        r = max(x, y);
    } else {
        r = min(x, y);
    }
    return r;
}
```

(This is perhaps a silly function to write, but it will serve as an example.)

If in turn we call this with symbolic inputs `a` and `b`, when we get to `max` we know that `a > b`, because we're inside the first branch of the `if` in `minmax`. This expression (that tells us what must be true to

get where we are) is called the *path condition* because it's associated with the execution path that we took to get to the current program point.

Now when we reach the `if` in `max`, it checks `b > a`. We can conclude that this is false: there is no satisfying assignment that makes `a > b && b > a` true. We can therefore skip the true branch of the `if` entirely; the symbolic return value we get from `max` is just `a`. What we have just done is called *path satisfiability checking*: checking whether a path that we're proposing to take (the true branch of the `if` in `max`) is reachable, which is true if and only if the complete path condition is satisfiable.

Assuming similar logic in `min`, the overall symbolic result we get from `minmax` is `if a > b then a else a`, which the symbolic execution engine will likely simplify further to just `a`. As noted, it's a silly function. But don't assume that path satisfiability is trivial. Every `assert` has a branch inside it, and the side of the branch that aborts is supposed to be unreachable. Detecting when it isn't is one of the primary applications of symbolic execution.

9.2 Symbolic Termination

Programs that contain loops create additional challenges. When we reach the condition of a loop, unless we can determine that the condition is definitely false, we generate two cases: one where the loop ends and one where we go around the loop again. This means that unless the symbolic program state evolves to a configuration where the condition is definitely false, we will keep generating more cases where we go around the loop again, and do so forever.

Loops that reach a definitely false state are said to *symbolically terminate*. For example, loops with concrete, constant bounds symbolically terminate: if the count starts at 0, and goes up by one on each loop iteration, and the condition is, say, `i < 15`, after unrolling the loop fifteen times `i` will be `15` and we stop.

Other loops may also symbolically terminate. The following contrived example symbolically terminates, because as it goes it accumulates (in its path condition) assertions about successive values of `i` not being zero mod 8. Because 5 and 8 are relatively prime, it will eventually accumulate all 8 possible values, of which one must be zero; at that point there is no satisfying assignment for `i` that continues the loop (this is within the ability of a solver to discover) and we stop.

```
uint8_t f(uint8_t i) {
  int done = 0;
  while (!done) {
    if (i % 8 == 0) done = 1;
    i += 5;
  }
  return i;
}
```

Counting loops with symbolic bounds in general do not symbolically terminate. However, loops with symbolic bounds where the symbolic bound also in turn has a concrete bound do.

```
for (i = 0; i < n; i++) {
    // do something
}
```

Given a concrete bound on `n`, there are only a finite number of values it can take, and consequently there are also a finite number of values for `i`, and eventually we exhaust them all. If `i` and `n` are math integers or bignums, this loop will never symbolically terminate without an explicit constraint on `n`. If (as in C code) they are machine integers, however, it will, because there is then an implicit maximum value for `n`. However, this will evaluate the loop as many times as there are positive machine integers of the type (less 1) which may be prohibitively slow and functionally equivalent to nontermination.

The inability to handle unbounded loops is a fundamental limitation of symbolic execution. (In program analysis terms, symbolic execution is *precise* but not *complete*.)

There are several approaches commonly taken in practice to approximate the behavior of unbounded loops and still get useful proof results. SAW's support for these is discussed in later chapters.

Note that disabling path satisfiability checking, which is an advanced feature, will also necessarily disable most of these checks. With path satisfiability checking disabled, a loop will only terminate if the program reaches a state where its condition evaluates to the constant `false`.

9.3 Setting up Symbolic Execution

A symbolic execution system is very similar to an interpreter, but with a different notion of what constitutes a value and it executes *all* paths through the program instead of just one.

The process of writing a specification for a piece of code is thus similar to building a test harness for that same code.

More specifically, the setup process for a test harness typically takes the following form:

1. Initialize or allocate any resources needed by the code. This typically means allocating memory and setting the initial values of variables.
2. Execute the code.
3. Check the desired properties of the system state after the code completes.

Accordingly, three pieces of information are particularly relevant to the symbolic execution process, and are therefore needed as input to the symbolic execution system:

- The initial (potentially symbolic) state of the system.
- The code to execute.
- The final state of the system, and which parts of it are relevant to the properties being tested.

In the next chapter, we'll describe how SAW's symbolic execution support operates in the context of these key concepts.

CHAPTER 10

Verifying Code Using Symbolic Execution

Proving things about Cryptol models can be quite useful; however, as discussed before, the most common use case for SAW is probably verifying code against a Cryptol model. There are three parts to this: writing the specification to verify against, loading the code to verify, and then running the verification. The specification is the most complicated part of this, so we'll take it up first.

There are three symbolic execution backends in SAW; they all wrap different languages around the Crucible symbolic execution engine. They handle LLVM bitcode, Java byte code, and Rust's MIR intermediate representation. (There is also a fourth experimental backend, interconnected with the LLVM backend, that handles raw x86_64 binaries.)

Each of these has its own collection of SAWScript builtins; the three sets of builtins are very similar but not quite identical. Similarly, when using the Python bindings each has its own interface, and the three are very similar but not quite identical.

10.1 Simple Example for Getting Started

> **Warning**
>
> This section is under construction!

10.2 Languages for Specifications

Each backend also has its own specification language. This is necessary because all three backends execute code in a wider world than Cryptol and SAWCore. SAWCore is a pure lambda calculus;

Cryptol is largely restricted to finite objects. LLVM, Java, and Rust all operate in a world that has pointers, memory allocation, mutable state, global variables, partial functions where some combinations of inputs may be invalid, and other real-world phenomena. All three of them also (and not uncoincidentally) have a broader set of types than either Cryptol or SAWCore, such as pointers.

Consequently, each backend has its own language that includes these types and their values. This is the language that specifications for code in that backend must be written in, and those specifications must typecheck in that language. In each case it is, like the specification languages for other production language verification frameworks, akin to but not quite the same as a pure subset of the original source language.

Unfortunately, at least for the time being, the backend specification languages are entirely implicit. The types and values of these languages appear as SAWScript program objects, but they have no concrete syntax.

Instead, types and values are assembled programmatically, and then assertions and specifications are assembled programmatically out of those. Values can be lifted out of Cryptol (and/or SAWCore) as needed. In the typical case of verifying code against a Cryptol model, the specification works by lifting the results of Cryptol-level computations into the specification language and relating them to the results produced by the target code.

There is thus an implicit correspondence between Cryptol types and certain types in each backend, and SAWScript operations that lift Cryptol values to specification-language values according to this correspondence.

The SAWScript type of LLVM type objects in the LLVM backend is `LLVMType`; the SAWScript type of LLVM value objects is `LLVMValue`. The function `llvm_term` lifts a Cryptol-level value of SAWScript type `Term` to an LLVM value object of type `LLVMValue`.

Similarly, in the JVM backend, types appear as `JavaType`, values as `JVMValue`, and the value-lifting function is `jvm_term`. In the MIR backend, types are `MIRType`, values are `MIRValue`, and the lifting function is `mir_term`.

Users frequently find this embedding, these functions, and the sometimes apparently arbitrary need for their use confusing. It is important to build a robust mental model of the multiple layers involved and how the mapping between them works. Cryptol values can be lifted to LLVM, JVM, or MIR values, and SAWScript is its own layer above both.

Rather than diving directly into the implicit backend languages (including the details of those mappings) without context, we'll first look in a general way at how specifications for code work.

10.3 Specifications

Given the above, it should be clear that in general the specification for an LLVM, JVM, or MIR function is more involved than just naming a corresponding Cryptol function, even if that Cryptol function is a complete behavioral specification.

SAW specifications for these backends (sometimes, somewhat inaccurately, also called "Crucible

specifications") each set up a call to the target function. This involves creating symbolic variables, defining the setup needed to make a call (called the *prestate*), providing (generally symbolic) arguments to the function, and then defining the expected results (called the *poststate*).

Defining the prestate and poststate can each be subdivided into two parts: first, allocating memory, and second, making assertions about symbolic variables and the contents of memory (called *preconditions* and *postconditions* respectively). The prestate and poststate logic are divided by the argument specification. This can be thought of as the declaration of the function arguments; it can also be thought of as the point in the specification where the target function gets invoked. There is a specific builtin for this in each back end; for example, in the LLVM backend it is `llvm_execute_func`. There must be exactly one call to this builtin in each specification.

Each backend provides a set of operations for creating symbolic (also called *fresh*) variables of various types. In general you want to prove that your function behaves a certain way for all valid inputs; the premise of verification via symbolic execution is that we can execute on inputs that range over many possible values.

Generally, for base types symbolic values can be generated at the Cryptol level and then lifted into the code specification layer. For complex types that have no Cryptol analogues, symbolic values can be created either by constructing specification-level values from symbolic base values (so for example, one might create a struct with two integer fields from two symbolic integer values lifted into the specification layer) or with convenience functions that create complex values and fill them with fresh values.

While in general fresh variables are created at the beginning of a specification (because these would be the quantifier-bound variables if the specification were written declaratively as a theorem) it is also possible to create new ones in the poststate logic. These are treated as outputs; essentially they're existentially bound in the output part of the specification.

Pointers and references (other than null or uninitialized pointers, in backends where they exist) must be created by explicitly allocating memory. Allocations done in the prestate logic are presumed to have been done before the function call, and the existence of those allocations is part of the overall precondition. Allocations done in the poststate logic are presumed to be done *by* the function call, and the resulting pointers are treated as outputs.

Each distinct memory region must be allocated separately, and separately allocated memory regions are (by default, with some exceptions) presumed to be distinct.

In situations where a function takes multiple pointer arguments that might or might not alias, it is usually necessary to write multiple specifications to cover the different cases.

Assertions relate symbolic variables to other symbolic variables, or to constants, or to computations from a Cryptol model. Assertions in the prestate logic create obligations that must be satisfied to make the call to the function; the function's code is thus entitled to assume they hold. Assertions in the poststate logic create obligations that the function must satisfy.

The return value of the function (if any) is specified in the poststate as a special kind of assertion.

This is all very abstract, so as an example, consider the following somewhat silly C code:

```
#include <stdlib.h>

unsigned *add(unsigned *x, unsigned *y) {
   unsigned *ret;

   ret = malloc(sizeof(*ret));
   *ret = *x + *y;
   return ret;
}
```

The function `add` takes two pointers to integers, and returns a pointer to a newly allocated integer that holds the sum. We use unsigned integers to avoid having to worry about whether the addition overflows, which would be undefined for signed integers.

We can write the following LLVM specification for it, which has all of the elements discussed above:

```
let add_spec = do {
   xval <- llvm_fresh_var "xval" (llvm_int 32);
   yval <- llvm_fresh_var "yval" (llvm_int 32);

   x <- llvm_alloc (llvm_int 32);
   y <- llvm_alloc (llvm_int 32);

   llvm_points_to x (llvm_term xval);
   llvm_points_to y (llvm_term yval);

   llvm_execute_func [x, y];

   ret <- llvm_alloc (llvm_int 32);

   llvm_points_to ret (llvm_term {{ xval + yval }});
   llvm_return ret;
};
```

The specification is always a SAWScript do-block, which will be executed top-to-bottom when we run the verification.

First we create two symbolic variables `xval` and `yval` to hold the values pointed to by the pointers `x` and `y`. These are 32-bit LLVM integers; however, `llvm_fresh_var` creates them at the Cryptol level and the SAWScript `xval` and `yval` variables have type `Term`. (The `llvm` in the name indicates that it works in LLVM specifications and with LLVM-level *types*.)

Then we allocate memory for the prestate. On entry, we want both pointers to be valid. In this specification we presume they should be disjoint, so we do two separate allocations, one for `x` and one for `y`. This gives us two pointer values.

(This will not match later calls to `add` that pass the same pointer as both arguments. If we want to allow that, we need a second spec that allocates only one pointer and therefore also has only one fresh value. Writing this spec by cutting down the one above should be a fairly straightforward exercise.)

Then we assert the prestate conditions: `x` points to `xval` and `y` points to `yval`. We need to lift `xval` and `yval` to the LLVM level with `llvm_term` because, as noted above, `llvm_fresh_var` created them at the Cryptol level.

Then we call the function and pass `x` and `y` as the arguments. We do not need `llvm_term` here because `x` and `y`, being pointers, are already LLVM-level values.

The rest of the spec is poststate logic: first we allocate another pointer for the return value (matching the allocation within the function). Then we assert that it points to, not another fresh variable but the sum of the existing ones, whatever that is. We can write that sum in a Cryptol block that refers to `xval` and `yval` precisely because those are Cryptol-level variables with Cryptol types. We wrap the Cryptol block in `llvm_term` because the Cryptol block is, unsurprisingly, a Cryptol-level value and needs to be lifted into the LLVM world.

The final line specifies that the return value of the function is the pointer `ret`. This is a special kind of poststate assertion. (You must provide a return value if the function returns a value. You must not if it doesn't. C and Java functions of return type `void` do not. Rust functions of return type `()` can be construed either way: you can specify that the return value is `()`, but you need not.) If you don't want to specify the return value exactly, you can create a fresh value for it in the poststate logic and return that, optionally restricting its value to a subset of possible values if desired.

We again don't need `llvm_term` with `ret` because it's an LLVM-level value.

In order to actually verify this function we also need to compile the C code:

```
clang -g -c -emit-llvm example.c -o example.bc
```

and we need two additional lines along with the specification itself in a SAWScript file:

```
bc <- llvm_load_module "example.bc";
llvm_verify bc "add" [] true add_spec z3;
```

This loads the compiler output, giving us an LLVM module handle `bc`, and then runs the verification by calling `llvm_verify`.

(Note that `jvm_verify` and `mir_verify` are essentially the same, except that they work with JVM and MIR code modules and specs.)

Running SAW on the resulting file gives the following output:

```
Loading file "example.saw"
Verifying add...
Simulating add...
Checking proof obligations add...
Proof succeeded! add
```

The arguments to `llvm_verify` are, first, the LLVM module handle; the name of the function; a list of what we call *override* specifications (more on that in a moment), a flag that says whether to enable path satisfiability checking (leave this set to true for now), the specification, and a proof script.

10.3.1 Overrides and Compositional Verification

`llvm_verify` returns a SAWScript value of type `LLVMSpec`. This represents a proved/verified specification. Here we ignore it; in a larger development you would likely capture it for later use.

This function does not call any other functions. However, imagine a function that calls `add`. When verifying *that* function, we could let the symbolic execution engine execute directly into `add`, or we could provide the specification for `add` that we just proved and have it apply that instead. Such specifications are called *override* specifications, because they override the direct execution of the function specified.

In the case of `add`, using the proved specification to override later executions probably is no faster than just executing the function directly, and might even be slower. However, for functions that do a lot of work, applying the proved specification will require a lot less computation than, essentially, redoing it every time. The use of override specifications is critical for verifying nontrivial code. This is called *compositional verification* because it allows composing specifications for different chunks of code.

If you want to use a proved specification as an override when calling `llvm_verify` (or `jvm_verify` or `mir_verify`), you can put it in the list that's empty in the example above.

It is also possible to write specifications and assume that they're valid instead of prove them, and then use them as overrides. This is useful for library code, where you might not have the source or where verifying the library is a whole additional project. The LLVM function for this is `llvm_unsafe_assume_spec`; it is akin to `llvm_verify` but takes only the LLVM module handle, function name, and spec. (There are corresponding `jvm_unsafe_assume_spec` and `mir_unsafe_assume_spec` functions as well.)

(One might instead think to provide a proof script that just admits all goals given to it. However, that will still run the symbolic execution, which still requires most of the specification setup and can still fail, so in practice it isn't useful.)

10.3.2 Additional Restrictions on Arrays

In all three backends the type construction for arrays requires a concrete SAWScript integer. Proof results are thus dependent on the array size being as indicated, and may not hold for other sizes.

Note however that it is intended to be possible to pass an array of length M to a function that expects a length N, where $M > N$, such that the function will just ignore the extra space.

In cases where this is not sufficient, it is also possible to make the specification a function parameterized by the length N and then prove it repeatedly for different sizes. See *the SAWScript programming chapter*.

10.3.3 Summary

A specification is a SAWScript do-block divided into two parts by the call to `llvm_execute_func`, `jvm_execute_func`, or `mir_execute_func`, which provides the arguments to the function being verified. The first half of the block contains prestate logic for creating fresh variables, allocating memory, and setting up the allowable start states. The second half of the block contains poststate logic for (potentially) creating more fresh variables, allocating more memory, and specifying the allowable output states.

We can now go into each backend's implicit specification language and its driving SAWScript builtins in more detail. Each backend has its own types and values, and its own SAWScript functions. However, there is a good deal of commonality across the backends.

10.4 The LLVM Backend and LLVM Specifications

In the LLVM backend, types appear in SAWScript as values of SAWScript type `LLVMType`, and values have the SAWScript type `LLVMValue`. LLVM code modules have the type `LLVMModule`, and the specification do-blocks have the type `LLVMSetup ()`.

The function argument declaration that divides the pre- and post- portions of a specification is done with `llvm_execute_func`.

10.4.1 LLVM Types

The following SAWScript functions construct LLVM types:

- `llvm_int` *n* produces an *n*-bit-wide machine integer/bitvector. Like Cryptol, SAW doesn't have separate signed and unsigned bitvector types. The `llvm_int` types correspond to the Cryptol bitvector types `[`*n*`]`. Lifting a Cryptol bitvector value to an LLVM value produces a value of this type.
- `llvm_float` is the type of (32-bit) single-precision floating point numbers, and `llvm_double` is the type of (64-bit) double-precision floating point numbers. These correspond to the Cryptol floating point types of the proper sizes. However, SAW's support for floating point is rudimentary so these types are currently of little use.
- `llvm_array` *n ty* produces an array type of element type *ty* that is *n* entries long. This type corresponds to the Cryptol sequence type `[`*n*`][`*cty*`]` where *cty* is the Cryptol type corresponding to *ty*, if it exists. Lifting a Cryptol sequence value to an LLVM value produces a value of this type. (Note that Cryptol bitvectors are also sequences. A Cryptol sequence of `Bit` / `Bool` is a bitvector and lifts to `llvm_int` of some size. A Cryptol sequence of some other type lifts to an `llvm_array`.)
- `llvm_struct_type` *tys* produces a struct type whose elements have types *tys*. *tys* is a list of `LLVMType`. This corresponds to a Cryptol tuple type containing the Cryptol types corresponding to *tys*, if they exist.

- `llvm_packed_struct_type` *`tys`* is like `llvm_struct_type` but produces a packed struct type. (That is, one that has no alignment padding between members.) This also corresponds to a Cryptol tuple type containing the Cryptol types corresponding to *`tys`*, if they exist.
- `llvm_pointer` *`ty`* is the type of a pointer to *`ty`*. This has no Cryptol equivalent.

Furthermore,

- `llvm_type` takes a string in LLVM's type syntax and produces the corresponding `LLVMType`. For example, `llvm_type "i32"` produces the same type as `llvm_int 32`.
- `llvm_alias` takes a string that's the name of a type alias and looks up the `LLVMType` for it.

When your LLVM module is the result of compiling C code, all ordinary C integer types map to the `llvm_int` type of the proper size.

The C `bool` type may appear as either `llvm_int 1` or `llvm_int 8`, depending on context: loose `bool`s are generally `llvm_int 1`, but members of structs and arrays will usually be `llvm_int 8`. This is due to a peculiarity in the way Clang compiles `bool` down to LLVM. When in doubt about how a `bool` is represented, check the LLVM bitcode by compiling your code with `clang -S -emit-llvm`.

10.4.2 LLVM Values

The primary way to produce LLVM values of base type is to lift Cryptol values with `llvm_term`. For example, `llvm_term {{ 123 : [32] }}` produces an LLVM value whose type is `llvm_int 32`.

LLVM values of compound type can be constructed with these functions:

- `llvm_array_value` *`vals`* constructs an LLVM array value; *`vals`* is a list of values to put in the array.
- `llvm_struct_value` *`vals`* constructs an LLVM struct value; *`vals`* is a list of values to put in the struct, in order.
- `llvm_packed_struct_value` *`vals`* is like `llvm_struct_value` but produces a packed struct.

The elements of LLVM values of compound type can be extracted with these functions:

- `llvm_elem` *`val n`* extracts element *`n`* of the LLVM array or struct (or packed struct) *`val`*. However, it only works on pointers: *`val`* should be a pointer that points at an array or struct, and the returned value is a pointer to the element. There is no way to extract elements from an array or struct value directly.
- `llvm_field` *`val f`* extracts the named field *`f`* of a struct or packed struct *`val`*. Like `llvm_elem`, it only works on pointers. The LLVM module that defines the type must have been compiled with debugging information. Sometimes also the debugging information is insufficient to identify the correct type, in which case one should fall back to `llvm_elem`.

- `llvm_union` *`val f`* is like `llvm_field` except that it operates on a union. It too only works on pointers, and the LLVM module that defines the type must likewise have been compiled with debugging information. Note that because the address of a union is the same as the address of all its members, this function only changes the type of the pointer. One can use `llvm_cast_pointer` instead. However, `llvm_union` avoids baking in the identity of the type of the union field, which is both convenient and improves robustness.

10.4.3 Binding Quantified Variables in LLVM

The basic function for this in the LLVM backend is `llvm_fresh_var _x_` *`ty`*. *`x`* provides an internal name for the variable; *`ty`* names its type. This type is an LLVM type (a value of type `LLVMType`) rather than a Cryptol type. However, it must be an LLVM type that has a corresponding Cryptol type, because the result is a Cryptol-level SAWScript value. (Thus, it has type `Term` and not type `LLVMValue`.)

For example:

```
x <- llvm_fresh_var "x" (llvm_int 32);
```

This produces a fresh 32-bit machine integer.

There is also an experimental function `llvm_fresh_cryptol_var` that takes a Cryptol type instead. Both of these functions produce Cryptol-level fresh values.

It is reasonable to use the same identifier for the internal name and for the SAWScript variable, and, where applicable, to make it the same name as used for the same thing in the code under verification.

The function `llvm_fresh_expanded_val` *`ty`* creates a fresh value of LLVM-level type (thus, the type argument is an `LLVMType`, and the result is an `LLVMValue`) and populates it recursively with fresh variables. This can be used for struct and array types.

In general for values of compound type it is possible to create fresh primitive values for the elements and create compound values with functions like `llvm_array_value`; it is also possible to use `llvm_fresh_expanded_val` and reason about the contents by using functions like `llvm_elem`. Which is preferable is in most cases purely a matter of convenience.

10.4.4 Allocation and Pointers in the LLVM Backend

LLVM pointer values, other than null and uninitialized ones, cannot be arbitrarily created; one must in general explicitly allocate memory to create a pointer. The pointer can then be passed as an argument to the function under verification, or used to initialize other values. Pointers allocated in the prestate logic are presumed to exist before the function is called; pointers allocated (in the `llvm_alloc` sense) in the poststate logic are expected to be allocated (in the `malloc` runtime sense) by the function.

There are two chief exceptions to pointers needing to be allocated:

- `llvm_null` is a null pointer.

- `llvm_fresh_pointer` creates a pointer that does not point to anything. Accessing it is an error. This provides a useful simplification for functions that handle, but do not dereference, pointers.

In the LLVM backend the primary allocation function is `llvm_alloc`. It takes an LLVM-level type (a value of type `LLVMType`), allocates memory to hold a value of that type, and produces an `LLVMValue` representing the pointer.

Note that allocating memory and reasoning about its contents are separate steps. In most cases each `llvm_alloc` should be paired with an `llvm_points_to` assertion (see below) that says something about the value found in the allocated memory. (Otherwise the memory is treated as uninitialized, and attempts to read from it will be treated as an error. Sometimes this is what you want; for example, a pointer argument whose purpose is to return a complex value should not be read from before it's written to, and thus leaving its contents uninitialized allows detecting such mistakes.)

There are multiple variants of `llvm_alloc`:

- `llvm_alloc_aligned` takes an extra integer argument *n* that's a power of 2, and allocates memory that's guaranteed to be aligned to an *n*-byte boundary.
- `llvm_alloc_readonly` creates a pointer that cannot be used for writing, like `const` pointers in C. The aliasing restrictions for allocations are relaxed for regions allocated with this function: read-only regions are allowed to alias one another.
- `llvm_alloc_readonly_aligned` is like both `llvm_alloc_aligned` and `llvm_alloc_readonly` at once.
- `llvm_alloc_with_size` takes an extra integer argument, which must be greater than the size of the type, and allocates that much memory. This is useful for dealing with types that are headers for memory blocks.
- `llvm_alloc_sym_init` is like `llvm_alloc` but also implicitly fills the allocated memory with fresh byte values.
- `llvm_symbolic_alloc` takes three arguments: a boolean, which if true makes the allocation read-only like `llvm_alloc_readonly`, a power-of-two integer, which specifies the alignment like in `llvm_alloc_aligned`, and instead of a type the third argument is a size in bytes.

There is also a function `llvm_cast_pointer` that takes a pointer value and an LLVM type, and yields a new pointer value with an updated pointed-to type. This is mostly useful for accessing pointers to unions in cases where `llvm_union` isn't available. Unsafe or undefined accesses constructed via pointer casts will fail.

10.4.5 LLVM Global Variables

The LLVM world includes global (including file-static) variables. Some of these get put in the data segment are mutable, and others get put in read-only data and cannot be changed at runtime. (Typically, globals declared in C with `const` end up in read-only data, but there are some corner cases

where that isn't true. If in doubt, check in the LLVM bitcode whether the variable is marked `constant`, or check the section in the output object file.)

By default, immutable global variables are always available for use, and mutable globals that a function doesn't touch can be ignored. However, each mutable global variable used by a function under verification must be explicitly enabled in its specification.

This requirement arises because using a mutable global variable creates an obligation to reason about its state. Explicitly enabling mutable globals allows SAW to easily check that all variables used are enabled, and all variables enabled have been covered in both the pre-state and post-state assertions.

The function `llvm_alloc_global` *`name`* enables the mutable global *`name`*. It returns nothing (not a pointer). (The name arises because under the covers it does actually allocate: it sets up the data-segment memory for the variable.)

To refer to the global, use the function `llvm_global` *`name`*; this returns a pointer to it.

If you want the initializer value for a global, you can get it with `llvm_global_initializer` *`name`*. Sometimes this will be the prestate value you want, more often not.

For example, consider this code:

```
int x = 0;

int f(int y) {
  x = x + 1;
  return x + y;
}

int g(int z) {
  x = x + 2;
  return x + z;
}
```

The following specifications are rejected because they do not allocate the global `x`. (And, even if they did, they provide neither a prestate value nor a poststate value for it.)

```
m <- llvm_load_module "./test.bc";

f_spec <- llvm_verify m "f" [] true (do {
    y <- llvm_fresh_var "y" (llvm_int 32);
    llvm_execute_func [llvm_term y];
    llvm_return (llvm_term {{ 1 + y : [32] }});
}) abc;

g_spec <- llvm_verify m "g" [] true (do {
    z <- llvm_fresh_var "z" (llvm_int 32);
```

(continues on next page)

(continued from previous page)

```
    llvm_execute_func [llvm_term z];
    llvm_return (llvm_term {{ 2 + z : [32] }});
}) abc;
```

If we want to assume that `f` will only be called once and never have `g` written before it, we could write this spec:

```
m <- llvm_load_module "./test.bc";


let init_global name = do {
  llvm_alloc_global name;
  llvm_points_to (llvm_global name)
                 (llvm_global_initializer name);
};

f_spec <- llvm_verify m "f" [] true (do {
    y <- llvm_fresh_var "y" (llvm_int 32);
    init_global "x";
    llvm_precond {{ y < 2^^31 - 1 }};
    llvm_execute_func [llvm_term y];
    llvm_return (llvm_term {{ 1 + y : [32] }});
}) abc;
```

which initializes `x` to whatever it is initialized to in the C code at the beginning of verification. (It also includes a constraint on the argument `y` to avoid undefined behavior arising from signed integer overflow.) However, it does not specify an output value for `x` so it cannot be used compositionally.

We could also write this more complete spec:

```
m <- llvm_load_module "test-signed.bc";

f_spec <- llvm_verify m "f" [] true (do {
    x <- llvm_fresh_var "x" (llvm_int 32);
    y <- llvm_fresh_var "y" (llvm_int 32);
    llvm_alloc_global "x";
    llvm_points_to (llvm_global "x") (llvm_term x);
    llvm_precond {{ x + y < 2^^31 - 1 }};
    llvm_execute_func [llvm_term y];
    llvm_points_to (llvm_global "x") (llvm_term {{ 1 + x : [32] }});
    llvm_return (llvm_term {{ 1 + x + y : [32] }});
}) abc;
```

which allows `x` to have any value on entry and includes a specification of its resulting value afterwards.

One can also restrict the starting value of `x` if, for example, the point of having it there is to keep track of how many calls to `f` and `g` there have been and it's not supposed to exceed some limit. Then later uses of these functions will need to require that the limit has not yet been exceeded.

There is a fly in this ointment, however, which is that it is possible for immutable globals to depend on mutable globals. This does not work in SAW's model. In particular, an immutable global initialized to the address of a mutable global will not work correctly.

```
int thingy_store;
int *const thingy = &thingy_store;
```

Here `thingy` is a immutable pointer (what it points to cannot be changed) that points to a mutable global. The internal initialization of `thingy` will fail before the specification can enable it with `llvm_alloc`.

To work around this problem, SAW has an experimental global allocation mode. After using `enable_experimental`, you can use one of these three `TopLevel` functions to choose, before calling `llvm_verify`, how LLVM globals are allocated:

- `llvm_alloc_constant_globals` specifies the default behavior, namely that immutable globals are automatically allocated and mutable globals are not. This is the safest mode and should be used unless one of the others is needed.
- `llvm_alloc_all_globals` specifies that *all* globals are to be automatically allocated, regardless of mutability. This will ensure that examples like the one above can be handled, but at the cost of expecting that the specification provide both pre-state and post-state for *all* mutable globals. This is not fully checked and unsafe in general. See *Compositional Verification and Mutable Global Variables* for discussion.
- `llvm_alloc_no_globals` disables all automatic initialization of globals. All allocation must be handled explicitly by the `LLVMSetup` block. This is in turn unsafe in the sense that immutable globals can be given arbitrary (and unrealizable) prestate values other than their actual initialization value, which then can lead to apparently successful verifications not grounded in reality. Always use `llvm_global_initializer` to generate the prestate values for immutable globals. For example:

  ```
  llvm_points_to (llvm_global "thingy") (llvm_global_initializer
  ↪"thingy");
  ```

Note that the value for a pointer like `thingy` in the above example, but that is *not* immutable (for example, might be set at program startup to point to one of several possible globals), can be specified with an assertion like the following:

```
llvm_points_to (llvm_global "thingy") (llvm_global "thingy_store");
```

10.4.6 Assertions in the LLVM Backend

The basic function for LLVM assertions is `llvm_assert`. It takes one argument, which is a `Term` (Cryptol-level value); frequently that term will be a Cryptol block written with double-braces. If in the prestate logic, it generates a precondition; in the poststate logic, it generates a postcondition.

The function `llvm_precond` is equivalent, except it is only allowed in the prestate logic. Similarly, `llvm_postcond` is only allowed in the poststate logic.

The function `llvm_equal` is comparable to using `llvm_assert` with an equality; it takes two arguments and asserts that they are equal. However, the arguments it takes are LLVM-level values (`LLVMValue`) rather than Cryptol-level values (`Term`), so it can express equalities that would be messy with `llvm_assert`. The use of `llvm_equal` when applicable can also sometimes lead to more efficient symbolic execution.

The function `llvm_return` *`val`* asserts that the value returned by the function is *`val`*. This is only allowed in the poststate logic.

The function `llvm_points_to` is the basic way to assert about a pointer. It takes two arguments that are both LLVM-level values; the first is the pointer and the second is the contents.

There are some variants of `llvm_points_to`:

- `llvm_conditional_points_to` *`cond ptr val`* asserts that the pointer *`ptr`* points to the value *`val`*, but only when *`cond`* is true. *`cond`* is a Cryptol-level value of type `Term` that can be symbolic.
- `llvm_points_to_at_type` *`ptr ty val`* casts the pointer *`ptr`* to point at *`ty`* before examining *`val`*. This can be used, for example, to read or write a prefix of an array. It can also be used in conjunction with C code that reinterprets the memory representation of one type in terms of another. One can also use `llvm_cast_pointer` instead.
- `llvm_conditional_points_to_at_type` *`cond ptr ty val`* is both the previous functions rolled together.
- `llvm_points_to_untyped` *`ptr val`* and `llvm_conditional_points_to_untyped` *`cond ptr val`* disable typechecking of the pointer and value entirely.
- `llvm_points_to_array_prefix` *`ptr arr n`* indicates that the pointer *`ptr`* points to the first *`n`* elements of *`arr`*. This is experimental.

10.4.7 LLVM Bitfields

SAW has experimental support for specifying `structs` with bitfields. (The material below requires using `enable_experimental`.)

Consider the following example:

```
struct s {
  uint8_t x:1;
```

(continues on next page)

(continued from previous page)

```
  uint8_t y:1;
};
```

Normally, a `struct` with two `uint8_t` fields would have an overall size of two bytes. However, because the `x` and `y` fields are declared with bitfield syntax, they are instead packed together into a single byte.

Because bitfields have somewhat unusual memory representations in LLVM, some special care is required to write SAW specifications involving bitfields. For this reason, there is a dedicated `llvm_points_to_bitfield` function for this purpose:

- `llvm_points_to_bitfield : LLVMValue -> String -> LLVMValue -> LLVMSetup ()`

The type of `llvm_points_to_bitfield` is similar that of `llvm_points_to`, except that it takes the name of a field within a bitfield as an additional argument. For example, here is how to assert that the `y` field in the `struct` example above should be `0`:

```
ss <- llvm_alloc (llvm_alias "struct.s");
llvm_points_to_bitfield ss "y" (llvm_term {{ 0 : [1] }});
```

Note that the type of the right-hand side value (`0`, in this example) must be a bitvector whose length is equal to the size of the field within the bitfield. In this example, the `y` field was declared as `y:1`, so `y`'s value must be of type `[1]`.

Note that the following specification is *not* equivalent to the one above:

```
ss <- llvm_alloc (llvm_alias "struct.s");
llvm_points_to (llvm_field ss "y") (llvm_term {{ 0 : [1] }});
```

`llvm_points_to` works quite differently from `llvm_points_to_bitfield` under the hood, so using `llvm_points_to` on bitfields will almost certainly not work as expected.

In order to use `llvm_points_to_bitfield`, one must also use the `enable_lax_loads_and_stores` command:

- `enable_lax_loads_and_stores: TopLevel ()`

The `enable_lax_loads_and_stores` command relaxes some of SAW's assumptions about uninitialized memory, which is necessary to make `llvm_points_to_bitfield` work under the hood. For example, reading from uninitialized memory normally results in an error in SAW, but with `enable_lax_loads_and_stores`, such a read will instead return a symbolic value. At present, `enable_lax_loads_and_stores` only works with What4-based tactics (e.g., `w4_unint_z3`); using it with SBV-based tactics (e.g., `sbv_unint_z3`) will result in an error.

Note that SAW relies on LLVM debug metadata in order to determine which struct fields reside within a bitfield. As a result, you must pass `-g` to `clang` when compiling code involving bitfields in order for SAW to be able to reason about them.

10.4.8 Loading LLVM Modules

To load LLVM code, simply provide the location of a valid bitcode file to the `llvm_load_module` function.

- `llvm_load_module : String -> TopLevel LLVMModule`

The resulting module handle can be passed to `llvm_verify` and similar functions.

The LLVM bitcode parser should generally work with LLVM versions between 3.5 and 21, though it may be incomplete for some versions. Debug metadata has changed somewhat throughout that version range, so is the most likely case of incompleteness. We aim to support every version after 3.5, however, so report any parsing failures as issues on GitHub (https://github.com/GaloisInc/saw-script/issues).

Tips on Compiling LLVM

For generating LLVM with `clang`, it can be helpful to:

- Turn on debugging symbols with `-g` so that SAW can find source locations of functions, names of variables, etc.
- Optimize with `-O1` so that the generated bitcode more closely matches the C/C++ source, making the results more comprehensible.
- Use `-fno-threadsafe-statics` to prevent `clang` from emitting unnecessary pthread code.
- Link all relevant bitcode with `llvm-link` (including, e.g., the C++ standard library when analyzing C++ code).

All SAW proofs include side conditions to rule out undefined behavior, and proofs will only succeed if all of these side conditions have been discharged. However the default SAW notion of undefined behavior is with respect to the semantics of LLVM, rather than C or C++. If you want to rule out undefined behavior according to the C or C++ standards, consider compiling your code with `-fsanitize=undefined` or one of the related flags[1] to `clang`.

Generally, you'll also want to use `-fsanitize-trap=undefined`, or one of the related flags, to cause the compiled code to use `llvm.trap` to indicate the presence of undefined behavior. Otherwise, the compiled code will call a separate function, such as `__ubsan_handle_shift_out_of_bounds`, for each type of undefined behavior, and SAW currently does not have built in support for these functions (though you could manually create overrides for them in a verification script).

[1] https://clang.llvm.org/docs/UsersManual.html#controlling-code-generation

Working With C++

The distance between C++ code and LLVM is greater than between C and LLVM, so some additional considerations come into play when analyzing C++ code with SAW.

The first key issue is that the C++ standard library is large and complex, and tends to be widely used by C++ applications. To analyze most C++ code, it will be necessary to link your code with a version of the `libc++` library[2] compiled to LLVM bitcode. The `wllvm` program[3] can be useful for this.

The C++ standard library includes a number of key global variables, and any code that touches them will require that they be initialized using `llvm_alloc_global`.

Many C++ names are slightly awkward to deal with in SAW. They may be mangled relative to the text that appears in the C++ source code. SAW currently only understands the mangled names. The `llvm-nm` program can be used to show the list of symbols in an LLVM bitcode file, and the `c++filt` program can be used to demangle them, which can help in identifying the symbol you want to refer to. In addition, C++ names from namespaces can sometimes include quote marks in their LLVM encoding. For example:

```
%"class.quux::Foo" = type { i32, i32 }
```

This can be mentioned in SAW by saying:

```
llvm_type "%\"class.quux::Foo\""
```

Finally, there is no support for calling constructors in specifications, so you will need to construct objects piece-by-piece using, e.g., `llvm_alloc` and `llvm_points_to`.

Multiple LLVM Bitcode Files

If you have multiple bitcode modules, you can join them together before loading using the `llvm-link` utility.

Alternatively, you can load them each independently with a separate `llvm_load_module` command and then combine them with the `llvm_combine_modules` function.

`llvm_combine_modules` *`main others`* links the modules in *`others`* to the module *`main`* and returns (in `TopLevel`) a new LLVM module handle. It is by no means a full-blown linker implementation and should only be used with groups of modules that were meant to be linked together. The modules should be listed in order from caller to callee, the same as on the command line of a Unix linker. If duplicate names appear, names in earlier modules will shadow names in later modules.

[2] https://libcxx.llvm.org/index.html
[3] https://github.com/travitch/whole-program-llvm

10.4.9 LLVM Verification

Verification of LLVM is done with the `llvm_verify` command.

```
llvm_verify :
  LLVMModule ->
  String ->
  [LLVMSpec] ->
  Bool ->
  LLVMSetup () ->
  ProofScript () ->
  TopLevel LLVMSpec
```

The first two arguments specify the module and function name to verify. The third argument specifies the list of already-verified specifications to use as overrides for compositional verification. The fourth argument specifies whether to do path satisfiability checking, and the fifth gives the specification of the function to be verified. Finally, the last argument gives the proof script to use for verification. The result is a proved specification that can be used to simplify verification of functions that call this one.

10.5 The JVM Backend and JVM Specifications

In the JVM backend, types appear as values of SAWScript type `JavaType` (not `JVMType`, by historical accident), and values have the type `JVMValue`. JVM code modules are class files and thus have the type `JavaClass`. JVM specification do-blocks have the type `JVMSetup ()`.

The function argument declaration that divides the pre- and post- portions of a specification is done with `jvm_execute_func`.

10.5.1 JVM Types

JVM types appear in SAWScript as values of type `JVMType`. The following SAWScript functions construct JVM types:

- `java_bool` is the Java `bool` type. This corresponds to the Cryptol `Bit` or `Bool` type. Lifting a Cryptol `Bit` value with `jvm_term` produces a Java `bool`.
- `java_byte` is the Java `byte` type, which is an 8-bit bitvector. This corresponds to the Cryptol `[8]` type.
- `java_char` is the Java `char` type, which is a 16-bit bitvector. (Recall that Java is stuck with UTF-16 strings for legacy reasons.) This corresponds to the Cryptol `[16]` type.
- `java_short` is the Java `short` type, which is also a 16-bit bitvector. This corresponds to the Cryptol `[16]` type.
- `java_int` is the Java `int` type, which is a 32-bit bitvector. This corresponds to the Cryptol `[32]` type.

- `java_long` is the Java `long` type, which is a 64-bit bitvector. This corresponds to the Cryptol `[64]` type.
- `java_float` and `java_double` are the Java `float` and `double` types, which are 32-bit and 64-bit floating point numbers respectively. These correspond to the Cryptol floating point types of the proper size. However, SAW's support for floating point is rudimentary so these types are currently of little use.
- `java_array` *`n ty`* is an array of length *`n`* and element type *`ty`*. This type corresponds to the Cryptol sequence type `[`*`n`*`][`*`cty`*`]` where *`cty`* is the Cryptol type corresponding to *`ty`*, assuming there is one. Lifting a Cryptol sequence value to a JVM value produces a value of this type. (Note that Cryptol bitvectors are also sequences. A Cryptol sequence of `Bit` / `Bool` is a bitvector and lifts to a Java integer type if one of the right size exists. Otherwise it fails. A Cryptol sequence of some other type lifts to an `java_array`.)
- `java_class` *`cl`* is the type of the Java class called *`cl`*. This does not correspond to any Cryptol type.

It is not possible to generate a Java array of `bool` with `jvm_term` and a Cryptol value; a Cryptol sequence of `Bit` will produce a machine integer type instead. As a workaround one can allocate a fresh array with `jvm_alloc_array` and assert about its elements individually. Allocation is discussed below in the next section.

10.5.2 JVM Values and Allocation

The primary way to produce JVM values of base type is to lift Cryptol values with `jvm_term`. For example, `jvm_term {{ 123 : [32] }}` produces a JVM value whose type is `java_int`.

All compound values in Java are instances or arrays, and all class instance or array values are pointers/references. This allows a simpler interface than in the LLVM and MIR backends: instances and arrays are allocated, and then one can assert directly about their fields and elements. There is no need to materialize instance or array values directly.

- `jvm_alloc_object` *`cl`* allocates an object instance of type *`cl`*.
- `jvm_alloc_array` *`n ty`* allocates an array of *`n`* elements of type *`ty`*.

These functions return `JVMValue` values whose JVM-level type is `java_class` *`cl`* and `java_array` *`n ty`* respectively. As in the Java language, these are implicitly pointers/references, and there are no explicit pointers or references needed.

Instances or arrays allocated in the prestate logic are presumed to exist before the function is called; those allocated (in the `jvm_alloc_object` sense) in the poststate logic are expected to be allocated (in the `new` runtime sense) by the function.

The Java null pointer is called `jvm_null`.

10.5.3 Binding Quantified JVM Values

The basic function in the JVM backend is `jvm_fresh_var` *`x`* *`ty`*. *`x`* is a string, which provides an internal name for the variable, and *`ty`* is the type to generate. This type is a JVM-level type, not a Cryptol type. However, it must be a JVM type that has a corresponding Cryptol type, because the result is a Cryptol-level value of type `Term`.

For example:

```
x <- jvm_fresh_var "x" java_int;
```

This produces a fresh 32-bit machine integer.

It is reasonable to use the same identifier for the internal name and for the SAWScript variable, and, where applicable, to make it the same name as used for the same thing in the code under verification.

10.5.4 JVM Static Variables

In the JVM backend, static values always exist and can be reasoned about with `jvm_static_field_is`.

There are some unresolved questions about initialization for static values; see issue #916 (https://github.com/GaloisInc/saw-script/issues/916).

10.5.5 JVM Assertions

The basic function for JVM assertions is `jvm_assert`. It takes one argument, which is a `Term` (Cryptol-level value); frequently that term will be a Cryptol block written with double-braces. If in the prestate logic, it generates a precondition; in the poststate logic, it generates a postcondition.

The function `jvm_precond` is equivalent, except it is only allowed in the prestate logic. Similarly, `jvm_postcond` is only allowed in the poststate logic.

The function `jvm_equal` is comparable to using `jvm_assert` with an equality; it takes two arguments and asserts that they are equal. However, the arguments it takes are JVM-level values (`JVMValue`) rather than Cryptol-level values (`Term`), so it can express equalities that would be messy with `jvm_assert`. The use of `jvm_equal` when applicable can also sometimes lead to more efficient symbolic execution.

The function `jvm_return` *`val`* asserts that the value returned by the function is *`val`*. This is only allowed in the poststate logic.

The following functions are used for reasoning about the values of class instance and array elements:

- `jvm_field_is` *`obj`* *`f`* *`val`* asserts that field *`f`* of an object value *`obj`* has value *`val`*. *`val`* is a JVM-level value.
- `jvm_static_field_is` *`f`* *`val`* asserts that the static field *`f`* (which may be qualified with a class name) has value *`val`*. *`val`* is a JVM-level value. If no class name is included, the class of the method being verified is assumed.

- `jvm_elem_is` *`arr n val`* asserts that the element *`n`* of the array *`arr`* has the value *`val`*. *`val`* is a JVM-level value.
- `jvm_array_is` *`arr val`* asserts that the array *`arr`* has the contents *`val`*, which is a *Cryptol*-level value of sequence type.

There are variant versions of each of these that do not take a value; instead they implicitly create a fresh value. These are convenient when the value produced is not specified or indeterminate/unpredictable. They can also be used to write partial specifications when detailed reasoning about the output value isn't currently necessary.

- `jvm_modifies_field` *`obj f`*
- `jvm_modifies_static_field` *`f`*
- `jvm_modifies_elem` *`arr n`*
- `jvm_modifies_array` *`arr`*

10.5.6 Loading JVM Code

Loading Java code is more complex than with the other backends, because of the more structured nature of Java packages. When running `saw`, three command-line options control where to look for classes:

- The `-b` flag takes the path where the `java` executable lives, which is used to locate the Java standard library classes and add them to the class database. Alternatively, one can put the directory where `java` lives *on the `PATH`*, which SAW will search if `-b` is not set.
- The `-j` flag takes the name of a JAR file as an argument and adds the contents of that file to the class database.
- The `-c` flag takes the name of a directory as an argument and adds all class files found in that directory (and its subdirectories) to the class database. By default, the current directory is included in the class path.

Most Java programs will only require setting the `-b` flag (or the `PATH`), as that is enough to bring in the standard Java libraries. Note that when searching the `PATH`, SAW makes assumptions about where the standard library classes live. These assumptions are likely to hold on JDK 7 or later, but they may not hold on older JDKs on certain operating systems. If you are using an old version of the JDK and SAW is unable to find a standard Java class, you may need to specify the location of the standard classes' JAR file with the `-j` flag (or, alternatively, with the `SAW_JDK_JAR` environment variable).

Once the class path is configured, you can pass the name of a class to the `java_load_class` function.

- `java_load_class : String -> TopLevel JavaClass`

The resulting `JavaClass` can be passed to `jvm_verify` and related functions.

Java class files from any JDK newer than version 6 should work. However, support for JDK 9 and later is experimental. Verifying code that only uses primitive data types is known to work well, but there are some as-of-yet unresolved issues in verifying code involving classes such as `String`. For more information on these issues, refer to this GitHub issue (https://github.com/GaloisInc/crucible/issues/641).

For Java, the only compilation flag that tends to be valuable is `-g` to retain information about the names of function arguments and local variables.

10.5.7 JVM Verification

Verification of JVM byte code is done with the `jvm_verify` command.

```
jvm_verify :
  JavaClass ->
  String ->
  [JVMSpec] ->
  Bool ->
  JVMSetup () ->
  ProofScript () ->
  TopLevel JVMSpec
```

The first two arguments specify the class and method name to verify. The third argument specifies the list of already-verified specifications to use as overrides for compositional verification. The fourth argument specifies whether to do path satisfiability checking, and the fifth gives the specification of the function to be verified. Finally, the last argument gives the proof script to use for verification. The result is a proved specification that can be used to simplify verification of functions that call this one.

10.6 The MIR backend and Rust/MIR Specifications

In the MIR backend, types appear as values of SAWScript type `MIRType`. There are also separate handles for Rust algebraic data types, which appear as SAWScript type `MIRAdt`. Values have the type `MIRValue`; code modules have the type `MIRModule`; and specification do-blocks have the type `MIRSetup ()`.

The function argument declaration that divides the pre- and post- portions of a specification is done with `mir_execute_func`.

10.6.1 MIR (Rust) Types

The following SAWScript functions construct MIR types:

- `mir_bool` is the Rust `bool` type. This corresponds to the Cryptol `Bit` or `Bool` type. Lifting a Cryptol `Bit` value with `mir_term` produces a Rust `bool`.
- `mir_char` is the Rust `char` type. It corresponds to the Cryptol `[32]` type.

- `mir_i8` and `mir_u8` are the Rust `i8` and `u8` types respectively. These are 8-bit bitvectors and (both) correspond to the Cryptol `[8]` type.
- `mir_i16` and `mir_u16` are the Rust `i16` and `u16` types respectively. These are 16-bit bitvectors and (both) correspond to the Cryptol `[16]` type.
- `mir_i32` and `mir_u32` are the Rust `i32` and `u32` types respectively. These are 32-bit bitvectors and (both) correspond to the Cryptol `[32]` type.
- `mir_i64` and `mir_u64` are the Rust `i64` and `u64` types respectively. These are 64-bit bitvectors and (both) correspond to the Cryptol `[64]` type.
- `mir_i128` and `mir_u128` are the Rust `i128` and `u128` types respectively. These are 128-bit bitvectors and (both) correspond to the Cryptol `[128]` type.
- `mir_isize` and `mir_usize` are the Rust `isize` and `usize` types respectively. These are pointer-sized bitvectors and, in SAW, both correspond to the Cryptol `[64]` type. For the time being, SAW treats all Rust code as 64-bit.
- `mir_f32` is the type of Rust single-precision floating point values. This corresponds to the Cryptol floating point type of the proper size. However, SAW's support for floating point is rudimentary so this type is of little use currently.
- `mir_f64` is the type of Rust double-precision floating point values. This corresponds to the Cryptol floating point type of the proper size. However, SAW's support for floating point is rudimentary so this type is of little use currently.
- `mir_tuple` *`tys`* is the type of a Rust tuple containing the types *`tys`*, in order. This corresponds to the Cryptol tuple type containing the corresponding Cryptol types, if they exist.
- `mir_array` *`n ty`* is the type of an array of element type *`ty`* that is *`n`* entries long. This type corresponds to the Cryptol sequence type `[`*`n`*`][`*`cty`*`]` where *`cty`* is the Cryptol type corresponding to *`ty`*, if it exists.
- `mir_adt` *`adt`* is the type of a Rust algebraic data type. The *`adt`* argument is a value of type `MIRAdt`, which can be looked up in a loaded MIR module by name. The `MIRAdt` type is a layer of indirection that makes ADT operations on non-ADT types ill-typed in SAWScript, which helps avoid various mistakes. Rust ADTs have no corresponding Cryptol type.
- `mir_ref` *`ty`* is the type of immutable references to *`ty`*. It has no corresponding Cryptol type.
- `mir_ref_mut` *`ty`* is the type of mutable references to *`ty`*. It has no corresponding Cryptol type.
- `mir_raw_pointer_const` *`ty`* is the type of immutable raw pointers to *`ty`*. It has no corresponding Cryptol type.
- `mir_raw_pointer_mut` *`ty`* is the type of mutable raw pointers to *`ty`*. It has no corresponding Cryptol type.
- `mir_slice` *`ty`* is the type of a (reference to a) Rust slice of element type *`ty`*. Currently SAW can only handle references to slices: `&[`*`ty`*`]` in Rust terms. This has no corresponding Cryptol

type.

- `mir_str` is the type of a (reference to a) Rust string slice. Currently SAW can only handle references to strings: `&str` in Rust terms. This has no corresponding Cryptol type.
- `mir_vec` *`mod ty`* is the type of a Rust vector of element type *`ty`*: `Vec<`*`ty`*`>` in Rust terms. *`mod`* is a handle for a MIR code module (see below) that is needed to look up some internal identifiers.

Lifting Cryptol bitvector values produces the corresponding sized Rust/MIR integer types; however, whether it is the signed or unsigned version is unspecified. Typechecking at the MIR/Rust level is relaxed so that the signed and unsigned versions are considered equivalent.

Also, it is not possible to generate a MIR array of `bool` with `mir_term` and a Cryptol value; a Cryptol sequence of `Bit` will produce a machine integer type instead. As a workaround one can create a fresh array value with `mir_fresh_expanded_value` (described below) and assert about its elements individually.

10.6.2 MIR Values

The primary way to produce MIR values of base type is to lift Cryptol values with `mir_term`. For example, `mir_term {{ 123 : [32] }}` produces a MIR value whose type is `mir_i32`.

MIR values of compound type are produced with the following functions:

- `mir_enum_value` *`adt name args`* produces a MIR value whose type is a Rust/MIR enumerated type / algebraic data type. *`adt`* is the type; ADTs can be retrieved using `mir_find_adt`. *`name`* is a string; it is the constructor name for the desired variant. This should be the short form of the name within the type, such as `"None"`; it does not need to be a fully-qualified identifier. *`args`* is a list of MIR-level values to use as the arguments to the data constructor. Note that `mir_enum_value` can only be used to construct a single specific variant. If you need to construct a symbolic enum value that can range over many potential variants, use `mir_fresh_expanded_value` instead.
- `mir_tuple_value` *`elems`* produces a MIR tuple value. *`elems`* provides the contents of (and defines the arity of) the tuple.
- `mir_array_value` *`ty elems`* produces a MIR array value. *`ty`* is the element type; *`elems`* is a list of values to populate the array with. The type must match the element list… unless the element list is empty.
- `mir_struct_value` *`adt vals`* produces a MIR struct value. *`adt`* gives the type (as with `mir_enum_value`, use `mir_find_adt` to look up the type handle) and *`vals`* gives values for the fields, in order.
- `mir_slice_value` *`arr`* generates a slice value from an array reference. The slice contains the whole array. This is equivalent to `&`*`arr`*`[..]` in Rust. The argument must be an allocated array reference (not an array value) whose Rust type is `&[`*`ty`*`; `*`n`*`]` or `&mut [`*`ty`*`; `*`n`*`]`. The resulting slice is immutable (`&[`*`ty`*`]`) or mutable (`&mut [`*`ty`*`]`) depending on the mutability of the base reference.

- `mir_slice_range_value` *`arr lo hi`* is like `mir_slice_value` but generates a slice that refers to a subrange of the array; i.e. `&arr[lo..hi]`. It includes the elements from index *`lo`* up to but not including index *`hi`*. It is an error for *`lo`* to exceed *`hi`* or for either to index past the end of the underlying array. Also note that the index arguments have the SAWScript type `Int`, that is, they are constants. It is not possible to create a slice with a symbolic length. If this limitation prevents you from using SAW, please file an issue on GitHub (https://github.com/GaloisInc/saw-script/issues).
- `mir_str_slice_value` *`arr`* creates a string slice (of Rust type `&str`) from an array reference whose element type is `u8`. It is otherwise the same as `mir_slice_value`. The array is expected to be a UTF-8-encoded sequence of bytes.
- `mir_str_slice_range_value` *`arr lo hi`* has the same relationship to `mir_str_slice_value` that `mir_slice_range_value` has to `mir_slice_value`.

The elements of MIR values of compound type can be extracted with these functions:

- `mir_elem_value` *`arr ix`* takes an array value and an index, and returns the value in the array at that index. This returns a MIR-level value (type `MIRValue`).
- `mir_elem_ref` *`arr ix`* is like `mir_elem_value`, but takes a reference (or a raw pointer) to an array and returns a reference (or, respectively, a raw pointer) to the element in the array. The reference or raw pointer must already point to an array value. (Unlike the analogous `llvm_elem`, `mir_elem_ref` cannot be used to initialize freshly-allocated memory.)
- `mir_field_value` *`str f`* takes a struct value *`str`* and a field name *`f`*, which should be a string, and returns the value of the field. For "tuple structs" with numbered fields, use a string containing an integer constant, such as `"1"`.
- `mir_field_ref` *`str f`* is like `mir_field_value`, but takes a reference (or a raw pointer) instead of an immediate struct value and returns a reference (or, respectively, a raw pointer) to the named field. The reference or raw pointer must already point to a struct value. (Unlike the analogous `llvm_elem` or `llvm_field`, `mir_field_ref` cannot be used to initialize freshly-allocated memory.)

It is also possible to create an if-then-else expression at the MIR value level using the function `mir_mux_values`. It takes three arguments: the first is a Cryptol-level `Term` whose underlying type should be boolean, and the second and third are MIR-level values to produce if the control value is true and false respectively. The control value may be symbolic.

10.6.3 Slice Example

Consider this Rust function, which accepts an arbitrary slice as an argument:

```
pub fn f(s: &[u32]) -> u32 {
    s[0] + s[1]
}
```

We can write a specification that passes a slice to the array `[1, 2, 3, 4, 5]` as an argument to `f`:

```
let f_spec_1 = do {
  a <- mir_alloc (mir_array 5 mir_u32);
  mir_points_to a (mir_term {{ [1, 2, 3, 4, 5] : [5][32] }});

  mir_execute_func [mir_slice_value a];

  mir_return (mir_term {{ 3 : [32] }});
};
```

Alternatively, we can write a specification that passes a part of this array over the range `[1..3]`, i.e., ranging from second element to the fourth. Because this is a half-open range, the resulting slice has length 2:

```
let f_spec_2 = do {
  a <- mir_alloc (mir_array 5 mir_u32);
  mir_points_to a (mir_term {{ [1, 2, 3, 4, 5] : [5][32] }});

  mir_execute_func [mir_slice_range_value a 1 3];

  mir_return (mir_term {{ 5 : [32] }});
};
```

Note that we are passing *references* to arrays to `mir_slice_value` and `mir_slice_range_value`. It would be an error to pass a bare array to these functions, so the following specification would be invalid:

```
let f_fail_spec = do {
  let arr = mir_term {{ [1, 2, 3, 4, 5] : [5][32] }};

  mir_execute_func [mir_slice_value arr]; // BAD: `arr` is not a␣
↪reference

  mir_return (mir_term {{ 3 : [32] }});
};
```

10.6.4 Notes on Strings

One unusual requirement about `mir_str_slice_value` and `mir_str_slice_range_value` is that they require the argument to be of type `&[u8; N]`, i.e., a reference to an array of bytes. This is an artifact of the way that strings are encoded in Cryptol. The following Cryptol expressions:

- `"A"`
- `"123"`
- `"Hello World"`

have the following types:

- `[1][8]`
- `[3][8]`
- `[11][8]`

This is because Cryptol strings are syntactic shorthand for sequences of bytes. The following Cryptol expressions are wholly equivalent to the string literals above:

- `[0x41]`
- `[0x31, 0x32, 0x33]`
- `[0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0x64]`

These represent the strings in the extended ASCII character encoding. The Cryptol sequence type `[N][8]` corresponds to the Rust type `[u8; N]`, so the requirement to have something of type `&[u8; N]` as an argument reflects this design choice.

Note that `mir_str_slice_value` and `mir_slice_value` applied to the same `u8` array reference are *not* the same thing. The two commands represent different types of Rust values. While both commands take the same argument type, `mir_str_slice_value` will return a value of Rust type `&str` (or `&mut str`), whereas `mir_slice_value` will return a value of Rust type `&[u8]` (or `&mut [u8]`). These Rust types are checked when you pass these values as arguments to Rust functions (using `mir_execute_func`) or when you return these values (using `mir_return`), and it is an error to supply a `&str` value in a place where a `&[u8]` value is expected (and vice versa).

As an example of how to write specifications involving string slices, consider this Rust function:

```
pub fn my_len(s: &str) -> usize {
    s.len()
}
```

We can use `mir_str_slice_value` to write a specification for `my_len` when it is given the string `"hello"` as an argument:

```
let my_len_spec = do {
  s <- mir_alloc (mir_array 5 mir_u8);
  mir_points_to s (mir_term {{ "hello" }});

  mir_execute_func [mir_str_slice_value s];

  mir_return (mir_term {{ 5 : [64] }});
};
```

Currently, Cryptol only supports characters that can be encoded in a single byte. As a result, it is not currently possible to take slices of strings with certain characters. For example, the string `"roșu"` cannot be used as a Cryptol expression, as the character `'ș'` would require 10 bits to represent instead

of 8. The alternative is to use UTF-8 to encode such characters. For instance, the UTF-8 encoding of the string `"roşu"` is `"ro\200\153u"`, where `"\200\153"` is a sequence of two bytes that represents the `'ş'` character.

SAW makes no attempt to ensure that string slices over a particular range aligns with UTF-8 character boundaries. For example, the following Rust code would panic:

```
    let rosu: &str = "roşu";
    let s: &str = &rosu[0..3];
    println!("{:?}", s);
```

```
thread 'main' panicked at 'byte index 3 is not a char boundary; it is␣
↪inside 'ş' (bytes 2..4) of `roşu`'
```

On the other hand, SAW will allow you define a slice of the form `mir_str_slice_range r 0 3`, where `r` is a reference to `"ro\200\153u"`. It is the responsibility of the SAW user to ensure that `mir_str_slice_range` indices align with character boundaries.

10.6.5 Finding MIR Types

Rust supports a greater degree of polymorphism than the languages behind the other backends; in particular Rust types can take parameters. This introduces various complications referring to type names.

We collectively refer to MIR `structs` and `enums` together as *algebraic data types*, or ADTs for short. The function `mir_find_adt` looks up a handle for an algebraic data type by its name and type arguments for its type parameters. It takes three arguments: the first is a MIR module handle, the second is a string that names the type, and the third is a list of type arguments. The `MIRAdt` value returned can be given to the `mir_adt` function to get a `MIRType`, or passed directly to the `mir_enum_value` and `mir_struct_value` functions.

Alternatively, one can use the function `mir_find_mangled_adt` to look up a particular instantiation of a type by its "mangled" name, such as `foo::Bar::_adt12345`. The code at the end corresponds to some particular set of type arguments. However, these codes are not stable (they change unpredictably at the whim of `rustc`) and generally must be discovered by grovelling manually in your `linked-mir.json` file. Thus one should use `mir_find_adt` instead whenever possible. `mir_find_mangled_adt` takes two arguments: a MIR module handle and a string containing the mangled name.

For example, consider this code:

```
pub struct S<A, B> {
    pub x: A,
    pub y: B,
}
```

(continues on next page)

(continued from previous page)

```
pub fn f() -> S<u8, u16> {
    S {
        x: 1,
        y: 2,
    }
}

pub fn g() -> S<u32, u64> {
    S {
        x: 3,
        y: 4,
    }
}
```

At the MIR level, the return types of `f` and `g` are distinct types each with their own names. For example,

- `S<u8, u16>` might give rise to `example/abcd123::S::_adt456`
- `S<u32, u64>` might give rise to `example/abcd123::S::_adt789`

These can then be looked up with:

- `mir_find_adt m "example::S" [mir_u8, mir_u16];`
- `mir_find_adt m "example::S" [mir_u32, mir_u64];`

or

- `mir_find_mangled_adt m "example::S::_adt456";`
- `mir_find_mangled_adt m "example::S::_adt789";`

but in the second case one must discover the codes `456` and `789`.

The lookup functions return a SAWScript value of type `MIRAdt`, which can then be used with `mir_adt` to get a `MIRType`, or with `mir_struct_value` and `mir_enum_value` to construct MIR-level values.

Const Generics

Rust ADTs can have *const generic* parameters that allow the ADT to be generic over constant values. For instance, the following Rust code declares a const generic parameter `N` on the struct `S`, as well as on the functions `f` and `g` that compute `S` values:

```
pub struct S<const N: usize> {
    pub x: [u32; N]
}
```

(continues on next page)

(continued from previous page)

```
pub fn f(y: [u32; 1]) -> S<1> {
    S { x: y }
}

pub fn g(y: [u32; 2]) -> S<2> {
    S { x: y }
}
```

Like with other forms of Rust generics, instantiating `S` with different constants will give rise to different identifiers in the compiled MIR code. SAW provides a `mir_const` function for specifying the values of constants used to instantiate const generic parameters:

- `mir_const : MIRType -> Term -> MIRType`

For instance, in order to look up `S<1>`, use `mir_const` in conjunction with `mir_find_adt` like so:

```
s_adt = mir_find_adt m "example::S" [mir_const mir_usize {{ 1 : [64] }}]
```

Unlike other forms of `MIRType`s, the type returned by `mir_const` is not a type that you can create values with. For instance, calling `mir_alloc` or `mir_fresh_var` at a type returned by `mir_const` will raise an error. `mir_const` is only useful for looking up ADTs via `mir_find_adt`.

At present, `mir_const` only supports looking up constant values with the types listed here (https://doc.rust-lang.org/1.86.0/reference/items/generics.html#r-items.generics.const.allowed-types) in the Rust Reference. Specifically, the `MIRType` argument must be one of the following, subject to the following restrictions:

- A primitive integer type, i.e., `mir_u{8,16,32,64,128,size}` or `mir_i{8,16,32,64,128,size}`. The `Term` argument must be a bitvector of the corresponding size. For instance, if the `MIRType` is `mir_u8`, then the `Term` must be a bitvector of type `[8]`.
- `mir_bool`. The `Term` argument must be of type `Bit`.
- `mir_char`. The `Term` argument must be of type `[32]`.

10.6.6 Finding MIR Values and Functions

Rust also supports polymorphic functions. The Rust compiler does not generate a single generic implementation for polymorphic functions; instead it generates a separate version for every different type instantiation. Like instantiated polymorphic types, these are distinguished by hash codes, and the codes are not stable and not easy to discover.

The function `mir_find_name` looks up a function by its name and a list of types used to instantiate its type parameters. (The first argument is a string, the second is a list of `MIRType`.) It returns the mangled name as a string. This string can then be passed to `mir_verify` and other related functions.

10.6.7 Lifetime Values

Rust ADTs and functions can have *lifetime* parameters as well as type parameters. The following Rust code declares a lifetime parameter `'a` on the struct `S`, and also on the function `f` that computes an `S` value:

```
pub struct S<'a> {
    pub x: &'a u32,
}

pub fn f<'a>(y: &'a u32) -> S<'a> {
    S { x: y }
}
```

When `mir-json` compiles a piece of Rust code that contains lifetime parameters, it will instantiate all of the lifetime parameters with a placeholder MIR type that is simply called `lifetime`. This is important to keep in mind when looking up ADTs with `mir_find_adt`, as you will also need to indicate to SAW that the lifetime parameter is instantiated with `lifetime`. In order to do so, use `mir_lifetime`. For example, here is how to look up `S` with `'a` instantiated to `lifetime`:

```
s_adt = mir_find_adt m "example::S" [mir_lifetime]
```

Note that this part of SAW's design is subject to change in the future. Ideally, users would not have to care about lifetimes at all at the MIR level; see this issue (https://github.com/GaloisInc/mir-json/issues/58) for further discussion on this point. If that issue is fixed, then we will likely remove `mir_lifetime`, as it will no longer be necessary.

10.6.8 Binding Quantified MIR Variables

The basic function for this in the MIR backend is `mir_fresh_var` *`x ty`*. *`x`* provides an internal name for the variable; *`ty`* gives its type. This type is a MIR type (a value of type `MIRType`) rather than a Cryptol type. However, it must be an MIR type that has a corresponding Cryptol type, because the result is a Cryptol-level SAWScript value. (Thus, it has type `Term` and not type `MIRValue`.)

For example:

```
x <- mir_fresh_var "x" mir_i32;
```

This produces a fresh 32-bit machine integer.

There is also an experimental function `mir_fresh_cryptol_var` that takes a Cryptol type instead. Both of these functions produce Cryptol-level fresh values.

It is reasonable to use the same identifier for the internal name and for the SAWScript variable, and, where applicable, to make it the same name as used for the same thing in the code under verification.

The function `mir_fresh_expanded_value` *`name ty`* creates a fresh value of MIR-level type (thus, the type argument is an `MIRType`, and the result is a `MIRValue`) and populates it recursively with

fresh variables. The *name* argument is used as a prefix for the names of the generated fresh variables. This can be used for struct and array types. However, types that are or contain references or raw pointers are not currently supported.

In general, to reason about values of compound type, it is possible to create fresh primitive values for the elements and create compound values with functions like `mir_array_value`. It is also possible to use `mir_fresh_expanded_value` and reason about the contents by using functions like `mir_elem_value`. Which is preferable is in most cases purely a matter of convenience.

10.6.9 Allocation, References, and Pointers in the MIR Backend

MIR references and raw pointer values cannot be arbitrarily created; one must in general explicitly allocate memory to create a reference or pointer. The reference or pointer so created can then be passed as a function argument or used to initialize other values. Pointers allocated in the prestate logic are presumed to exist before the function is called; pointers allocated (in the `mir_alloc` sense) in the poststate logic are expected to be allocated (in the runtime sense) by the function.

The main allocator functions in the MIR backend are:

- `mir_alloc` *ty* allocates an immutable reference.
- `mir_alloc_mut` *ty* allocates a mutable reference.

Because Rust (and therefore MIR) distinguishes immutable references from mutable references at the type level, it's important to use the correct allocation function.

The allocation cannot be used to allocate slices (including string slices). Slices are shared references to other allocations; they can be constructed directly using `mir_slice_value` and friends after allocating the underlying reference.

There are also four functions for allocating raw pointers:

- `mir_alloc_raw_ptr_const` *ty* allocates a constant raw pointer to *ty*: `*const` *ty* in Rust terms.
- `mir_alloc_raw_ptr_mut` *ty* allocates a mutable raw pointer to *ty*: `*mut` *ty* in Rust terms.
- `mir_alloc_raw_ptr_const_multi` *n ty* allocates a constant raw pointer to a sequence/array of *n* _ty_s.
- `mir_alloc_raw_ptr_mut_multi` *n ty* allocates a mutable raw pointer to a sequence/array of *n* _ty_s.

In low-level (and unsafe) Rust code, it is possible to get a raw pointer into an allocation of multiple values and do pointer arithmetic, much like C arrays. These pointers must be generated using the last two functions. A pointer thus generated can be matched against a larger allocation; a spec whose prestate allocates a multi-raw-pointer of length *n* can be used as an override spec in a context where the pointer is actually longer, and a spec whose postate allocates a multi-raw-pointer of length *n* will verify even if the function allocates more space than that. That is, these functions declare an allocation that contains at least *n* values, and the actual allocation in the code can be larger.

Note that allocating memory and reasoning about its contents are separate steps. In most cases each `mir_alloc` should be paired with an `mir_points_to` assertion (see below) that says something about the value found in the allocated memory. (Otherwise the memory is treated as uninitialized, and attempts to read from it will be treated as an error. This is sometimes necessary to model unsafe Rust code.)

The following convenience functions are also available.

- `mir_ref_of` *`val`* and `mir_ref_of_mut` *`val`* are convenience functions that combine `mir_alloc` (or `mir_alloc_mut`) with the `mir_points_to` assertion.
- `mir_vec_of` *`name ty vals`* is an experimental convenience function that creates a value of type `Vec<`*`ty`*`>`. `Vec` (https://doc.rust-lang.org/std/vec/struct.Vec.html) is a commonly used data type in the Rust standard library. The *`vals`* argument is a `MIRValue` that's a MIR array (not a list of `MIRValue`s) used to populate the `Vec`. This can be created with `mir_array_value` or lifted from a Cryptol sequence value. The *`name`* argument is used as a prefix for the names of the internal symbolic variables inside the `Vec`. (Effectively, it is the name of the symbolic `Vec` being built.)

  `Vec` is just a regular struct and not a special language construct, so technically no special SAW support is needed. However, you would need to explicitly specify all the internal details and invariants of `Vec`, and that can get quite messy.

  This function may change in the future; there are unresolved internal design questions.

There is a function `mir_cast_raw_ptr` that takes a raw pointer value and a MIR type, and yields a new pointer value with an updated pointed-to type. This is an unsafe operation needed for handling certain kinds of unsafe code. The pointer cannot actually be accessed unless it is cast back to its original type. (It is thus different from `llvm_cast_pointer`, which allows reinterpreting memory.)

10.6.10 MIR Static Variables

Rust static values (global variables) can be referenced with the function `mir_static` *`name`*, which returns a reference to the static called *`name`*. (The name follows the same qualification rules as other MIR names.) This reference is mutable if and only if the static is mutable. It can be reasoned about in the prestate and poststate logic like any other reference.

The initialization value for a static can be fetched with `mir_static_initializer` *`name`*. Sometimes this will be the prestate value you want.

Explicit allocation like `llvm_alloc_global` is not required, even for mutable statics.

Here is an example involving static items:

```
// statics.rs
static     S1: u8 = 1;
static mut S2: u8 = 2;

pub fn f() -> u8 {
```

(continues on next page)

(continued from previous page)

```
    // Reading a mutable static item requires an `unsafe` block due to
    // aliasing and concurrency-related concerns. We have no aliasing,
    // and are only concerned about the behavior of this program in a
    // single-threaded context.
    let s2 = unsafe { S2 };
    S1 + s2
}
```

We can write a specification for `f` like so:

```
// statics.saw

let f_spec = do {
  mir_points_to (mir_static "statics::S2")
                (mir_static_initializer "statics::S2");
  // Note that we do not need to initialize S1, as immutable static
  // items are implicitly initialized in every specification.

  mir_execute_func [];

  mir_return (mir_term {{ 3 : [8] }});
};

m <- mir_load_module "statics.linked-mir.json";

mir_verify m "statics::f" [] false f_spec z3;
```

In order to use a specification involving mutable static items for compositional verification, it is required to specify the value of all mutable static items using the `mir_points_to` command in the specification's postconditions. For further explanation, see the *Compositional Verification and Mutable Global Variables* section.

10.6.11 Assertions in the MIR Backend

The basic function for MIR assertions is `mir_assert`. It takes one argument, which is a `Term` (Cryptol-level value); frequently that term will be a Cryptol block written with double-braces. If in the prestate logic, it generates a precondition; in the poststate logic, it generates a postcondition.

The function `mir_precond` is equivalent, except it is only allowed in the prestate logic. Similarly, `mir_postcond` is only allowed in the poststate logic.

The function `mir_equal` is comparable to using `mir_assert` with an equality; it takes two arguments and asserts that they are equal. However, the arguments it takes are MIR-level values (`MIRValue`) rather than Cryptol-level values (`Term`), so it can express equalities that would be messy

with `mir_assert`. The use of `mir_equal` when applicable can also sometimes lead to more efficient symbolic execution.

The function `mir_return` *`val`* asserts that the value returned by the function is *`val`*. This is only allowed in the poststate logic.

The function `mir_points_to` is the basic way to assert about a pointer. It takes two arguments that are both MIR-level values; the first is the reference (or raw pointer) and the second is the contents.

There is a variant of `mir_points_to` for raw pointers that point at sequences of values: `mir_points_to_multi`. The first argument must be a raw pointer; the second should be a `MIRValue` of array type. The pointer must have been allocated with `mir_alloc_raw_ptr_const_multi` or `mir_alloc_raw_ptr_mut_multi` and should contain space for at least as many values as the argument array holds. The array is copied sequentially; it is not necessarily the case that the pointed-to values are the contents of a Rust array. Using a `MIRValue` allows lifting a Cryptol sequence with `mir_term`, which is often convenient.

10.6.12 Loading MIR Modules

To load a piece of Rust code, first compile it to a MIR JSON file, as described immediately below, and then provide the location of the JSON file to the `mir_load_module` function:

- `mir_load_module : String -> TopLevel MIRModule`

SAW currently supports Rust code that can be built with a March 21, 2026 Rust nightly (https://static.rust-lang.org/dist/2026-03-21/). If you encounter a Rust feature that SAW does not support, please report it on GitHub (https://github.com/GaloisInc/saw-script/issues).

Compiling MIR

When verifying Rust code, SAW processes Rust's MIR (mid-level intermediate representation) language. In particular, SAW analyzes a particular form of MIR that our `mir-json` (https://github.com/GaloisInc/mir-json) tool produces. You will need to install `mir-json` and run it on Rust code in order to produce MIR JSON files that SAW can load. You will also need to use `mir-json` to build custom versions of the Rust standard libraries that are more suited to verification purposes.

If you are working from a checkout of the `saw-script` repo, you can install the `mir-json` tool and the custom Rust standard libraries by performing the following steps:

1. Make sure you have the proper versions of the `crucible` (https://github.com/GaloisInc/crucible) and `mir-json` submodules like so:

```
$ git submodule update deps/crucible deps/mir-json
```

2. Navigate to the `mir-json` submodule:

```
$ cd deps/mir-json
```

3. Follow the instructions laid out in the `mir-json` installation instructions (https://github.com/GaloisInc/mir-json#installation-instructions) in order to install `mir-json`.
4. Run the `mir-json-translate-libs` script in the `mir-json` submodule:

   ```
   $ mir-json-translate-libs
   ```

   This will compile the custom versions of the Rust standard libraries using `mir-json`, placing the results under the `rlibs` subdirectory.
5. Finally, define a `SAW_RUST_LIBRARY_PATH` environment variable that points to the newly created `rlibs` subdirectory:

   ```
   $ export SAW_RUST_LIBRARY_PATH=<...>/mir-json/rlibs
   ```

For `cargo`-based projects, `mir-json` provides a `cargo` subcommand called `cargo saw-build` that builds a JSON file suitable for use with SAW. `cargo saw-build` integrates directly with `cargo`, so you can pass flags to it like any other `cargo` subcommand. For example:

```
# Make sure that SAW_RUST_LIBRARY_PATH is defined, as described above
$ cargo saw-build <other cargo flags>
<snip>
linking 11 mir files into <...>/example-364cf2df365c7055.linked-mir.json
<snip>
```

Note that:

- The full output of `cargo saw-build` here is omitted. The important part is the `.linked-mir.json` file that appears after `linking X mir files into`, as that is the JSON file that must be loaded with SAW.
- `SAW_RUST_LIBRARY_PATH` should point to the MIR JSON files for the Rust standard library.

`mir-json` also supports compiling individual `.rs` files through `mir-json`'s `saw-rustc` command. As the name suggests, it accepts all of the flags that `rustc` accepts. For example:

```
# Make sure that SAW_RUST_LIBRARY_PATH is defined, as described above
$ saw-rustc example.rs <other rustc flags>
<snip>
linking 11 mir files into <...>/example.linked-mir.json
<snip>
```

10.6.13 MIR Verification

Verification of Rust code in the MIR backend is done with the `mir_verify` command.

```
mir_verify :
  MIRModule ->
  String ->
  [MIRSpec] ->
  Bool ->
  MIRSetup () ->
  ProofScript () ->
  TopLevel MIRSpec
```

The first two arguments specify the module and function name to verify. The third argument specifies the list of already-verified specifications to use as overrides for compositional verification. The fourth argument specifies whether to do path satisfiability checking, and the fifth gives the specification of the function to be verified. Finally, the last argument gives the proof script to use for verification. The result is a proved specification that can be used to simplify verification of functions that call this one.

The `String` supplied as an argument to `mir_verify` is expected to be a function *identifier*. This should have one of the following forms:

- `<crate name>/<disambiguator>::<function path>`
- `<crate name>::<function path>`

Where:

- `<crate name>` is the name of the crate in which the function is defined. (If you produced your MIR JSON file by compiling a single `.rs` file with `saw-rustc`, then the crate name is the same as the name of the file, but without the `.rs` file extension.)
- `<disambiguator>` is a hash of the crate and its dependencies. In extreme cases, it is possible for two different crates to have identical crate names, in which case the disambiguator must be used to distinguish between the two crates. In the common case, however, most crate names will correspond to exactly one disambiguator, and you are allowed to leave out the `/<disambiguator>` part of the `String` in this case. If you supply an identifier with an ambiguous crate name and omit the disambiguator, then SAW will raise an error.
- `<function path>` is the path to the function within the crate. Sometimes, this is as simple as the function name itself. In other cases, a function path may involve multiple *segments*, depending on the module hierarchy for the program being verified. For instance, a `read` function located in `core/src/ptr/mod.rs` will have the identifier:

  ```
  core::ptr::read
  ```

  where `core` is the crate name and `ptr::read` is the function path, which has two segments `ptr` and `read`. There are also some special forms of segments that appear for functions defined in certain language constructs. For instance, if a function is defined in an `impl` block, then it will have `{impl}` as one of its segments, e.g.,

```
core::ptr::const_ptr::{impl}::offset
```

If you are in doubt about what the full identifier for a given function is, consult the MIR JSON file for your program.

The Rust compiler generates a separate instance for each use of a polymorphic function at a different type. These instances have a compiler-generated name, so (as discussed above) the easiest way to refer to them is by using the command `mir_find_name : MIRModule -> String -> [MIRType] -> String`. Given a Rust module, the name of a polymorphic function, and a list of types, `mir_find_name` will return the name of the corresponding instantiation. It fails if the polymorphic function or the given instantiation are not found.

10.6.14 MIR error messages

SAW includes call stacks when reporting certain classes of MIR-related error messages that arise during symbolic execution. For instance, if you write a SAW specification for the `g` function:

```
pub fn f(x: *const u32) -> u32 {
    unsafe { *x }
}

pub fn g() -> u32 {
    let x: *const u32 = std::ptr::null();
    f(x)
}
```

Then symbolic execution will fail when it attempts to dereference a null pointer while simulating the `f` function. By default, this error message will look something like the following:

```
Symbolic execution failed.
Abort due to assertion failure:
  test.rs:2:14: 2:16: error: in test/11acf56f::f[0]
  attempted to read empty mux tree
  Context:
    test.rs:2:14: 2:16: test/11acf56f::f[0]
    test.rs:8:5: 8:9: test/11acf56f::g[0]
```

Note that the `Context:` includes both `f` and `g`. Call stacks can be helpful for determining what path through the program is used to reach an error, although they come with the tradeoff that they make error messages longer. SAW offers the following configuration options for tweaking these error messages:

- `mir_set_exception_context_none : TopLevel ()`: Call stacks are not displayed at all.
- `mir_set_exception_context_limited : Int -> TopLevel ()`: Call stacks are

displayed up to a limited number, where the supplied `Int` is the limit. This option is the default setting, where the default limit is 10.

- `mir_set_exception_context_unlimited : TopLevel ()`: Call stacks are displayed, and the full call stack is always printed.

10.7 Compositional Verification

As mentioned briefly above, SAW's specification scheme for code allows for compositional reasoning. That is, when proving properties of a given method or function, we can make use of properties we have already proved about its callees rather than analyzing them anew. This enables us to reason about much larger and more complex systems than otherwise possible.

The `llvm_verify`, `jvm_verify`, and `mir_verify` functions return values of type `LLVMSpec`, `JVMSpec`, and `MIRSpec`, respectively. These values are opaque objects that internally contain both the information provided in the associated `LLVMSetup`, `JVMSetup`, or `MIRSetup` specification do-blocks, respectively, and the results of the verification process.

Any of these `Spec` objects can be passed in via the third argument of the `..._verify` functions. For any function or method specified by one of these parameters, the simulator will not follow calls to the associated target. Instead, it will perform the following steps:

- Check that all `llvm_points_to` and `llvm_precond` statements (or the corresponding JVM or MIR statements) in the specification are satisfied.
- Update the simulator state and optionally construct a return value as described in the specification.

More concretely, suppose we have an `add` function (simpler than the one earlier in the chapter, which was supposed to also illustrate memory allocation) and a doubling function written in terms of it:

```
unsigned add(unsigned x, unsigned y) {
    return x + y;
}
unsigned dbl(unsigned *x) {
    return add(x, x);
}
```

We can write a specification for `add`:

```
let add_spec = do {
   x <- llvm_fresh_var "x" (llvm_int 32);
   y <- llvm_fresh_var "y" (llvm_int 32);
   llvm_execute_func [llvm_term x, llvm_term y];
   llvm_return (llvm_term {{ x + y }});
};
```

And one for `dbl`, which is very similar to the one for `add`:

```
let dbl_spec = do {
   x <- llvm_fresh_var "x" (llvm_int 32);
   llvm_execute_func [llvm_term x];
   llvm_return (llvm_term {{ x + x }});
};
```

We can verify `add` and then verify `dbl` using what we proved about `add`:

```
add_theorem <- llvm_verify bc "add" [] true add_spec z3;
dbl_theorem <- llvm_verify bc "dbl" [add_theorem] true dbl_spec z3;
```

When `dbl` calls `add`, `llvm_verify` uses the specification we proved instead of executing through it again. In this case, it doesn't save computational effort, since `add` is so simple. However, it illustrates the approach.

Imagine we were doing something less trivial and `add` took ten minutes to verify. The compositional verification of `dbl` would, instead of taking another ten minutes to verify, be almost instantaneous.

This example also illustrates a limitation of the technique: it only allows reusing the *proof effort*. The specification for `dbl` still needs to be a complete characterization of its behavior, and you will notice it does not share any logic with the specification for `add`. Reusing *specification effort* by sharing elements of the specification is a different problem. For logic more complicated than addition, one can factor out elements of the behavioral specification into a shared Cryptol model. (Sharing elements of the call setup and theorem statement requires SAWScript programming, which is an advanced topic. See *The SAWScript Language*.)

10.7.1 Compositional Verification and Mutable Allocations

A common pitfall when using compositional verification is to reuse a specification that underspecifies the value of a mutable allocation. In general, doing so can lead to unsound verification, so SAW goes to great length to check for this.

Here is an example of this pitfall in an LLVM verification. Given this C code:

```
void side_effect(uint32_t *a) {
  *a = 0;
}

uint32_t foo(uint32_t x) {
  uint32_t b = x;
  side_effect(&b);
  return b;
}
```

And the following SAW specifications:

```
let side_effect_spec = do {
  a_ptr <- llvm_alloc (llvm_int 32);
  a_val <- llvm_fresh_var "a_val" (llvm_int 32);
  llvm_points_to a_ptr (llvm_term a_val);

  llvm_execute_func [a_ptr];
};

let foo_spec = do {
  x <- llvm_fresh_var "x" (llvm_int 32);

  llvm_execute_func [llvm_term x];

  llvm_return (llvm_term x);
};
```

Should SAW be able to verify the `foo` function against `foo_spec` using compositional verification? That is, should the following be expected to work?

```
side_effect_ov <- llvm_verify m "side_effect" [] false side_effect_spec␣
↪z3;
llvm_verify m "foo" [side_effect_ov] false foo_spec z3;
```

A literal reading of `side_effect_spec` would suggest that the `side_effect` function allocates `a_ptr` but then does nothing with it, implying that `foo` returns its argument unchanged. This is incorrect, however, as the `side_effect` function actually changes its argument to point to `0`, so the `foo` function ought to return `0` as a result. SAW should not verify `foo` against `foo_spec`, and indeed it does not.

The problem is that `side_effect_spec` underspecifies the value of `a_ptr` in its postconditions, which can lead to the potential unsoundness seen above when `side_effect_spec` is used in compositional verification. To prevent this source of unsoundness, SAW will *invalidate* the underlying memory of any mutable pointers (i.e., those declared with `llvm_alloc`, not `llvm_alloc_global`) allocated in the preconditions of compositional override that do not have a corresponding `llvm_points_to` statement in the postconditions. Attempting to read from invalidated memory constitutes an error, as can be seen in this portion of the error message when attempting to verify `foo` against `foo_spec`:

```
invalidate (state of memory allocated in precondition (at side.
↪saw:3:12) not described in postcondition)
```

To fix this particular issue, add an `llvm_points_to` statement to `side_effect_spec`:

```
let side_effect_spec = do {
  a_ptr <- llvm_alloc (llvm_int 32);
```

(continues on next page)

(continued from previous page)

```
  a_val <- llvm_fresh_var "a_val" (llvm_int 32);
  llvm_points_to a_ptr (llvm_term a_val);

  llvm_execute_func [a_ptr];

  // This is new
  llvm_points_to a_ptr (llvm_term {{ 0 : [32] }});
};
```

After making this change, SAW will reject `foo_spec` for a different reason, as it claims that `foo` returns its argument unchanged when it actually returns `0`.

Note that invalidating memory itself does not constitute an error, so if the `foo` function never read the value of `b` after calling `side_effect(&b)`, then there would be no issue. It is only when a function attempts to *read* from invalidated memory that an error is thrown. In general, it can be difficult to predict when a function will or will not read from invalidated memory, however. For this reason, it is recommended to always specify the values of mutable allocations in the postconditions of your specs, as it can avoid pitfalls like the one above.

The same pitfalls apply to compositional MIR verification, with a couple of key differences. In MIR verification, mutable references are allocated using `mir_alloc_mut`. Here is a Rust version of the pitfall program above:

```
pub fn side_effect(a: &mut u32) {
    *a = 0;
}

pub fn foo(x: u32) -> u32 {
    let mut b: u32 = x;
    side_effect(&mut b);
    b
}
```

```
let side_effect_spec = do {
  a_ref <- mir_alloc_mut mir_u32;
  a_val <- mir_fresh_var "a_val" mir_u32;
  mir_points_to a_ref (mir_term a_val);

  mir_execute_func [a_ref];
};

let foo_spec = do {
  x <- mir_fresh_var "x" mir_u32;
```

(continues on next page)

(continued from previous page)

```
  mir_execute_func [mir_term x];

  mir_return (mir_term {{ x }});
};
```

Just like above, if you attempted to prove `foo` against `foo_spec` using compositional verification:

```
side_effect_ov <- mir_verify m "test::side_effect" [] false side_effect_
↪spec z3;
mir_verify m "test::foo" [side_effect_ov] false foo_spec z3;
```

Then SAW would throw an error, as `side_effect_spec` underspecifies the value of `a_ref` in its postconditions. `side_effect_spec` can similarly be repaired by adding a `mir_points_to` statement involving `a_ref` in `side_effect_spec`'s postconditions.

MIR verification differs slightly from LLVM verification in how it catches underspecified mutable allocations when using compositional overrides. The LLVM memory model achieves this by invalidating the underlying memory in underspecified allocations. The MIR memory model, on the other hand, does not have a direct counterpart to memory invalidation. As a result, any MIR overrides must specify the values of all mutable allocations in their postconditions, *even if the function that calls the override never uses the allocations*.

To illustrate this point more finely, suppose that the `foo` function had instead been defined like this:

```
pub fn foo(x: u32) -> u32 {
    let mut b: u32 = x;
    side_effect(&mut b);
    42
}
```

Here, it does not particularly matter what effects the `side_effect` function has on its argument, as `foo` will now return `42` regardless. Still, if you attempt to prove `foo` by using `side_effect` as a compositional override, then it is strictly required that you specify the value of `side_effect`'s argument in its postconditions, even though the answer that `foo` returns is unaffected by this. This is in contrast with LLVM verification, where one could get away without specifying `side_effect`'s argument in this example, as the invalidated memory in `b` would never be read.

10.7.2 Compositional Verification and Mutable Global Variables

Just like with local mutable allocations (see the previous section), specifications used in compositional overrides must specify the values of mutable global variables in their postconditions. To illustrate this using LLVM verification, here is a variant of the C program from the previous example that uses a mutable global variable `a`:

```
uint32_t a = 42;

void side_effect(void) {
  a = 0;
}

uint32_t foo(void) {
  side_effect();
  return a;
}
```

If we attempted to verify `foo` against this `foo_spec` specification using compositional verification:

```
let side_effect_spec = do {
  llvm_alloc_global "a";
  llvm_points_to (llvm_global "a") (llvm_global_initializer "a");

  llvm_execute_func [];
};

let foo_spec = do {
  llvm_alloc_global "a";
  llvm_points_to (llvm_global "a") (llvm_global_initializer "a");

  llvm_execute_func [];

  llvm_return (llvm_global_initializer "a");
};

side_effect_ov <- llvm_verify m "side_effect" [] false side_effect_spec␣
↪z3;
llvm_verify m "foo" [side_effect_ov] false foo_spec z3;
```

Then SAW would reject it, as `side_effect_spec` does not specify what `a`'s value should be in its postconditions. Just as with local mutable allocations, SAW will invalidate the underlying memory in `a`, and subsequently reading from `a` in the `foo` function will throw an error. The solution is to add an `llvm_points_to` statement in the postconditions that declares that `a`'s value is set to `0`.

The same concerns apply to MIR verification, where mutable global variables are referred to as `static mut` items. (See the *MIR static items* section for more information). Here is a Rust version of the program above:

```
static mut A: u32 = 42;
```

(continues on next page)

(continued from previous page)

```
pub fn side_effect() {
    unsafe {
        A = 0;
    }
}

pub fn foo() -> u32 {
    side_effect();
    unsafe { A }
}
```

```
let side_effect_spec = do {
  mir_points_to (mir_static "test::A") (mir_static_initializer "test::A
↪");

  mir_execute_func [];
};

let foo_spec = do {
  mir_points_to (mir_static "test::A") (mir_static_initializer "test::A
↪");

  mir_execute_func [];

  mir_return (mir_static_initializer "test::A");
};

side_effect_ov <- mir_verify m "side_effect" [] false side_effect_spec␣
↪z3;
mir_verify m "foo" [side_effect_ov] false foo_spec z3;
```

Just as above, we can repair this by adding a `mir_points_to` statement in `side_effect_spec`'s postconditions that specifies that `A` is set to `0`.

Recall from the previous section that MIR verification is stricter than LLVM verification when it comes to specifying mutable allocations in the postconditions of compositional overrides. This is especially true for mutable static items. In MIR verification, any compositional overrides must specify the values of all mutable static items in the entire program in their postconditions, *even if the function that calls the override never uses the static items*. For example, if the `foo` function were instead defined like this:

```
pub fn foo() -> u32 {
    side_effect();
```

(continues on next page)

(continued from previous page)

```
    42
}
```

Then it is still required for `side_effect_spec` to specify what `A`'s value will be in its postconditions, despite the fact that this has no effect on the value that `foo` will return.

10.8 Using Ghost State

In some cases, information relevant to verification is not directly present in the concrete state of the program being verified. This can happen for at least two reasons:

- When providing specifications for external functions, for which source code is not present. The external code may read and write global state that is not directly accessible from the code being verified.
- When the abstract specification of the program naturally uses a different representation for some data than the concrete implementation in the code being verified does.

One solution to these problems is the use of *ghost* state. This can be thought of as additional global state that is visible only to the verifier. Ghost state with a given name can be declared at the top level with the following function:

- `declare_ghost_state : String -> TopLevel Ghost`

Ghost state variables do not initially have any particular type, and can store data of any type. Given an existing ghost variable the following functions can be used to specify its value:

- `llvm_ghost_value : Ghost -> Term -> LLVMSetup ()`
- `jvm_ghost_value : Ghost -> Term -> JVMSetup ()`
- `mir_ghost_value : Ghost -> Term -> MIRSetup ()`

These can be used in either prestate or poststate, to specify the value of ghost state either before or after the execution of the function, respectively.

10.9 Extracting Code

In many simple cases (such as the `max` function to return the largest of two values), the relevant inputs and outputs are immediately apparent. The function takes two integer arguments, always uses both of them, and returns a single integer value, making no other changes to the program state.

In cases like this, a direct translation to SAWCore is possible, given only an identification of which code to execute. Three functions exist to handle such simple code, one for each Crucible backend:

- `llvm_extract : LLVMModule -> String -> TopLevel Term`
- `jvm_extract : JavaClass -> String -> TopLevel Term`

- `mir_extract : MIRModule -> String -> TopLevel Term`

The structure of these extraction functions is essentially identical. The first argument describes where to look for code (in an LLVM module, Java class, or MIR module, loaded as described above). The second argument is the name of the method or function to extract.

When the extraction functions complete, they return a `Term` corresponding to the value returned by the function or method as a function of its arguments.

These functions currently work only for code that has specific argument and result types:

- For `llvm_extract`, the extracted function must take some fixed number of integral parameters and return an integral result.
- For `jvm_extract`, the extracted function's argument and result types must be scalar types (i.e., not classes or arrays).
- For `mir_extract`, the extracted function's argument and result types must be a primitive integer type (e.g., `u8` or `i8`), a `bool`, a `char`, an array, or a tuple.

Although it is disallowed to extract functions that use pointers, classes, or references in the extracted function's type signature, the implementation of the extracted function is allowed to allocate memory during execution. Also note the following requirements for interacting with global variables:

- For `llvm_extract`, the extracted function is allowed to read from immutable global variables during execution, but it is not allowed to read or write from mutable global variables during execution.
- For `jvm_extract`, the extracted function is allowed to read from or write to any class field or static field (regardless of mutability) during execution. The class and static fields will be given their initial values during extraction (unless they are overwritten during execution).
- For `mir_extract`, the extracted function is allowed to read from immutable static items during execution, and it is allowed to write to mutable static items during execution. The extracted function is not allowed to read from a mutable static item during execution unless the function has written another value to the static item earlier during execution.

10.10 A Heap-Based Example

To tie all of the command descriptions from the previous sections together, consider the case of verifying the correctness of a C program that computes the dot product of two vectors, where the length and value of each vector are encapsulated together in a `struct`.

The dot product can be concisely specified in Cryptol as follows:

```
dotprod : {n, a} (fin n, fin a) => [n][a] -> [n][a] -> [a]
dotprod xs ys = sum (zip (*) xs ys)
```

To implement this in C, let's first consider the type of vectors:

```
typedef struct {
    uint32_t *elts;
    uint32_t size;
} vec_t;
```

This struct contains a pointer to an array of 32-bit elements, and a 32-bit value indicating how many elements that array has.

We can compute the dot product of two of these vectors with the following C code (which uses the size of the shorter vector if they differ in size).

```
uint32_t dotprod_struct(vec_t *x, vec_t *y) {
    uint32_t size = MIN(x->size, y->size);
    uint32_t res = 0;
    for(size_t i = 0; i < size; i++) {
        res += x->elts[i] * y->elts[i];
    }
    return res;
}
```

The entirety of this implementation can be found in the `examples/llvm/dotprod_struct.c` file in the `saw-script` repository.

To verify this program in SAW, it will be convenient to define a couple of utility functions (which are generally useful for many heap-manipulating programs). First, combining allocation and initialization to a specific value can make many scripts more concise:

```
let alloc_init ty v = do {
    p <- llvm_alloc ty;
    llvm_points_to p v;
    return p;
};
```

This creates a pointer `p` pointing to enough space to store type `ty`, and then indicates that the pointer points to value `v` (which should be of that same type).

A common case for allocation and initialization together is when the initial value should be entirely symbolic.

```
let ptr_to_fresh n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init ty (llvm_term x);
    return (x, p);
};
```

This function returns the pointer just allocated along with the fresh symbolic value it points to.

Given these two utility functions, the `dotprod_struct` function can be specified as follows:

```
let dotprod_spec n = do {
    let nt = llvm_term {{ `n : [32] }};
    (xs, xsp) <- ptr_to_fresh "xs" (llvm_array n (llvm_int 32));
    (ys, ysp) <- ptr_to_fresh "ys" (llvm_array n (llvm_int 32));
    let xval = llvm_struct_value [ xsp, nt ];
    let yval = llvm_struct_value [ ysp, nt ];
    xp <- alloc_init (llvm_alias "struct.vec_t") xval;
    yp <- alloc_init (llvm_alias "struct.vec_t") yval;
    llvm_execute_func [xp, yp];
    llvm_return (llvm_term {{ dotprod xs ys }});
};
```

Any instantiation of this specification is for a specific vector length `n`, and assumes that both input vectors have that length. That length `n` automatically becomes a type variable in the subsequent Cryptol expressions, and the backtick operator is used to reify that type as a bit vector of length 32.

The entire script can be found in the `dotprod_struct-crucible.saw` file alongside `dotprod_struct.c`.

Running this script results in the following:

```
Loading file "dotprod_struct.saw"
Proof succeeded! dotprod_struct
Registering override for `dotprod_struct`
  variant `dotprod_struct`
Symbolic simulation completed with side conditions.
Proof succeeded! dotprod_wrap
```

10.11 An Extended Example

To tie together many of the concepts in this manual, we now present a non-trivial verification task in its entirety. All of the code for this example can be found in the `examples/salsa20` directory of the SAWScript repository (https://github.com/GaloisInc/saw-script).

10.11.1 Salsa20 Overview

Salsa20 is a stream cipher developed in 2005 by Daniel J. Bernstein, built on a pseudorandom function utilizing add-rotate-XOR (ARX) operations on 32-bit words[4]. Bernstein himself has provided several public domain implementations of the cipher, optimized for common machine architectures. For the mathematically inclined, his specification for the cipher can be found here (http://cr.yp.to/snuffle/spec.pdf).

[4] https://en.wikipedia.org/wiki/Salsa20

The repository referenced above contains three implementations of the Salsa20 cipher: A reference Cryptol implementation (which we take as correct in this example), and two C implementations, one of which is from Bernstein himself. For this example, we focus on the second of these C implementations, which more closely matches the Cryptol implementation. Full verification of Bernstein's implementation is available in `examples/salsa20/djb`, for the interested. The code for this verification task can be found in the files named according to the pattern `examples/salsa20/(s|S)alsa20.*`.

10.11.2 Specifications

We now take on the actual verification task. This will be done in two stages: We first define some useful utility functions for constructing common patterns in the specifications for this type of program (i.e. one where the arguments to functions are modified in-place.) We then demonstrate how one might construct a specification for each of the functions in the Salsa20 implementation described above.

Utility Functions

We first define the function `alloc_init : LLVMType -> Term -> LLVMSetup LLVMValue`.

`alloc_init ty v` returns a `LLVMValue` consisting of a pointer to memory allocated and initialized to a value `v` of type `ty`. `alloc_init_readonly` does the same, except the memory allocated cannot be written to.

```
import "Salsa20.cry";

let alloc_init ty v = do {
    p <- llvm_alloc ty;
    llvm_points_to p (llvm_term v);
    return p;
};

let alloc_init_readonly ty v = do {
    p <- llvm_alloc_readonly ty;
    llvm_points_to p (llvm_term v);
    return p;
};
```

We now define `ptr_to_fresh : String -> LLVMType -> LLVMSetup (Term, LLVMValue)`.

`ptr_to_fresh n ty` returns a pair `(x, p)` consisting of a fresh symbolic variable `x` of type `ty` and a pointer `p` to it. `n` specifies the name that SAW should use when printing `x`. `ptr_to_fresh_readonly` does the same, but returns a pointer to space that cannot be written to.

```
let ptr_to_fresh n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init ty x;
    return (x, p);
};

let ptr_to_fresh_readonly n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init_readonly ty x;
    return (x, p);
};
```

Finally, we define `oneptr_update_func : String -> LLVMType -> Term -> LLVMSetup ()`.

`oneptr_update_func n ty f` specifies the behavior of a function that takes a single pointer (with a printable name given by `n`) to memory containing a value of type `ty` and mutates the contents of that memory. The specification asserts that the contents of this memory after execution are equal to the value given by the application of `f` to the value in that memory before execution.

```
let oneptr_update_func n ty f = do {
    (x, p) <- ptr_to_fresh n ty;
    llvm_execute_func [p];
    llvm_points_to p (llvm_term {{ f x }});
};
```

The `quarterround` operation

The C function we wish to verify has type `void s20_quarterround(uint32_t *y0, uint32_t *y1, uint32_t *y2, uint32_t *y3)`.

The function's specification generates four symbolic variables and pointers to them in the pre-condition/setup stage. The pointers are passed to the function during symbolic execution via `llvm_execute_func`. Finally, in the postcondition/return stage, the expected values are computed using the trusted Cryptol implementation and it is asserted that the pointers do in fact point to these expected values.

```
let quarterround_setup : LLVMSetup () = do {
    (y0, p0) <- ptr_to_fresh "y0" (llvm_int 32);
    (y1, p1) <- ptr_to_fresh "y1" (llvm_int 32);
    (y2, p2) <- ptr_to_fresh "y2" (llvm_int 32);
    (y3, p3) <- ptr_to_fresh "y3" (llvm_int 32);

    llvm_execute_func [p0, p1, p2, p3];
```

(continues on next page)

(continued from previous page)

```
    let zs = {{ quarterround [y0,y1,y2,y3] }}; // from Salsa20.cry
    llvm_points_to p0 (llvm_term {{ zs@0 }});
    llvm_points_to p1 (llvm_term {{ zs@1 }});
    llvm_points_to p2 (llvm_term {{ zs@2 }});
    llvm_points_to p3 (llvm_term {{ zs@3 }});
};
```

Simple Updating Functions

The following functions can all have their specifications given by the utility function `oneptr_update_func` implemented above, so there isn't much to say about them.

```
let rowround_setup =
    oneptr_update_func "y" (llvm_array 16 (llvm_int 32)) {{ rowround }};

let columnround_setup =
    oneptr_update_func "x" (llvm_array 16 (llvm_int 32)) {{ columnround␣
↪}};

let doubleround_setup =
    oneptr_update_func "x" (llvm_array 16 (llvm_int 32)) {{ doubleround␣
↪}};

let salsa20_setup =
    oneptr_update_func "seq" (llvm_array 64 (llvm_int 8)) {{ Salsa20 }};
```

32-Bit Key Expansion

The next function of substantial behavior that we wish to verify has the following prototype:

```
void s20_expand32( uint8_t *k
                 , uint8_t n[static 16]
                 , uint8_t keystream[static 64]
                 )
```

This function's specification follows a similar pattern to that of `s20_quarterround`, though for extra assurance we can make sure that the function does not write to the memory pointed to by `k` or `n` using the utility `ptr_to_fresh_readonly`, as this function should only modify `keystream`. Besides this, we see the call to the trusted Cryptol implementation specialized to `a=2`, which does 32-bit key expansion (since the Cryptol implementation can also specialize to `a=1` for 16-bit keys). This specification can easily be changed to work with 16-bit keys.

```
let salsa20_expansion_32 = do {
    (k, pk) <- ptr_to_fresh_readonly "k" (llvm_array 32 (llvm_int 8));
    (n, pn) <- ptr_to_fresh_readonly "n" (llvm_array 16 (llvm_int 8));

    pks <- llvm_alloc (llvm_array 64 (llvm_int 8));

    llvm_execute_func [pk, pn, pks];

    let rks = {{ Salsa20_expansion`{a=2}(k, n) }};
    llvm_points_to pks (llvm_term rks);
};
```

32-bit Key Encryption

Finally, we write a specification for the encryption function itself, which has type

```
enum s20_status_t s20_crypt32( uint8_t *key
                             , uint8_t nonce[static 8]
                             , uint32_t si
                             , uint8_t *buf
                             , uint32_t buflen
                             )
```

As before, we can ensure this function does not modify the memory pointed to by `key` or `nonce`. We take `si`, the stream index, to be 0. The specification is parameterized on a number `n`, which corresponds to `buflen`. Finally, to deal with the fact that this function returns a status code, we simply specify that we expect a success (status code 0) as the return value in the postcondition stage of the specification.

```
let s20_encrypt32 n = do {
    (key, pkey) <- ptr_to_fresh_readonly "key" (llvm_array 32 (llvm_int␣
↪8));
    (v, pv) <- ptr_to_fresh_readonly "nonce" (llvm_array 8 (llvm_int␣
↪8));
    (m, pm) <- ptr_to_fresh "buf" (llvm_array n (llvm_int 8));

    llvm_execute_func [ pkey
                      , pv
                      , llvm_term {{ 0 : [32] }}
                      , pm
                      , llvm_term {{ `n : [32] }}
                      ];

    llvm_points_to pm (llvm_term {{ Salsa20_encrypt (key, v, m) }});
```

(continues on next page)

(continued from previous page)

```
    llvm_return (llvm_term {{ 0 : [32] }});
};
```

10.11.3 Verifying Everything

Finally, we can verify all of the functions. Notice the use of compositional verification and that path satisfiability checking is enabled for those functions with loops not bounded by explicit constants. Notice that we prove the top-level function for several sizes; this is due to the limitation that SAW can only operate on finite programs (while Salsa20 can operate on any input size.)

```
let main : TopLevel () = do {
    m      <- llvm_load_module "salsa20.bc";
    qr     <- llvm_verify m "s20_quarterround" []      false␣
↪quarterround_setup   abc;
    rr     <- llvm_verify m "s20_rowround"     [qr]    false rowround_
↪setup       abc;
    cr     <- llvm_verify m "s20_columnround"  [qr]    false␣
↪columnround_setup    abc;
    dr     <- llvm_verify m "s20_doubleround"  [cr,rr] false␣
↪doubleround_setup    abc;
    s20    <- llvm_verify m "s20_hash"         [dr]    false salsa20_
↪setup        abc;
    s20e32 <- llvm_verify m "s20_expand32"     [s20]   true  salsa20_
↪expansion_32 abc;
    s20encrypt_63 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 63) abc;
    s20encrypt_64 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 64) abc;
    s20encrypt_65 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 65) abc;

    print "Done!";
};
```

10.12 Verifying Cryptol FFI functions

SAW has special support for verifying the correctness of Cryptol's `foreign` functions (https://galoisinc.github.io/cryptol/master/FFI.html), implemented in a language such as C which compiles to LLVM, provided that there exists a reference Cryptol implementation (https://galoisinc.github.io/cryptol/master/FFI.html#cryptol-implementation-of-foreign-functions) of the function as well. Since the way in which `foreign` functions are called is precisely specified by the Cryptol FFI, SAW is able to generate a `LLVMSetup ()` spec directly from the type of a Cryptol

`foreign` function. This is done with the `llvm_ffi_setup` command, which is experimental and requires `enable_experimental;` to be run beforehand.

- `llvm_ffi_setup : Term -> LLVMSetup ()`

For instance, for the simple imported Cryptol foreign function `foreign add : [32] -> [32] -> [32]` we can obtain a `LLVMSetup` spec simply by writing

```
let add_setup = llvm_ffi_setup {{ add }};
```

which behind the scenes expands to something like

```
let add_setup = do {
  in0 <- llvm_fresh_var "in0" (llvm_int 32);
  in1 <- llvm_fresh_var "in1" (llvm_int 32);
  llvm_execute_func [llvm_term in0, llvm_term in1];
  llvm_return (llvm_term {{ add in0 in1 }});
};
```

10.12.1 Polymorphism

In general, Cryptol `foreign` functions can be polymorphic, with type parameters of kind `#` (number types), representing e.g. the sizes of input sequences. However, any individual `LLVMSetup ()` spec only specifies the behavior of the LLVM function on inputs of concrete sizes. We handle this by allowing the argument term of `llvm_ffi_setup` to contain any necessary type arguments in addition to the Cryptol function name, so that the resulting term is monomorphic. The user can then define a parameterized specification simply as a SAWScript function in the usual way. For example, for a function `foreign f : {n, m} (fin n, fin m) => [n][32] -> [m][32]`, we can obtain a parameterized `LLVMSetup` spec by

```
let f_setup (n : Int) (m : Int) = llvm_ffi_setup {{ f`{n, m} }};
```

Note that the `Term` parameter that `llvm_ffi_setup` takes is restricted syntactically to the format described above (`{{ fun`{tyArg0, tyArg1, ..., tyArgN} }}`), and cannot be any arbitrary term.

10.12.2 Supported types

`llvm_ffi_setup` supports all Cryptol types that are supported by the Cryptol FFI, with the exception of `Integer`, `Rational`, `Z`, and `Float`. `Integer`, `Rational`, and `Z` are not supported since they are translated to `gmp` arbitrary-precision types which are hard for SAW to handle without additional overrides. There is no fundamental obstacle to supporting `Float`, and in fact `llvm_ffi_setup` itself does work with Cryptol floating point types, but the underlying functions such as `llvm_fresh_var` do not, so until that is implemented `llvm_ffi_setup` can generate a spec involving floating point types but it cannot actually be run.

Note also that for the time being only the `c` calling convention is supported. Support for the recently-added `abstract` calling convention has not been written yet. See issue #2546 (https://github.com/GaloisInc/saw-script/issues/2546).

10.12.3 Performing the verification

The resulting `LLVMSetup ()` spec can be used with the existing `llvm_verify` function to perform the actual verification. And the `LLVMSpec` output from that can be used as an override as usual for further compositional verification.

```
f_ov <- llvm_verify mod "f" [] true (f_setup 3 5) z3;
```

As with the Cryptol FFI itself, SAW does not manage the compilation of the C source implementations of `foreign` functions to LLVM bitcode. For the verification to be meaningful, is expected that the LLVM module passed to `llvm_verify` matches the compiled dynamic library actually used with the Cryptol interpreter. Alternatively, on x86_64 Linux, SAW can perform verification directly on the `.so` ELF file with the experimental `llvm_verify_x86` command.

10.13 Multi-Step Proofs and Loop Invariants with Cuts

The LLVM backend (and so far, only the LLVM backend) has the ability to divide a function under verification into two (or more) parts and apply assertions at the middle point as well as the beginning and end. This is called a "cut", after the similar operation in proof derivations, and we call the point of division a "cutpoint".

This feature is highly experimental (not to mention complicated and confusing) and should be approached with caution.

At the code level, the cutpoint is a call to an empty placeholder function with a magic name. This does require modifying the code, although not in a way that has any material effect.

When verifying, SAW recognizes the magic name when loading the LLVM module and transforms the code by moving the rest of the code in the real function to the placeholder function and replacing it with a call to the placeholder.

Then one may write SAW specifications for the original function name (representing the whole original function) and for the placeholder function (representing only the second part of it).

The magic names SAW recognizes and applies this feature to are any function names beginning with `__cutpoint__`.

A simple example of a two-stage function, adapted from SAW's test suite (see `intTests/test_cutpoint`):

```
#include <stdlib.h>

extern size_t __cutpoint__add2(size_t *);
```

(continues on next page)

(continued from previous page)

```
size_t add2(size_t x) {
  ++x;
  __cutpoint__add2(&x);
  ++x;
  return x;
}
```

This function adds two to its argument in separate steps, and we'll verify each half separately.

Note that the function `__cutpoint_add2` does not actually exist in this example, and does not need to for verification. It only exists to mark the cut. (If you wanted to run this code, you'd provide an empty implementation that does nothing.)

The corresponding SAW specifications are, first for `add2`:

```
let add2_spec = do {
  x <- llvm_fresh_var "x" (llvm_int 64);
  llvm_execute_func [llvm_term x];
  llvm_return (llvm_term {{ x + 2 }});
};
```

And for `__cutpoint__add2`:

```
let cutpoint_add2_spec = do {
  p <- llvm_alloc (llvm_int 64);
  x <- llvm_fresh_var "x" (llvm_int 64);
  llvm_points_to p (llvm_term x);
  llvm_execute_func [p];
  llvm_return (llvm_term {{ x + 1 }});
};
```

There are several things to note here. First, the spec for `add2` spans the whole function: the return value it specifies is the overall return value, not the value of `x` at the cutpoint. There is no direct way to address the value of `x` at the cutpoint in the spec for the original function. The spec for the cutpoint function receives that value of `x`, and can write assertions about it, but there is in turn no easy way to get at the original value of `x` as of entry to `add2`. (The only way to do so is to change the code around to preserve it in another variable, which is almost always undesirable.) The verification will check that that the value of `x` as of the cutpoint satisfies the precondition given in the cutpoint function spec. However, for complex code these limitations may make writing that precondition difficult or impossible. (We might extend or alter the behavior in the future to make it possible. As noted above the feature is experimental; it has some sharp edges, of which this is one.)

Second, the spec for `cutpoint_add2` always covers the entire remainder of the function. If you use two cutpoints to divide a function into three parts, the first spec spans the whole function, the second

spans the second and third part, and the third spans only the third part. There is (currently) no way to flip this around so the first spec spans just the first part of the function and the last spans the whole function.

Third, the value of x is passed to the cutpoint through a pointer. This is necessary to make the propagation of the values through the transformed code work properly. It is also necessary to pass the values of all live variables, via pointers, to the cutpoint function, in the order they appear in the LLVM stack frame. Determining which variables these are and what order they appear in may require disassembling the LLVM bitcode (with `llvm-dis` or SAW's `:llvmdis` REPL command) and inspecting it. As of this writing, if any of this is not correct, SAW crashes deep inside the logic that performs the code transformation, and, alas, with a fairly mysterious message about being unable to find register values.

Now, to verify this function you do the following, assuming that `m` is the LLVM module:

```
cp <- llvm_verify m "__cutpoint__add2#add2" [] true cutpoint_add2_spec␣
↪z3;
llvm_verify m "add2" [cp] true add2_spec z3;
```

That is: first you verify the cutpoint function against its spec, then you use it as an override when verifying the original function.

Note the extra bit in the name in the first `llvm_verify` call: you must provide the name of the parent function after a `#`. SAW needs this to find the transformed code.

10.13.1 Applying Cuts Under Conditionals

If you place a cutpoint in a block that is conditionally executed (e.g. inside an if whose control is not constant) the cutpoint function still continues for the entire remainder of the function. Thus, the spec for the cutpoint function must include the behavior of all the code that occurs after it, not just the contents of the conditional block. However, it does not need to reason about the other branch of the conditional at all.

Similarly, if you place cutpoints in both sides of a conditional, both cutpoint specs must cover the entire remainder of the function. However, in some situations this can allow being more specific about what happens than is easily done without the cutpoints.

This can sometimes be helpful for handling functions with diamond-shaped control flow.

You cannot, unfortunately, splice the execution back together after the conditionals have ended.

Note that while the spec for the whole function must still give some overall return value, that characterizes what happens after both sides of the conditional, it need not necessarily be fully specific about it.

10.13.2 Using Cut with Loops

While dividing a large or complicated sequential function into multiple parts can be useful in its own right, the real purpose of the cut feature is to allow reasoning about loops by providing loop invariants.

When you place a cutpoint in the condition of a loop, the cutpoint function becomes the rest of the original function… which is to say, it executes the loop body and then comes back to the loop condition. In particular, the loop is converted into recursion and the recursive call becomes another call to the cutpoint function. This makes the precondition of the cutpoint function a loop invariant.

It also means you can escape SAW's restriction about loops being concretely bounded. The trick is to first assume the spec for the cutpoint function, then use that as an override when proving it. The proof will check that the loop body correctly assumes and then restores the loop invariant, and the override using the assumed version will let SAW conclude that the loop invariant remains true after the recursive call without having to execute through it again.

However, beware: this technique only provides partial correctness. It does not allow reasoning about the termination of the loop. It allows you to prove that *if* the loop terminates, the original function returns according to the spec. If the loop doesn't terminate, it doesn't terminate.

Here is an example, also adapted from the test suite:

```
#include <stdlib.h>

extern size_t __cutpoint__inv(size_t*, size_t*, size_t*) __attribute__
↪((noduplicate));

size_t count_n(size_t n) {
  size_t c = 0;
  for (size_t i = 0; __cutpoint__inv(&n, &c, &i), i < n; ++i) {
    ++c;
  }
  return c;
}
```

This rather naively counts up to `n`. SAW can't handle this function by default, or at least not in general: you can prove it works for any given value of `n`, but proving a general theorem about it for all `n` requires it to try the loop all `n` times. (Because `n` is a bitvector, that's at least not infinitely many times; however, because it's a 64-bit bitvector, it is too many to check in any effective amount of time.)

Instead we can write this spec for the cutpoint:

```
let ptr_to_fresh n ty = do {
  p <- llvm_alloc ty;
  x <- llvm_fresh_var n ty;
  llvm_points_to p (llvm_term x);
```

(continues on next page)

(continued from previous page)

```
  return (p, x);
};

let inv_spec = do {
  (pn, n) <- ptr_to_fresh "n" (llvm_int 64);
  (pc, c) <- ptr_to_fresh "c" (llvm_int 64);
  (pi, i) <- ptr_to_fresh "i" (llvm_int 64);
  llvm_precond {{ 0 <= i /\ i <= n }};
  llvm_execute_func [pn, pc, pi];
  llvm_return (llvm_term {{ c + (n - i) }});
};
```

This is about what one would expect: it takes all three of the live variables as pointer arguments, and the loop invariant is the loop condition extended to the case where the loop terminates.

The return value seems slightly unexpected; surely the return value should be just `n`? If you try that, it doesn't work. But that's because the loop invariant isn't strong enough; it doesn't say anything about the relationship of `c` and `i`.

What's here is true even if they are out of sync: when the loop terminates (if it terminates), `i` is equal to `n` and we return `c`, whatever it is, which is what the C code does.

One can also write this spec:

```
let inv_spec = do {
  (pn, n) <- ptr_to_fresh "n" (llvm_int 64);
  (pc, c) <- ptr_to_fresh "c" (llvm_int 64);
  (pi, i) <- ptr_to_fresh "i" (llvm_int 64);
  llvm_precond {{ 0 <= i /\ i <= n /\ c == i }};
  llvm_execute_func [pn, pc, pi];
  llvm_return (llvm_term {{ n }});
};
```

and this will also verify.

However, in either case this spec for the original function will still verify successfully:

```
let count_n_spec = do {
  n <- llvm_fresh_var "n" (llvm_int 64);
  llvm_execute_func [llvm_term n];
  llvm_return (llvm_term n);
};
```

Verifying this function only reaches the loop when `c` and `n` are equal, and that's sufficient for SAW to show that, because `i` is equal to `n` when the loop terminates (if it terminates), `c` is also equal to `n`.

Choosing the correct loop invariant is a sometimes subtle art. (It is not in any way SAW-specific and much has been written about it elsewhere.)

Note however that as with the sequential usage, the whole rest of the function after the loop belongs to the cutpoint function; if there is code after the loop, the cutpoint spec needs to account for it.

This is how you actually run the verification:

```
inv <- llvm_unsafe_assume_spec m "__cutpoint__inv#count_n" inv_spec;
llvm_verify m "__cutpoint__inv#count_n" [inv] false inv_spec abc;
llvm_verify m "count_n" [inv] false count_n_spec abc;
```

As described above, first you assume the spec for the cutpoint function, then you verify it using the assumed version as an override for the recursive call. (This is a proof by induction: the code leading up to the loop is the base case that establishes the inductive hypothesis, which is the loop invariant; then the loop body is the inductive case. Then for any finite number of iterations the loop invariant holds, and so it holds when the loop terminates… if it terminates.)

Notice that the cutpoint function is labeled `noduplicate`; this prevents LLVM from doing optimizations that might break SAW's ability to do the cut transform.

CHAPTER 11

Verifying Hardware Using Yosys

SAW has experimental support for analysis of hardware descriptions written in VHDL (via GHDL (https://github.com/ghdl/ghdl-yosys-plugin)) through an intermediate representation produced by Yosys (https://yosyshq.net/yosys/). This generally follows the same conventions and idioms used in the rest of SAWScript.

11.1 Processing VHDL With Yosys

Given a VHDL file `test.vhd` containing an entity `test`, one can generate an intermediate representation `test.json` suitable for loading into SAW:

```
$ ghdl -a test.vhd
$ yosys
...
Yosys 0.10+1 (git sha1 7a7df9a3b4, gcc 10.3.0 -fPIC -Os)
yosys> ghdl test

1. Executing GHDL.
Importing module test.

yosys> write_json test.json

2. Executing JSON backend.
```

It can sometimes be helpful to invoke additional Yosys passes between the `ghdl` and `write_json` commands. For example, at present SAW does not support the `$pmux` cell type. Yosys is able to

convert `$pmux` cells into trees of `$mux` cells using the `pmuxtree` command. We expect there are many other situations where Yosys' considerable library of commands is valuable for pre-processing.

11.2 Example: Ripple-Carry Adder

Consider three VHDL entities. First, a half-adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity half is
  port (
    a : in std_logic;
    b : in std_logic;
    c : out std_logic;
    s : out std_logic
  );
end half;

architecture halfarch of half is
begin
  c <= a and b;
  s <= a xor b;
end halfarch;
```

Next, a one-bit adder built atop that half-adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity full is
  port (
    a : in std_logic;
    b : in std_logic;
    cin : in std_logic;
    cout : out std_logic;
    s : out std_logic
  );
end full;

architecture fullarch of full is
  signal half0c : std_logic;
  signal half0s : std_logic;
  signal half1c : std_logic;
```

(continues on next page)

(continued from previous page)

```
begin
  half0 : entity work.half port map (a => a, b => b, c => half0c, s =>␣
↪half0s);
  half1 : entity work.half port map (a => half0s, b => cin, c => half1c,
↪ s => s);
  cout <= half0c or half1c;
end fullarch;
```

Finally, a four-bit adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity add4 is
  port (
    a : in std_logic_vector(3 downto 0);
    b : in std_logic_vector(3 downto 0);
    res : out std_logic_vector(3 downto 0)
  );
end add4;

architecture add4arch of add4 is
  signal full0cout : std_logic;
  signal full1cout : std_logic;
  signal full2cout : std_logic;
  signal ignore : std_logic;
begin
  full0 : entity work.full port map (a => a(0), b => b(0), cin => '0',␣
↪cout => full0cout, s => res(0));
  full1 : entity work.full port map (a => a(1), b => b(1), cin =>␣
↪full0cout, cout => full1cout, s => res(1));
  full2 : entity work.full port map (a => a(2), b => b(2), cin =>␣
↪full1cout, cout => full2cout, s => res(2));
  full3 : entity work.full port map (a => a(3), b => b(3), cin =>␣
↪full2cout, cout => ignore, s => res(3));
end add4arch;
```

Using GHDL and Yosys, we can convert the VHDL source above into a format that SAW can import. If all of the code above is in a file `adder.vhd`, we can run the following commands:

```
$ ghdl -a adder.vhd
$ yosys -p 'ghdl add4; write_json adder.json'
```

Side Note: this assumes the ghdl plugin was properly installed, see

https://github.com/ghdl/ghdl-yosys-plugin for installation details.

The produced file `adder.json` can then be loaded into SAW with `yosys_import`.

> **Side Note:** different versions of Yosys and GHDL may produce slightly different naming for the inner modules. TabbyCAD may produce different inner module names as well, so you would need to adjust the script below accordingly to account for different module names.
>
> The generated module names are not guaranteed to be valid Cryptol field names, so it is a good idea to sanity check your JSON file before loading, and at least removing parenthesis from the names with sed: `sed -i '' 's/)//g' adder.json && sed -i '' 's/(//g' adder.json` This preprocessing is needed until #2841 (https://github.com/GaloisInc/saw-script/issues/2841) is fixed.

```
$ saw
...
sawscript> enable_experimental
sawscript> m <- yosys_import "adder.json"
sawscript> :type m
Term
sawscript> print (type m)
[23:57:14.492] {add4 : {a : [4], b : [4]} -> {res : [4]},
 full_Bfullarch : {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
 ↪,
 half_Bhalfarch : {a : [1], b : [1]} -> {c : [1], s : [1]}}
```

`yosys_import` returns a `Term` with a Cryptol record type, where the fields correspond to each VHDL module. We can access the fields of this record like we would any Cryptol record, and call the functions within like any Cryptol function.

```
sawscript> print (type {{ m.add4 }})
[00:00:25.255] {a : [4], b : [4]} -> {res : [4]}
sawscript> print (eval_int {{ (m.add4 { a = 1, b = 2 }).res }})
[00:02:07.329] 3
```

We can also use all of SAW's infrastructure for asking solvers about `Term`s, such as the `sat` and `prove` commands. For example:

```
sawscript> sat w4 {{ m.add4 === \_ -> { res = 5 } }}
[00:04:41.993] Sat: [_ = (5, 0)]
sawscript> prove z3 {{ m.add4 === \inp -> { res = inp.a + inp.b } }}
[00:05:43.659] Valid
sawscript> prove yices {{ m.add4 === \inp -> { res = inp.a - inp.b } }}
[00:05:56.171] Invalid: [_ = (8, 13)]
```

The full library of `ProofScript` tactics is available in this setting. If necessary, proof tactics like

`simplify` can be used to rewrite goals before querying a solver.

Special support is provided for the common case of equivalence proofs between HDL modules and other `Term`s (e.g. Cryptol functions, other HDL modules, or "extracted" imperative LLVM or JVM code). The command `yosys_verify` has an interface similar to `llvm_verify`: given a specification, some lemmas, and a proof tactic, it produces evidence of a proven equivalence that may be passed as a lemma to future calls of `yosys_verify`. For example, consider the following Cryptol specifications for one-bit and four-bit adders:

```
cryfull :  {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
cryfull inp = { cout = [cout], s = [s] }
  where [cout, s] = zext inp.a + zext inp.b + zext inp.cin

cryadd4 : {a : [4], b : [4]} -> {res : [4]}
cryadd4 inp = { res = inp.a + inp.b }
```

If the Cryptol code above is in a file named `adder.cry`, we can prove equivalence between `cryfull` and the VHDL `full` module:

```
sawscript> import "adder.cry"
sawscript> full_spec <- yosys_verify {{ m.full }} [] {{ cryfull }} []␣
↪w4;
```

The result `full_spec` can then be used as an "override" when proving equivalence between `cryadd4` and the VHDL `add4` module:

```
sawscript> add4_spec <- yosys_verify {{ m.add4 }} [] {{ cryadd4 }}␣
↪[full_spec] w4;
```

The above could also be accomplished through the use of `prove_print` and term rewriting, but it is much more verbose.

`yosys_verify` may also be given a list of preconditions under which the equivalence holds. For example, consider the following Cryptol specification for `full` that ignores the `cin` bit (save them into a `cryfullnocarry.cry` file):

```
cryfullnocarry :  {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
cryfullnocarry inp = { cout = [cout], s = [s] }
  where [cout, s] = zext inp.a + zext inp.b
```

This is not equivalent to `full` in general, but it is if constrained to inputs where `cin = 0`. We may express that precondition like so:

```
sawscript> import "cryfullnocarry.cry"
sawscript> full_nocarry_spec <- yosys_verify {{ m.full_Bfullarch }} [{{\
↪(inp : {a : [1], b : [1], cin : [1]}) -> inp.cin == 0}}] {{␣
```

(continues on next page)

(continued from previous page)

```
↪cryfullnocarry }} [] w4;
```

The resulting override `full_nocarry_spec` may still be used in the proof for `add4` (this is accomplished by rewriting to a conditional expression).

11.3 API Reference

The following commands must first be enabled using `enable_experimental`.

- `yosys_import : String -> TopLevel Term` produces a `Term` given the path to a JSON file produced by the Yosys `write_json` command. The resulting term is a Cryptol record, where each field corresponds to one HDL module exported by Yosys. Each HDL module is in turn represented by a function from a record of input port values to a record of output port values. For example, consider a Yosys JSON file derived from the following VHDL entities:

```
entity half is
  port (
    a : in std_logic;
    b : in std_logic;
    c : out std_logic;
    s : out std_logic
  );
end half;

entity full is
  port (
    a : in std_logic;
    b : in std_logic;
    cin : in std_logic;
    cout : out std_logic;
    s : out std_logic
  );
end full;
```

  The resulting `Term` will have the type:

```
{ half : {a : [1], b : [1]} -> {c : [1], s : [1]}
, full : {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
}
```

- `yosys_verify : Term -> [Term] -> Term -> [YosysTheorem] -> ProofScript () -> TopLevel YosysTheorem` proves equality between an HDL module and a specification. The first parameter is the HDL module - given a record `m` from `yosys_import`,

this will typically look something like `{{ m.foo }}`. The second parameter is a list of preconditions for the equality. The third parameter is the specification, a term of the same type as the HDL module, which will typically be some Cryptol function or another HDL module. The fourth parameter is a list of "overrides", which witness the results of previous `yosys_verify` proofs. These overrides can be used to simplify terms by replacing use sites of submodules with their specifications.

Note that `Term`s derived from HDL modules are "first class", and are not restricted to `yosys_verify`: they may also be used with SAW's typical `Term` infrastructure like `sat`, `prove_print`, term rewriting, etc. `yosys_verify` simply provides a convenient and familiar interface, similar to `llvm_verify` or `jvm_verify`.

CHAPTER 12

The SAWScript Language

SAWScript is an application-level scripting language used for scripting proof developments, running proofs, assembling specifications that relate code to Cryptol models, and also applying proof tactics and scripting individual proofs. It is not itself either a proof language or a specification language: proofs themselves are expressed in SAWCore, and for the most part the models that underlie specifications are written in Cryptol. (The role of SAWScript in code specifications is to relate program elements to Cryptol fragments, not to write the specifications directly.)

SAWScript is a strongly typed language, and a pure functional language, with monads for sequencing execution. There are five monads in SAWScript; these are all built in to the language and more cannot be created. These monads and their purposes are:

- `TopLevel`: start proofs, control things, operate SAW
- `ProofScript`: apply tactics and solve proof goals
- `LLVMSetup`: assemble specifications for functions imported from LLVM bitcode
- `JVMSetup`: assemble specifications for functions imported from Java byte code
- `MIRSetup`: assemble specifications for functions imported via Rust MIR

Each of these essentially provides a different SAWScript environment for the stated purpose. The language is the same for each, but the primitive functions in each monad that one can use are different in each. (The `LLVMSetup`, `JVMSetup`, and `MIRSetup` environments are very similar to one another, but some details vary.)

The pure (non-monadic) fragment of SAWScript is very small and has little practical use; most things one does are written in one or another of the monads.

SAWScript is normally run from files, and a file consists of a series of statements executed top-to-bottom; however, a REPL is also provided that executes one statement at a time interactively.

12.1 Lexical Structure and Basic Syntax

The syntax of SAWScript is reminiscent of functional languages such as Cryptol, Haskell and ML. In particular, functions are applied by writing them next to their arguments rather than by using parentheses and commas. Rather than writing `f(x, y)`, write `f x y`.

Comments are written as in C, Java, and Rust (among many other languages). All text from `//` until the end of a line is ignored:

```
let x = 3; // this is a comment
```

Additionally, all text between `/*` and `*/` is ignored, regardless of whether the line ends. Unlike in C, `/* */` comments nest.

```
/*
 * This is a comment
 *   /* and so is this */
 */
let x = 3;
```

Identifiers begin with a letter and, like Haskell and ML, can contain letters, digits, underscore, and single-quote.

```
let x = 3;
let x' = 4;
let x2 = 6;
```

Integer constants can be prefixed with `0x` and `0X` for hex, `0o` and `0O` for octal, or `0b` and `0B` for binary. Otherwise they are read as decimal. Floating-point constants are not currently supported.

```
let x = 0x30; /* 48 */
let x = 0o30; /* 24 */
let x = 0b11; /* 3 */
let x = 0100; /* 100 */
```

String constants are written in double quotes, and can contain conventional escape sequences like `\t` and `\n`.

```
print "Hello!\nI am a proof agent!";
```

The forms `\72`, `\o72`, and `\x72` produce the character numbered 72 in decimal, octal, and hex respectively. Character numbers are Unicode code points.

The full set of escape sequences recognized in string literals is as follows:

Table 1: Escape sequences for control characters

Number	By letter	By control-letter	By ASCII name	Notes
0		`\^@`	`\NUL`	null
1		`\^A`	`\SOH`	
2		`\^B`	`\STX`	
3		`\^C`	`\ETX`	
4		`\^D`	`\EOT`	
5		`\^E`	`\ENQ`	
6		`\^F`	`\ACK`	
7	`\a`	`\^G`	`\BEL`	beep
8	`\b`	`\^H`	`\BS`	backspace
9	`\t`	`\^I`	`\HT`	tab
10	`\n`	`\^J`	`\LF`	newline
11	`\v`	`\^K`	`\VT`	vertical tab
12	`\f`	`\^L`	`\FF`	form feed
13	`\r`	`\^M`	`\CR`	carriage return
14		`\^N`	`\SO`	
15		`\^O`	`\SI`	
16		`\^P`	`\DLE`	
17		`\^Q`	`\DC1`	
18		`\^R`	`\DC2`	
19		`\^S`	`\DC3`	
20		`\^T`	`\DC4`	
21		`\^U`	`\NAK`	
22		`\^V`	`\SYN`	
23		`\^W`	`\ETB`	
24		`\^X`	`\CAN`	
25		`\^Y`	`\EM`	
26		`\^Z`	`\SUB`	
27		`\^[`	`\ESC`	escape
28		`\^\`	`\FS`	
29		`\^]`	`\GS`	
30		`\^^`	`\RS`	
31		`\^_`	`\US`	
127			`\DEL`	delete / rubout

By the traditional usage of `^` with control characters, one might expect `\^?` to also produce DEL, but it does not, apparently because that escape sequence is not supported in Haskell.

Table 2: Other escape sequences

Sequence	Produces
`\\`	literal backslash
`\"`	double-quote character
`\'`	single-quote character
`\SP`	space (character 32)
`\123`	character 123 (in decimal)
`\o123`	character 123 in octal (83 in decimal)
`\x123`	character 123 in hex (291 in decimal)

The numeric escape sequences (the last three forms) are processed by first collecting all legal digits, then converting that to a number and rejecting numbers that are too large or illegal Unicode. Thus for example `\o339` produces an escape (character 27) followed by '9', because 9 isn't a legal octal digit. Conversely, `\xaaaaaaaaa` will fail, even though there's a possible interpretation of it as a valid code point followed by more `a`s. Digits immediately after a numeric escape can be separated from it by using the empty escape sequence `\&`.

Text enclosed in double curly braces (`{{ }}`) is treated as Cryptol code (in particular, a Cryptol expression) and parsed by the Cryptol parser. Text enclosed in curly braces and bars (`{| |}`) is parsed as a Cryptol type. Cryptol types are expressions at the SAWScript level, as discussed below.

```
let xs = {{ [1, 2, 3] }};
let t = {| Integer |};
```

The following identifiers are reserved words:

and	do	hiding	import	let	submodule	typedef
as	else	if	in	rec	then	

The following identifiers for built-in types are also currently treated as reserved words. This may change in the future, as handling them as reserved words is neither necessary nor particularly desirable.

AIG	CrucibleSetup	JavaSetup	MIRSpec	TopLevel
Bool	Int	LLVMSetup	ProofScript	Type
CFG	JVMMethodSpec	LLVMSpec	String	
CrucibleMethodSpec	JVMSpec	MIRSetup	Term	

12.2 Types

SAWScript is strongly typed, as mentioned above. The following basic types are available:

- `Bool` (booleans; values are spelled `true` and `false`)
- `Int` (unbounded integers)
- `String` (Unicode strings)

There are also the following derived types:

- Tuples. Tuple types are written as tuples of types, that is, a comma-separated list of types in parentheses. An empty pair of parentheses gives the unit type. (The unit type has one value, also written `()`; a value of type `()` contains no information. It is like `void` in C and Java.) There are no monoples (tuples of arity 1).
- Lists. List types are written with a type name in square brackets: `[Int]` is a list of integers. Lists must be homogeneous; there is only one element type for the whole list.
- Records. Record types are written with curly braces and a comma-separated list of field names and type signatures set off with colons: `{ name: String, length: Int }`. Record types are purely structural, that is, all record types with the same fields of the same types are the same type. (Note that record *values* are written with equal signs in place of the colons.)
- Functions. Function types are written with a right-arrow `->`. Functions are curried, so a function of two arguments and a function of one argument that returns a function of one argument are the same type, and function types with multiple arguments have multiple arrows. SAWScript supports named optional arguments; these arguments appear in function types with the argument name and a question mark, like `a?Int`. Functions with the same (by name and type) optional parameters are the same type, regardless of the order the parameters appear in. The implementation will print them in alphabetical order. Note that functions are not allowed to have *only* optional arguments.

As already mentioned there are five monad types:

- `TopLevel` (the type of generic SAW actions)
- `ProofScript` (the type of proof tactic actions)
- `LLVMSetup` (the type of actions producing LLVM specifications)
- `JVMSetup` (the type of actions producing JVM specifications)
- `MIRSetup` (the type of actions producing MIR specifications)

Monad types take an argument; this is the type the monad yields when it's run. For example, the type `ProofScript Int` is a computation in the ProofScript monad that produces an integer when run. (In type systems jargon, monads have kind `* -> *`.)

12.2.1 Other Built-In Types

There are numerous further built-in types used for various verification tasks.

Cryptol-related types:

- `Term` (the type of Cryptol expressions and SAWCore value terms)
- `Type` (the type of Cryptol values and SAWCore type terms)
- `CryptolModule` (a handle for a Cryptol module; see *Imports* below)

Proof-related types:

- `SatResult` (the result produced by the `sat` operation)
- `ProofResult` (the result produced by the `prove` operation)
- `Simpset` (a simplification set; see *Rewriting*)
- `Theorem` (a proved theorem)
- `Ghost` (a piece of ghost state used during verification)

Types related to LLVM verification:

- `LLVMModule` (a handle for a loaded module of LLVM bitcode)
- `LLVMType` (the type of LLVM-level types)
- `LLVMValue` (the type of LLVM-level values)
- `LLVMSpec` (a proved LLVM specification)
- `CrucibleMethodSpec` (an obsolete alternate name for `LLVMSpec`)

See *LLVM Types*.

Types related to JVM verification:

- `JavaClass` (a handle for a loaded Java byte code class)
- `JavaType` (the type of Java-level types)
- `JVMValue` (the type of Java-level values)
- `JVMSpec` (a proved JVM specification)

See *JVM Types*.

Types related to MIR verification:

- `MIRModule` (a handle for a loaded module of MIR code)
- `MIRType` (the type of MIR-level types)
- `MIRAdt` (an additional type for MIR-level algebraic data types)
- `MIRValue` (the type of MIR-level values)

- `MIRSpec` (a proved MIR specification)

See *MIR Types*.

Types related to Yosys verification:

- `YosysSequential` (a handle for a loaded Yosys sequential HDL module)
- `YosysTheorem` (a proved Yosys specification)

Other builtin types:

- `AIG` (an and-inverter graph; see *AIG Values and Proofs*)
- `CFG` (a control-flow graph)
- `FunctionProfile` (a type used by certain analysis operations)
- `ModuleSkeleton` (a type used by the LLVM skeleton feature)
- `FunctionSkeleton` (a type used by the LLVM skeleton feature)
- `SkeletonState` (a type used by the LLVM skeleton feature)
- `BisimTheorem` (a type used by the bisimulation prover; see *Bisimulaton Prover*)

12.2.2 Type Inference

SAWScript supports Hindley-Milner style type inference, including parametric polymorphism, so it is rarely necessary to provide explicit type annotations.

Some exceptions exist:

- Record inference is not supported, and functions are not row-polymorphic. Use type signatures for function arguments of record type.
- Similarly, tuple arity inference is not supported. Use type signatures for function arguments of tuple type that are accessed via tuple field selectors. (See below).

The concrete syntax `{a}` is used for forall-quantification of type schemes, following Cryptol.

12.3 Statements

Like most programming languages, SAWScript is made up of statements and expressions. (There are also declarations, which are syntactically statements.) Expressions produce a value; statements do not.

The syntactic top level of a SAWScript script is a sequence of statements, each followed by a semicolon. These are executed in order. Do-blocks, a form of expression introduced below, also consist of sequences of statements followed by semicolons. These are executed in order when the block itself is executed. A statement can be either a typedef, a let-binding, a monad-bind, or an import.

Typedefs consist of the keyword `typedef`, an identifier, an equals sign `=`, and a type name. These create aliases for existing types. For example, to make `Foo` another name for the `Int` type:

```
typedef Foo = Int;
```

The statement form of let-binding consists of the keyword `let`, an identifier, optionally a colon and a type name, and then an equal sign `=` and an expression. The expression is evaluated, purely (not monadically), and the resulting value is bound as a new variable. At the syntactic top level the resulting variable is global and is accessible everywhere. Within a do-block the scope of the variable extends to the end of that do-block. For example, `let x = 3;` binds `x` to 3, and `let x : String = 3;` produces a type error.

If more than one identifier is given, the second and subsequent identifiers are treated as the names of parameters and the result is a function declaration. The expression is not evaluated until an argument is applied for each parameter. Parameter names can be given their own type signatures by enclosing the name, a colon, and the type name in parentheses. For example:

```
let intPair (x: Int) (y: Int) = (x, y);
```

Optional named parameters are declared by providing a default value with `?=`. This example allows passing either `x` or `y` by name, or both, and each defaults to zero if not passed explicitly.

```
let intPairOpt (x?=0 : Int) (y?=0 : Int) () = (x, y);
```

An additional unit parameter is given because functions are not allowed to have *only* optional arguments.

Recursive functions can be written using the keyword `rec` in place of `let`. Groups of mutually recursive functions can be written using `rec` for the first function, and then `and` for each successive function, and then finally a terminating semicolon.

Values of tuple type can be unpacked by writing a tuple pattern (zero or more variable names or nested tuple patterns in parentheses) instead of a plain identifier. You can explicitly ignore / throw away a value by let-binding it to the reserved variable name _.

It is legal (but useless) to write a function and call it _. It is not legal to write a function that uses a tuple pattern in place of its name.

By default the outward-facing and function-internal names of an optional parameter are the same. A different name can be used internally using the following syntax:

```
let intPairOpt (x@a ?= 0 : Int) (y@b ?= 0 : Int) () = (a, b);
```

The right-hand side of the `@` can be any pattern, such as a tuple, or _ to ignore a value.

The default value of an optional parameter will normally be a constant; however, you can use any expression. Using functions that are effectful is not recommended, and the resulting observable behavior may not be consistent between different releases of SAW. Default values for optional parameters of

recursive functions can call the function (or any function in the same group) recursively if needed. This too is not recommended, if only because it is confusing to reason about.

Binding a variable name that already exists will, in general, create a new variable for the new value; the old value is left alone and references to it are unchanged.

There is an exception to this: global variables at the syntactic top level can be bound with the special variant syntax `let rebindable ...`. These variables can then be updated by a subsequent `let rebindable`; in effect they are mutable globals. Updates must be with the same type, which must be monomorphic. See Issue 1646 (https://github.com/GaloisInc/saw-script/issues/1646) for further discussion and the history of this (somewhat dubious) feature. Note that `rec rebindable` is not permitted, and variables bound with `<-` cannot be rebindable either. Rebinding a `rebindable` variable with a *non*-rebindable definition masks it with a new immutable version, but does *not* update it.

One can also write `let {{ ... }};`, in which the double-braces can contain not just an expression but any Cryptol declaration or let-binding. This is a convenient way to insert Cryptol type declarations; it also allows using Cryptol binding patterns that cannot readily be expressed directly in SAWScript.

12.3.1 Monad Bind

Monad bind is akin to let-binding, except that it requires an expression of monad type. The syntax is a variable name, a left arrow `<-`, and an expression. The expression is first evaluated purely; then, roughly speaking, the resulting monadic action is executed in its monad, producing a value which is stored in the variable name.

Precisely speaking, the monad action is bound into the chain of actions associated with the current do-block (the syntactic top level being functionally an arbitrary long do-block) and the actions do not actually *happen* until that block, and the block it is bound into, and so forth, and eventually the actions at the syntactic top level, are all actually executed by the SAWScript interpreter.

If you are not familiar with the use of monads to sequence actions and carry state around, we recommend you seek out one of the many explanations available online.

Note however that SAWScript is unlike Haskell in that it executes eagerly: expressions are evaluated when they are encountered, as in conventional languages, rather than only when their values are used somewhere. Only the execution of monadic actions is delayed.

Monad bind can also use _ or tuple patterns on the left hand side. Also, leaving the variable name and the arrow off is entirely equivalent to using _; that is, `e;` is the same as `_ <- e;`.

The scopes of monad-bound variables are the same as the scopes of let-bound variables: they go out of scope at the end of the current do-block, and are global at the syntactic top level.

12.3.2 Imports

Import statements import Cryptol code. (To bring in more SAWScript code instead, use `include` or `include_once`. See the next section.)

An import statement begins with the keyword `import` followed by a module name. (This can be the file name in double quotes, typically including the `.cry` suffix. It can also be an unquoted Cryptol module name, with qualifiers as needed.)

The module name may be followed with `as` and another module name; this performs a `qualified import` where the definitions in the module are accessible using the given alternate name as a prefix. (Without this, they are imported directly into the current Cryptol namespace, much as in ordinary Cryptol code.) For example, if `Foo.cry` contains a definition `Bar`, after `import "Foo.cry" as Foo;` that definition is accessible as `Foo::Bar`. After just `import "Foo.cry";` it is accessible unqualified as `Bar`.

Any qualifier name may then be followed by a comma-separated list of names, in parentheses; only these definitions are imported and all others are ignored. Alternatively, if the parenthesized list is preceded by the keyword `hiding`, the named definitions are the ones ignored and the rest are imported.

An alternative way to import Cryptol modules is to use the `cryptol_load` built-in action, which returns a handle of type `CryptolModule`. This works the same as a qualified import: after `Foo <- cryptol_load "Foo.cry";` one can refer to definitions in the module using `Foo` as a qualifier (e.g. `Foo::Bar`). One can also look up definitions explicitly using the `cryptol_extract` builtin: `bar <- cryptol_extract Foo "Bar";`.

12.3.3 Includes

Additional SAWScript code can be loaded with the `include` and `include_once` statements.

`include` takes a string constant and loads and executes a file of SAWScript code. The code is run in the current context and scope; e.g. it can be inside a function or do-block. Beware that as of this writing executing things at the syntactic top level (e.g. the syntactic top level of an included file) from inside a nested context can have odd effects.

Note the terminology: `import` is for bringing in Cryptol, `include` is for bringing in more SAWScript.

`include_once` is like `include`, except that if the same file has already been included it does nothing. (The file is the "same" based on the filename, absolutized via the Haskell equivalent of `realpath`, and not on OS-level markers of file identity.)

SAWScript does not have a module system and there is no more structured way to load SAWScript code.

12.4 Expressions

Base expressions include constants (integer constants, string constants, the unit value `()`, and the empty list `[]`), Cryptol expressions in double curly braces, Cryptol types enclosed in `{| |}`, variable lookups including names of builtins, and subexpressions in parentheses.

Comma-separated lists of two or more expressions in parentheses are tuple literals and produce tuple values. List literals are comma-separated expressions enclosed in square brackets `[]`. Record literals are comma-separated lists in single braces `{ }`, where each field is assigned with a field name, an equal sign =, and a subexpression. (As mentioned above, record *types* have a similar syntax but use colons instead of equal signs.)

```
let data = {
   name = "John Smith",
   address = "55 Elm St.",
   city = "Portland",
   state = "Oregon",
   phone = "555-1234"
};
```

An expression followed by a dot and an identifier looks up the so-named field in a record value: `data.phone`. An expression followed by a dot and an integer constant looks up the nth field of a tuple value. Tuple indexes are zero-based.

Juxtaposition of expressions does function application, like in Haskell and ML: `f x` applies the function `f` to an argument `x`. Since functions are curried, applying some of the arguments to a function produces a function/closure that still expects the rest. (This is called *partial application*.)

SAWScript supports optional named arguments; `f a=x` applies the function `f` to the argument `x`, treating it as the named argument with name `a`. If `x` is a complex expression that requires parentheses, you can extend the parentheses to include the argument name as well, for tidiness: `f (a=x)`. Optional arguments can be applied in any order (with respect to each other and to positional arguments), and, obviously, can be left off if desired. Functions with optional arguments can also be partially applied. However, the call site that applies the last positional argument completes the call. There can be further optional arguments after the last positional argument in that call site; however, once those are applied the call is done, any non-supplied optional arguments are set to their default values, and the function body is evaluated.

There are no infix operators. Arithmetic (and computation on boolean values) is done in Cryptol blocks.

There are four kinds of compound expressions: let-bindings, lambdas, ifs, and do-blocks.

The expression forms of let-binding are the same as the statement forms (they can be single values, functions, recursive systems, have tuple patterns, etc.) except that instead of having the form `let x = e;` they have the form `let x = e1 in e2`. The `e2` is another expression, which is evaluated after the value for `x` is computed, and the scope of `x` is limited to that expression.

The syntax `\` followed by a parameter list (with the same syntax as described for functions above), a right arrow, and a subexpression forms a lambda: the expression has function type and takes one or more arguments. When an argument value for each parameter has been applied, the expression on the right hand side (called the *body*) is evaluated. Lambdas are basically in-place function definitions. For example:

```
for [1, 2, 3] (\x -> print x);
```

Functions let-bound with function syntax and with lambda syntax are equivalent:

```
let f x = (0, x);
let g = \x -> (0, x);
```

If-expressions have the syntax `if e1 then e2 else e3`; the first (or *control*) expression `e1`, which must be of boolean type, is evaluated. If the result is true, `e2` is then evaluated; otherwise, `e3` is evaluated. The other expression is ignored.

Do-blocks are introduced by the keyword `do`, and consist of that keyword followed by a list of statements surrounded by braces. A do-block is an expression of monadic type, and is roughly comparable to a lambda expression; much as a lambda does not evaluate until all arguments are applied, a do-block does not evaluate until the monadic context is applied. Thus during pure evaluation do-blocks do not execute.

The last statement in a do-block is restricted to the expression-only form. It cannot bind a value to a variable; rather, the value it produces is the result value of the whole do-block.

(This will often, but not always, be the `return` combinator that just produces a value without executing anything.)

12.4.1 Language-Level Builtins

A number of things that are written with syntax in typical languages are done with builtin functions in SAWScript. These can be executed as statements where needed.

These include:

- `for` takes a list and a monadic action of type `a -> m b`, runs the action on each element of the list, and returns a list of the results.
- `caseSatResult` and `caseProofResult` take a `SatResult` or `ProofResult` value (respectively) and monadic actions for each possibility, and run the action corresponding to the case they find in the result argument. (This is a workaround for not having a `match` or `case` expression.)
- `undefined` is a "pure" value that crashes if it is reached, much like `undefined` in Haskell. It is much like throwing an exception.
- `fail` is much like `undefined` but is an action in the `TopLevel` monad and takes a user-supplied string to print. It is also much like throwing an exception.

- `fails` takes a monadic action and expects it to fail, catching and reporting the resulting failure, much like a `try`/`catch` construction. It can only catch failures that occur within the monadic action; in particular to catch the failure of a pure function, it must be enclosed in a do-block when passed to `fails`. Otherwise the failure happens evaluating the function before `fails` starts. See issue #2424 (https://github.com/GaloisInc/saw-script/issues/2424) for further discussion.

12.4.2 Library-Level Builtins

There is no SAWScript prelude as such, just a collection of builtins that resemble a standard library. Here are some of the functions available:

String Functions

- `show : {a} a -> String` computes the textual representation of its argument in the same way as `print`, but instead of displaying the value it returns it as a `String` value for later use in the program. This can be useful for constructing more detailed messages later.
- `str_concat : String -> String -> String` concatenates two `String` values, and can also be useful with `show`.
- `str_concats : sep?String -> [String] -> String` concatenates a list of `String` values, taking an optional separator string as well.

List Functions

- `concat : {a} [a] -> [a] -> [a]` takes two lists and returns the concatenation of the two. Note that this is more usually called `append`; the function more usually called `concat` is called `XXX` in SAWScript.
- `head : {a} [a] -> a` returns the first element of a list.
- `tail : {a} [a] -> [a]` returns everything except the first element.
- `length : {a} [a] -> Int` counts the number of elements in a list.
- `null : {a} [a] -> Bool` indicates whether a list is empty (has zero elements).
- `nth : {a} [a] -> Int -> a` returns the element at the given position, with `nth l 0` being equivalent to `head l`.

Calling `head` or `tail` on an empty list, or asking `nth` for an element past the end of the list, will produce a runtime error.

Timing Functions

- `time : {a} TopLevel a -> TopLevel a` runs any other `TopLevel` command and prints out the time it took to execute.
- `with_time : {a} TopLevel a -> TopLevel (Int, a)` returns both the original result of the timed command and the time taken to execute it (in milliseconds), without printing anything in the process.
- `timeout : {a} Int -> TopLevel a -> TopLevel a` takes a timeout in milliseconds and an action to run. If the action takes longer, it is stopped and `timeout` fails.
- `timeout_handle : {a} Int -> TopLevel a -> TopLevel a -> TopLevel a` is like `timeout` but takes an additional action that is run if the operation times out.

The `timeout` and `timeout_handle` builtins, like the `fails` builtin, can only handle timeouts in monadic actions. If you need a timeout for a pure function, wrap it in a do-block.

System Functions

- `get_opt` *`n`* returns the current script's command-line argument *`n`*. Argument 0 is the script name; higher indices represent later arguments.
- `get_nopts ()` returns the number of arguments given to the current script. As in the shell, the arguments to the script follow the script name on `saw`'s own command line.
- `get_env` *`name`* (where *`name`* is a `String`) returns the value of an environment variable in SAW's process environment.
- `read_bytes` *`filename`*, whose return type is `TopLevel Term`, reads a file, treating it as binary, and returns the contents as a list of bytes. The underlying type of the resulting `Term` is `[`*`n`*`][8]` for some *`n`* corresponding to the length of the file.
- `exec` *`program args input-text`* is a `TopLevel` function that runs an external program. The *`input-text`* is sent as the program's standard input. The return value is the program's resulting standard output, as a `String`. Its standard error is not captured and will print to the terminal. The list of argument strings *`args`* should include a first entry to be the program's `argv[0]`.
- `exit` *`code`*, whose full type is `Int -> TopLevel ()`, stops execution of the current script and returns the exit code *`code`* to the operating system.

Verification Builtins

There are many other builtins for doing various kinds of verification tasks. These have been introduced already, or will be introduced later, elsewhere in the manual.

See the *SAWScript Commands Reference* in the reference manual for the comprehensive list. You can also use `:search` in the REPL to find them by type.

12.5 The SAWScript REPL

The most common way to run SAWScript is from a file; however, the `saw` executable also offers an interactive read-eval-print loop ("REPL") for experimental and debugging use.

The REPL prints a `sawscript>` prompt. At this prompt one can enter a SAWScript statement, and the SAWScript interpreter will execute it. The final semicolon may be left off. If this produces a value other than `()`, the interpreter will print the value.

The value must either be of type `TopLevel a` for some type `a`, or pure (not in any monad).

It is also possible to run the REPL in the ProofScript context by using the `proof_subshell` builtin. This is an advanced topic; see *Interactive Proofs*.

The REPL also accepts some REPL-level commands that begin with `:`. The most immediately useful of these is `:type` or `:t`, which prints the type of a SAWScript expression. (For example, `:t 3` prints `Int`.) The `:h` or `:help` command by itself lists the REPL commands, and if given the name of a SAWScript value or function, prints its documentation.

The `:search` command takes one or more type names (separated by spaces; where necessary use parentheses) and searches the current variable environment for values that mention all these types.

For example, `:search String -> String` finds the `str_concat` function mentioned above.

See *the REPL reference* for further details.

CHAPTER 13

The Python Bindings

Warning

This section is under construction!

CHAPTER 14

Interactive Proofs

> **Warning**
>
> This section is under construction!

14.1 Transforming Term Values

The three primary functions of SAW are *extracting* models (`Term` values) from programs, *transforming* those models, and *proving* properties about models using external provers. We've seen how to construct `Term` values from a range of sources. Now we show how to use the various term transformation features available in SAW.

14.1.1 Rewriting

Rewriting a `Term` consists of applying one or more *rewrite rules* to it, resulting in a new `Term`. A rewrite rule in SAW can be specified in multiple ways:

- as the definition of a function that can be unfolded,
- as a term of Boolean type (or a function returning a Boolean) that is an equality statement, and
- as a term of *equality type* with a body that encodes a proof that the equality in the type is valid.

In each case the term logically consists of two sides and describes a way to transform the left side into the right side. Each side may contain variables (bound by enclosing lambda expressions) and is therefore a *pattern* which can match any term in which each variable represents an arbitrary sub-term. The left-hand pattern describes a term to match (which may be a sub-term of the full term being

rewritten), and the right-hand pattern describes a term to replace it with. Any variable in the right-hand pattern must also appear in the left-hand pattern and will be instantiated with whatever sub-term matched that variable in the original term.

For example, say we have the following Cryptol function:

```
\(x:[8]) -> (x * 2) + 1
```

We might for some reason want to replace multiplication by a power of two with a shift. We can describe this replacement using an equality statement in Cryptol (a rule of form 2 above):

```
\(y:[8]) -> (y * 2) == (y << 1)
```

Interpreting this as a rewrite rule, it says that for any 8-bit vector (call it `y` for now), we can replace `y * 2` with `y << 1`. Using this rule to rewrite the earlier expression would then yield:

```
\(x:[8]) -> (x << 1) + 1
```

The general philosophy of rewriting is that the left and right patterns, while syntactically different, should be semantically equivalent. Therefore, applying a set of rewrite rules should not change the fundamental meaning of the term being rewritten. SAW is particularly focused on the task of proving that some logical statement expressed as a `Term` is always true. If that is in fact the case, then the entire term can be replaced by the term `True` without changing its meaning. The rewriting process can in some cases, by repeatedly applying rules that themselves are known to be valid, reduce a complex term entirely to `True`, which constitutes a proof of the original statement. In other cases, rewriting can simplify terms before sending them to external automated provers that can then finish the job. Sometimes this simplification can help the automated provers run more quickly, and sometimes it can help them prove things they would otherwise be unable to prove by applying reasoning steps (rewrite rules) that are not available to the automated provers.

In practical use, rewrite rules can be aggregated into `Simpset` ("simplification set") values in SAWScript. A few pre-defined `Simpset` values exist:

- `empty_ss : Simpset` is the empty set of rules. Rewriting with it should have no effect, but it is useful as an argument to some of the functions that construct larger `Simpset` values.
- `basic_ss : Simpset` is a collection of rules that are useful in most proof scripts.
- `cryptol_ss : () -> Simpset` includes a collection of Cryptol-specific rules. Some of these simplify away the abstractions introduced in the translation from Cryptol to SAWCore, which can be useful when proving equivalence between Cryptol and non-Cryptol code. Leaving these abstractions in place is appropriate when comparing only Cryptol code, however, so `cryptol_ss` is not included in `basic_ss`.

The following function can apply a `Simpset`:

- `rewrite : Simpset -> Term -> Term` applies a `Simpset` to an existing `Term` to produce a new `Term`.

To make this more concrete, we examine how the rewriting example sketched above, to convert multiplication into shift, can work in practice. We simplify everything with `cryptol_ss` as we go along so that the `Term`s don't get too cluttered. First, we declare the term to be transformed:

```
sawscript> let term = rewrite (cryptol_ss ()) {{ \(x:[8]) -> (x * 2) +␣
↪1 }}
sawscript> print_term term
\(x : Prelude.Vec 8 Prelude.Bool) ->
  Prelude.bvAdd 8 (Prelude.bvMul 8 x (Prelude.bvNat 8 2))
    (Prelude.bvNat 8 1)
```

Next, we declare the rewrite rule:

```
sawscript> let rule = rewrite (cryptol_ss ()) {{ \(y:[8]) -> (y * 2) ==␣
↪(y << 1) }}
sawscript> print_term rule
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(y : x@1) ->
      Cryptol.ecEq x@1 (Cryptol.PCmpWord 8)
        (Prelude.bvMul 8 y (Prelude.bvNat 8 2))
        (Prelude.bvShiftL 8 Prelude.Bool 1 Prelude.False y
           (Prelude.bvNat 1 1))
```

The primary interface to rewriting uses the `Theorem` type instead of the `Term` type, as shown in the signatures for `addsimp` and `addsimps`.

- `addsimp : Theorem -> Simpset -> Simpset` adds a single `Theorem` to a `Simpset`.
- `addsimps : [Theorem] -> Simpset -> Simpset` adds several `Theorem` values to a `Simpset`.

A `Theorem` is essentially a `Term` that is proven correct in some way. In general, a `Theorem` can be any statement, and may not be useful as a rewrite rule. However, if it has an appropriate shape it can be used for rewriting. In the *"Proofs about Terms"* section, we'll describe how to construct `Theorem` values from `Term` values.

For the time being, we'll assume we've proved our `rule` term correct in some way, and have a `Theorem` named `rule_thm`.

Finally, we apply the rule to the target term:

```
sawscript> let result = rewrite (addsimp rule_thm empty_ss) term
sawscript> print_term result
\(x : Prelude.Vec 8 Prelude.Bool) ->
  Prelude.bvAdd 8
    (Prelude.bvShiftL 8 Prelude.Bool 1 Prelude.False x
```

(continues on next page)

(continued from previous page)

```
      (Prelude.bvNat 1 1))
    (Prelude.bvNat 8 1)
```

In the absence of user-constructed `Theorem` values, there are some additional built-in rules that are not included in either `basic_ss` and `cryptol_ss` because they are not always beneficial, but that can sometimes be helpful or essential. The `cryptol_ss` simpset includes rewrite rules to unfold all definitions in the `Cryptol` SAWCore module, but does not include any of the terms of equality type.

- `add_defs : [String] -> Simpset -> Simpset` adds unfolding rules for the functions with the given names to the given `Simpset`.
- `core_thm : String -> Theorem` parses a SAWCore term of type `Prop` from the SAWCore `Prelude` or any other loaded SAWCore module.
- `add_core_thms : [String] -> Simpset -> Simpset` adds equality-typed terms from the SAWCore `Prelude` or any other loaded SAWCore module to a `Simpset`.

Finally, it's possible to construct a theorem from an arbitrary SAWCore expression (rather than a Cryptol expression), using the `core_axiom` function.

- `core_axiom : String -> Theorem` creates a `Theorem` from a `String` in SAWCore syntax. Any `Theorem` introduced by this function is assumed to be correct, so use it with caution.

14.1.2 Folding and Unfolding

A SAWCore term can be given a name using the `define` function, and is then by default printed as that name alone. A named subterm can be "unfolded" so that the original definition appears again.

- `define : String -> Term -> TopLevel Term`
- `unfold_term : [String] -> Term -> Term`

For example:

```
sawscript> let t = {{ 0x22 }}
sawscript> print_term t
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
sawscript> t' <- define "t" t
sawscript> print_term t'
t
sawscript> let t'' = unfold_term ["t"] t'
sawscript> print_term t''
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
```

This process of folding and unfolding is useful both to make large terms easier for humans to work with and to make automated proofs more tractable. We'll describe the latter in more detail when we discuss interacting with external provers.

In some cases, folding happens automatically when constructing Cryptol expressions. Consider the following example:

```
sawscript> let t = {{ 0x22 }}
sawscript> print_term t
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
sawscript> let {{ t' = 0x22 }}
sawscript> print_term {{ t' }}
t'
```

This illustrates that a bare expression in Cryptol braces gets translated directly to a SAWCore term. However, a Cryptol *definition* gets translated into a *folded* SAWCore term. In addition, because the second definition of `t` occurs at the Cryptol level, rather than the SAWScript level, it is visible only inside Cryptol braces. Definitions imported from Cryptol source files are also initially folded and can be unfolded as needed.

14.1.3 Other Built-in Transformation and Inspection Functions

In addition to the `Term` transformation functions described so far, a variety of others also exist.

- `beta_reduce_term : Term -> Term` takes any sub-expression of the form `(\x -> t) v` in the given `Term` and replaces it with a transformed version of `t` in which all instances of `x` are replaced by `v`.
- `replace : Term -> Term -> Term -> TopLevel Term` replaces arbitrary subterms. A call to `replace x y t` replaces any instance of `x` inside `t` with `y`.

Assessing the size of a term can be particularly useful during benchmarking. SAWScript provides two mechanisms for this.

- `term_size : Term -> Int` calculates the number of nodes in the Directed Acyclic Graph (DAG) representation of a `Term` used internally by SAW. This is the most appropriate way of determining the resource use of a particular term.
- `term_tree_size : Term -> Int` calculates how large a `Term` would be if it were represented by a tree instead of a DAG. This can, in general, be much, much larger than the number returned by `term_size`, and serves primarily as a way of assessing, for a specific term, how much benefit there is to the term sharing used by the DAG representation.

Finally, there are a few commands related to the internal SAWCore type of a `Term`.

- `check_term : Term -> TopLevel ()` checks that the internal structure of a `Term` is well-formed and that it passes all of the rules of the SAWCore type checker.

- `type : Term -> Type` returns the type of a particular `Term`, which can then be used to, for example, construct a new fresh variable with `fresh_symbolic`.

14.1.4 Loading and Storing Terms

Most frequently, `Term` values in SAWScript come from Cryptol, JVM, or LLVM programs, or some transformation thereof. However, it is also possible to obtain them from various other sources.

- `parse_core : String -> Term` parses a `String` containing a term in SAWCore syntax, returning a `Term`.
- `read_core : String -> TopLevel Term` is like `parse_core`, but obtains the text from the given file and expects it to be in the simpler SAWCore external representation format, rather than the human-readable syntax shown so far.
- `read_aig : String -> TopLevel Term` returns a `Term` representation of an And-Inverter-Graph (AIG) file in AIGER format.
- `read_bytes : String -> TopLevel Term` reads a constant sequence of bytes from a file and represents it as a `Term`. Its result will always have Cryptol type `[n][8]` for some `n`.

It is also possible to write `Term` values into files in various formats, including: AIGER (`write_aig`), CNF (`write_cnf`), SAWCore external representation (`write_core`), and SMT-Lib version 2 (`write_smtlib2`).

- `write_aig : String -> Term -> TopLevel ()`
- `write_cnf : String -> Term -> TopLevel ()`
- `write_core : String -> Term -> TopLevel ()`
- `write_smtlib2 : String -> Term -> TopLevel ()`

14.2 Proofs about Terms

The goal of SAW is to facilitate proofs about the behavior of programs. It may be useful to prove some small fact to use as a rewrite rule in later proofs, but ultimately these rewrite rules come together into a proof of some higher-level property about a software system.

Whether proving small lemmas (in the form of rewrite rules) or a top-level theorem, the process builds on the idea of a *proof script* that is run by one of the top level proof commands.

- `prove_print : ProofScript () -> Term -> TopLevel Theorem` takes a proof script (which we'll describe next) and a `Term`. The `Term` should be of function type with a return value of `Bool` (`Bit` at the Cryptol level). It will then use the proof script to attempt to show that the `Term` returns `True` for all possible inputs. If it is successful, it will print `Valid` and return a `Theorem`. If not, it will abort.

- `sat_print : ProofScript () -> Term -> TopLevel ()` is similar except that it looks for a *single* value for which the `Term` evaluates to `True` and prints out that value, returning nothing.
- `prove_core : ProofScript () -> String -> TopLevel Theorem` proves and returns a `Theorem` from a string in SAWCore syntax.

14.2.1 Automated Tactics

The simplest proof scripts just specify the automated prover to use. The `ProofScript` values `abc` and `z3` select the ABC and Z3 theorem provers, respectively, and are typically good choices.

For example, combining `prove_print` with `abc`:

```
sawscript> t <- prove_print abc {{ \(x:[8]) -> x+x == x*2 }}
Valid
sawscript> t
Theorem (let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.TCNum 8
      x@3 = Cryptol.PArithSeqBool x@2
    }
 in (x : x@1)
-> Prelude.EqTrue
     (Cryptol.ecEq x@1 (Cryptol.PCmpSeqBool x@2)
        (Cryptol.ecPlus x@1 x@3 x x)
        (Cryptol.ecMul x@1 x@3 x
           (Cryptol.ecNumber (Cryptol.TCNum 2) x@1
              (Cryptol.PLiteralSeqBool x@2)))))
```

Similarly, `sat_print` will show that the function returns `True` for one specific input (which it should, since we already know it returns `True` for all inputs):

```
sawscript> sat_print abc {{ \(x:[8]) -> x+x == x*2 }}
Sat: [x = 0]
```

In addition to these, the `bitwuzla`, `boolector`, `cvc5`, `mathsat`, and `yices` provers are available. The internal decision procedure `rme`, short for Reed-Muller Expansion, is an automated prover that works particularly well on the Galois field operations that show up, for example, in AES.

In more complex cases, some pre-processing can be helpful or necessary before handing the problem off to an automated prover. The pre-processing can involve rewriting, beta reduction, unfolding, the use of provers that require slightly more configuration, or the use of provers that do very little real work.

14.2.2 Proof Script Diagnostics

During development of a proof, it can be useful to print various information about the current goal. The following tactics are useful in that context.

- `print_goal : ProofScript ()` prints the entire goal in SAWCore syntax.
- `print_goal_consts : ProofScript ()` prints a list of unfoldable constants in the current goal.
- `print_goal_depth : Int -> ProofScript ()` takes an integer argument, `n`, and prints the goal up to depth `n`. Any elided subterms are printed with a `...` notation.
- `print_goal_size : ProofScript ()` prints the number of nodes in the DAG representation of the goal.

14.2.3 Rewriting in Proof Scripts

One of the key techniques available for completing proofs in SAWScript is the use of rewriting or transformation. The following commands support this approach.

- `simplify : Simpset -> ProofScript ()` works just like `rewrite`, except that it works in a `ProofScript` context and implicitly transforms the current (unnamed) goal rather than taking a `Term` as a parameter.
- `goal_eval : ProofScript ()` will evaluate the current proof goal to a first-order combination of primitives.
- `goal_eval_unint : [String] -> ProofScript ()` works like `goal_eval` but avoids expanding or simplifying the given names.

14.2.4 Other Transformations

Some useful transformations are not easily specified using equality statements, and instead have special tactics.

- `beta_reduce_goal : ProofScript ()` works like `beta_reduce_term` but on the current goal. It takes any sub-expression of the form `(\x -> t) v` and replaces it with a transformed version of `t` in which all instances of `x` are replaced by `v`.
- `unfolding : [String] -> ProofScript ()` works like `unfold_term` but on the current goal.

Using `unfolding` is mostly valuable for proofs based entirely on rewriting, since the default behavior for automated provers is to unfold everything before sending a goal to a prover. However, with some provers it is possible to indicate that specific named subterms should be represented as uninterpreted functions.

- `unint_bitwuzla : [String] -> ProofScript ()`
- `unint_cvc5 : [String] -> ProofScript ()`

- `unint_yices : [String] -> ProofScript ()`
- `unint_z3 : [String] -> ProofScript ()`

The list of `String` arguments in these cases indicates the names of the subterms to leave folded, and therefore present as uninterpreted functions to the prover. To determine which folded constants appear in a goal, use the `print_goal_consts` function described above.

Ultimately, we plan to implement a more generic tactic that leaves certain constants uninterpreted in whatever prover is ultimately used (provided that uninterpreted functions are expressible in the prover).

Note that each of the `unint_*` tactics have variants that are prefixed with `sbv_` and `w4_`. The `sbv_`-prefixed tactics make use of the SBV library to represent and solve SMT queries:

- `sbv_unint_bitwuzla : [String] -> ProofScript ()`
- `sbv_unint_cvc5 : [String] -> ProofScript ()`
- `sbv_unint_yices : [String] -> ProofScript ()`
- `sbv_unint_z3 : [String] -> ProofScript ()`

The `w4_`-prefixed tactics make use of the What4 library instead of SBV:

- `w4_unint_bitwuzla : [String] -> ProofScript ()`
- `w4_unint_cvc5 : [String] -> ProofScript ()`
- `w4_unint_yices : [String] -> ProofScript ()`
- `w4_unint_z3 : [String] -> ProofScript ()`

In most specifications, the choice of SBV versus What4 is not important, as both libraries are broadly compatible in terms of functionality. There are some situations where one library may outperform the other, however, due to differences in how each library represents certain SMT queries. There are also some experimental features that are only supported with What4 at the moment, such as `enable_lax_loads_and_stores`.

14.2.5 Caching Solver Results

SAW has the capability to cache the results of tactics which call out to automated provers. This can save a considerable amount of time in cases such as proof development and CI, where the same proof scripts are often run repeatedly without changes.

This caching is available for all tactics which call out to automated provers at runtime: `abc`, `boolector`, `cvc5`, `mathsat`, `yices`, `z3`, `rme`, and the family of `unint` tactics described in the previous section.

When solver caching is enabled and one of the tactics mentioned above is encountered, if there is already an entry in the cache corresponding to the call then the cached result is used, otherwise the appropriate solver is queried, and the result saved to the cache. Entries are indexed by a SHA256 hash of the exact query to the solver (ignoring variable names), any options passed to the solver, and

the names and full version strings of all the solver backends involved (e.g. ABC and SBV for the `abc` tactic). This ensures cached results are only used when they would be identical to the result of actually running the tactic.

The simplest way to enable solver caching is to set the environment variable *`SAW_SOLVER_CACHE_PATH`*. With this environment variable set, `saw` and `saw-remote-api` will automatically keep an LMDB (http://www.lmdb.tech/doc/) database at the given path containing the solver result cache. Setting this environment variable globally therefore creates a global, concurrency-safe solver result cache used by all newly created `saw` or `saw-remote-api` processes. Note that when this environment variable is set, SAW does not create a cache at the specified path until it is actually needed.

There are also a number of SAW commands related to solver caching.

- `set_solver_cache_path` is like setting `SAW_SOLVER_CACHE_PATH` for the remainder of the current session, but opens an LMDB database at the specified path immediately. If a cache is already in use in the current session (i.e. through a prior call to `set_solver_cache_path` or through `SAW_SOLVER_CACHE_PATH` being set and the cache being used at least once) then all entries in the cache already in use will be copied to the new cache being opened.
- `set_solver_cache_timeout` sets the cache's timeout (in microseconds) used for database lookups and inserts. The default timeout value is 2,000,000 microseconds (2 seconds). This is a reasonably large timeout for most cache operations, but it may be convenient to increase this timeout for especially large proof goals.
- `clean_mismatched_versions_solver_cache` will remove all entries in the solver result cache which were created using solver backend versions which do not match the versions in the current environment. This can be run after an update to clear out any old, unusable entries from the solver cache. This command can also be run directly from the command line through the `--clean-mismatched-versions-solver-cache` command-line option.
- `print_solver_cache` prints to the console all entries in the cache whose SHA256 hash keys start with the given hex string. Providing an empty string results in all entries in the cache being printed.
- `print_solver_cache_stats` prints to the console statistics including the size of the solver cache, where on disk it is stored, and some counts of how often it has been used during the current session.

For performing more complicated database operations on the set of cached results, the file `solver_cache.py` is provided with the Python bindings of the SAW Remote API. This file implements a general-purpose Python interface for interacting with the LMDB databases kept by SAW for solver caching.

Below is an example of using solver caching with `saw -v Debug`. Only the relevant output is shown, the rest abbreviated with "…".

```
sawscript> set_solver_cache_path "example.cache"
sawscript> prove_print z3 {{ \(x:[8]) -> x+x == x*2 }}
```

(continues on next page)

(continued from previous page)

```
[22:13:00.832] Caching result: d1f5a76e7a0b7c01 (SBV 9.2, Z3 4.8.7 - 64␣
↪bit)
...
sawscript> prove_print z3 {{ \(new:[8]) -> new+new == new*2 }}
[22:13:04.122] Using cached result: d1f5a76e7a0b7c01 (SBV 9.2, Z3 4.8.7␣
↪- 64 bit)
...
sawscript> prove_print (w4_unint_z3_using "qfnia" []) \
                                  {{ \(x:[8]) -> x+x == x*2 }}
[22:13:09.484] Caching result: 4ee451f8429c2dfe (What4 v1.3-29-g6c462cd␣
↪using qfnia, Z3 4.8.7 - 64 bit)
...
sawscript> print_solver_cache "d1f5a76e7a0b7c01"
[22:13:13.250] SHA:␣
↪d1f5a76e7a0b7c01bdfe7d0e1be82b4f233a805ae85a287d45933ed12a54d3eb
[22:13:13.250] - Result: unsat
[22:13:13.250] - Solver: "SBV->Z3"
[22:13:13.250] - Versions: SBV 9.2, Z3 4.8.7 - 64 bit
[22:13:13.250] - Last used: 2023-07-25 22:13:04.120351 UTC

sawscript> print_solver_cache "4ee451f8429c2dfe"
[22:13:16.727] SHA:␣
↪4ee451f8429c2dfefecb6216162bd33cf053f8e66a3b41833193529449ef5752
[22:13:16.727] - Result: unsat
[22:13:16.727] - Solver: "W4 ->z3"
[22:13:16.727] - Versions: What4 v1.3-29-g6c462cd using qfnia, Z3 4.8.7␣
↪- 64 bit
[22:13:16.727] - Last used: 2023-07-25 22:13:09.484464 UTC

sawscript> print_solver_cache_stats
[22:13:20.585] == Solver result cache statistics ==
[22:13:20.585] - 2 results cached in example.cache
[22:13:20.585] - 2 insertions into the cache so far this run (0 failed␣
↪attempts)
[22:13:20.585] - 1 usage of cached results so far this run (0 failed␣
↪attempts)
```

14.2.6 Other External Provers

In addition to the built-in automated provers already discussed, SAW supports more generic interfaces to other arbitrary theorem provers supporting specific interfaces.

- `external_aig_solver : String -> [String] -> ProofScript ()` supports

theorem provers that can take input as a single-output AIGER file. The first argument is the name of the executable to run. The second argument is the list of command-line parameters to pass to that executable. Any element of this list equal to `"%f"` will be replaced with the name of the temporary AIGER file generated for the proof goal. The output from the solver is expected to be in DIMACS solution format.

- `external_cnf_solver : String -> [String] -> ProofScript ()` works similarly but for SAT solvers that take input in DIMACS CNF format and produce output in DIMACS solution format.

14.2.7 Offline Provers

For provers that must be invoked in more complex ways, or to defer proof until a later time, there are functions to write the current goal to a file in various formats, and then assume that the goal is valid through the rest of the script.

- `offline_aig : String -> ProofScript ()`
- `offline_cnf : String -> ProofScript ()`
- `offline_extcore : String -> ProofScript ()`
- `offline_smtlib2 : String -> ProofScript ()`
- `offline_unint_smtlib2 : [String] -> String -> ProofScript ()`

These support the AIGER, DIMACS CNF, shared SAWCore, and SMT-Lib v2 formats, respectively. The shared representation for SAWCore is described in the `saw-script` repository (https://github.com/GaloisInc/saw-script/blob/master/doc/extcore.md). The `offline_unint_smtlib2` command represents the folded subterms listed in its first argument as uninterpreted functions.

14.2.8 Finishing Proofs without External Solvers

Some proofs can be completed using unsound placeholders, or using techniques that do not require significant computation.

- `assume_unsat : ProofScript ()` indicates that the current goal should be assumed to be unsatisfiable. This is an alias for `assume_valid`. Users should prefer to use `admit` instead.
- `assume_valid : ProofScript ()` indicates that the current goal should be assumed to be valid. Users should prefer to use `admit` instead
- `admit : String -> ProofScript ()` indicates that the current goal should be assumed to be valid without proof. The given string should be used to record why the user has decided to assume this proof goal.
- `quickcheck : Int -> ProofScript ()` runs the goal on the given number of random inputs, and succeeds if the result of evaluation is always `True`. This is unsound, but can be

helpful during proof development, or as a way to provide some evidence for the validity of a specification believed to be true but difficult or infeasible to prove.

- `trivial : ProofScript ()` states that the current goal should be trivially true. This tactic recognizes instances of equality that can be demonstrated by conversion alone. In particular it is able to prove `EqTrue x` goals where `x` reduces to the constant value `True`. It fails if this is not the case.

14.2.9 Multiple Goals

The proof scripts shown so far all have a single implicit goal. As in many other interactive provers, however, SAWScript proofs can have multiple goals. The following commands can introduce or work with multiple goals. These are experimental and can be used only after `enable_experimental` has been called.

- `goal_apply : Theorem -> ProofScript ()` will apply a given introduction rule to the current goal. This will result in zero or more new subgoals.
- `goal_assume : ProofScript Theorem` will convert the first hypothesis in the current proof goal into a local `Theorem`
- `goal_insert : Theorem -> ProofScript ()` will insert a given `Theorem` as a new hypothesis in the current proof goal.
- `goal_intro : String -> ProofScript Term` will introduce a quantified variable in the current proof goal, returning the variable as a `Term`.
- `goal_when : String -> ProofScript () -> ProofScript ()` will run the given proof script only when the goal name contains the given string.
- `goal_exact : Term -> ProofScript ()` will attempt to use the given term as an exact proof for the current goal. This tactic will succeed whenever the type of the given term exactly matches the current goal, and will fail otherwise.
- `split_goal : ProofScript ()` will split a goal of the form `Prelude.and prop1 prop2` into two separate goals `prop1` and `prop2`.

14.2.10 Proof Failure and Satisfying Assignments

The `prove_print` and `sat_print` commands print out their essential results (potentially returning a `Theorem` in the case of `prove_print`). In some cases, though, one may want to act programmatically on the result of a proof rather than displaying it.

The `prove` and `sat` commands allow this sort of programmatic analysis of proof results. To allow this, they use two types we haven't mentioned yet: `ProofResult` and `SatResult`. These are different from the other types in SAWScript because they encode the possibility of two outcomes. In the case of `ProofResult`, a statement may be valid or there may be a counter-example. In the case of `SatResult`, there may be a satisfying assignment or the statement may be unsatisfiable.

- `prove : ProofScript SatResult -> Term -> TopLevel ProofResult`

- `sat : ProofScript SatResult -> Term -> TopLevel SatResult`

To operate on these new types, SAWScript includes a pair of functions:

- `caseProofResult : {b} ProofResult -> b -> (Term -> b) -> b` takes a `ProofResult`, a value to return in the case that the statement is valid, and a function to run on the counter-example, if there is one.
- `caseSatResult : {b} SatResult -> b -> (Term -> b) -> b` has the same shape: it returns its first argument if the result represents an unsatisfiable statement, or its second argument applied to a satisfying assignment if it finds one.

14.2.11 AIG Values and Proofs

Most SAWScript programs operate on `Term` values, and in most cases this is the appropriate representation. It is possible, however, to represent the same function that a `Term` may represent using a different data structure: an And-Inverter-Graph (AIG). An AIG is a representation of a Boolean function as a circuit composed entirely of AND gates and inverters. Hardware synthesis and verification tools, including the ABC tool that SAW has built in, can do efficient verification and particularly equivalence checking on AIGs.

To take advantage of this capability, a handful of built-in commands can operate on AIGs.

- `bitblast : Term -> TopLevel AIG` represents a `Term` as an `AIG` by "blasting" all of its primitive operations (things like bit-vector addition) down to the level of individual bits.
- `load_aig : String -> TopLevel AIG` loads an `AIG` from an external AIGER file.
- `save_aig : String -> AIG -> TopLevel ()` saves an `AIG` to an external AIGER file.
- `save_aig_as_cnf : String -> AIG -> TopLevel ()` writes an `AIG` out in CNF format for input into a standard SAT solver.

CHAPTER 15

Extraction to Rocq

In addition to the (semi-)automatic and compositional proof modes already discussed above, SAW has support for exporting Cryptol and SAWCore values as terms to the Rocq proof assistant[5]. This is intended to support more manual proof efforts for properties that go beyond what SAW can support (for example, proofs requiring induction) or for connecting to preexisting formalizations in Rocq of useful algorithms (e.g. the fiat crypto library[6]).

This support consists of two related pieces. The first piece is a library of formalizations of the primitives underlying Cryptol and SAWCore and various supporting concepts that help bridge the conceptual gap between SAW and Rocq. The second piece is a term translation that maps the syntactic forms of SAWCore onto corresponding concepts in Rocq syntax, designed to dovetail with the concepts defined in the support library. SAWCore is a quite similar language to the core calculus underlying Rocq, so much of this translation is quite straightforward; however, the languages are not exactly equivalent, and there are some tricky cases that mostly arise from Cryptol code that can only be partially supported. We will note these restrictions later in the manual.

We currently support Rocq version 9.1.0.

15.1 Support Library

In order to make use of SAW's extraction capabilities, one must first compile the support library using Rocq so that the included definitions and theorems can be referenced by the extracted code. From the top directory of the SAW source tree, the source code for this support library can be found in the `saw-core-rocq/rocq` subdirectory. In this subdirectory you will find a `_RocqProject` and a

[5] https://rocq-prover.org
[6] https://github.com/mit-plv/fiat-crypto

`Makefile`. A simple `make` invocation should be enough to compile all the necessary files, assuming Rocq is installed and `rocq` is available in the user's `PATH`. HTML documentation for the support library can also be generated by `make html` from the same directory.

Once the library is compiled, the recommended way to import it into your subsequent development is by adding the following lines to your `_RocqProject` file:

```
-Q <SAWDIR>/saw-core-rocq/rocq/generated/CryptolToRocq CryptolToRocq
-Q <SAWDIR>/saw-core-rocq/rocq/handwritten/CryptolToRocq CryptolToRocq
```

Here `<SAWDIR>` refers to the location on your system where the SAWScript source tree is checked out. This will add the relevant library files to the `CryptolToRocq` namespace, where the extraction process will expect to find them.

The support library for extraction is broken into two parts: those files which are handwritten, versus those that are automatically generated. The handwritten files are generally fairly readable and are reasonable for human inspection; they define most of the interesting pipe-fitting that allows Cryptol and SAWCore definitions to connect to corresponding Rocq concepts. In particular the file `SAWCoreScaffolding.v` file defines most of the bindings of base types to Rocq types, and the `SAWCoreVectorsAsRocqVectors.v` defines the core bitvector operations. The automatically generated files are direct translations of the SAWCore source files (`saw-core/prelude/Prelude.sawcore` and `cryptol-saw-core/saw/Cryptol.sawcore`) that correspond to the standard libraries for SAWCore and Cryptol, respectively.

The autogenerated files are intended to be kept up-to-date with changes in the corresponding `sawcore` files, and end users should not need to generate them. Nonetheless, if they are out of sync for some reason, these files may be regenerated using the `saw` executable by running `(cd saw-core-rocq; saw saw/generate_scaffolding.saw)` from the top-level of the SAW source directory before compiling them with Rocq as described above.

You may also note some additional files and concepts in the standard library, such as `CompM.v`, and a variety of lemmas and definitions related to it. These definitions are related to the "heapster" system, which form a separate use-case for the SAWCore to Rocq translation. These definitions will not be used for code extracted from Cryptol.

15.2 Cryptol module extraction

There are several modes of use for the SAW to Rocq term extraction facility, but the easiest to use is whole Cryptol module extraction. This will extract all the definitions in the given Cryptol module, together with it's transitive dependencies, into a single Rocq module which can then be compiled and pulled into subsequence developments.

Suppose we have a Cryptol source file named `source.cry` and we want to generate a Rocq file named `output.v`. We can accomplish this by running the following commands in saw (either directly from the `saw` command prompt, or via a script file)

```
write_rocq_cryptol_module "source.cry" "output.v" [] [];
```

In this default mode, identifiers in the Cryptol source will be directly translated into identifiers in Rocq. This may occasionally cause problems if source identifiers clash with Rocq keywords or preexisting definitions. The third argument to `write_rocq_cryptol_module` can be used to remap such names if necessary by giving a list of `(in,out)` pairs of names. The fourth argument is a list of source identifiers to skip translating, if desired. Authoritative online documentation for this command can be obtained directly from the `saw` executable via `:help write_rocq_cryptol_module`.

The resulting "output.v" file will have some of the usual hallmarks of computer-generated code; it will have poor formatting and, explicit parenthesis and fully-qualified names. Thankfully, once loaded into Rocq, the Rocq pretty-printer will do a much better job of rendering these terms in a somewhat human-readable way.

15.3 Proofs involving uninterpreted functions

It is possible to write a Cryptol module that references uninterpreted functions by using the `primitive` keyword to declare them in your Cryptol source. Primitive Cryptol declarations will be translated into Rocq section variables; as usual in Rocq, uses of these section variables will be translated into additional parameters to the definitions from outside the section. In this way, consumers of the translated module can instantiate the declared Cryptol functions with corresponding terms in subsequent Rocq developments.

Although the Cryptol interpreter itself will not be able to compute with declared but undefined functions of this sort, they can be used both to provide specifications for functions to be verified with `llvm_verify` or `jvm_verify` and also for Rocq extraction.

For example, if I write the following Cryptol source file:

```
primitive f : Integer -> Integer

g : Integer -> Bool
g x = f (f x) > 0
```

After extraction, the generated term `g` will have Rocq type:

```
(Integer -> Integer) -> Integer -> Bool
```

15.4 Translation limitations and caveats

Translation from Cryptol to Rocq has a number of fundamental limitations that must be addressed. The most severe of these is that Cryptol is a fully general-recursive language, and may exhibit runtime errors directly via calls to the `error` primitive, or via partial operations (such as indexing a sequence

out-of-bounds). The internal language of Rocq, by contrast, is a strongly-normalizing language of total functions. As such, our translation is unable to extract all Cryptol programs.

15.4.1 Recursive programs

The most severe of the currently limitations for our system is that the translation is unable to translate any recursive Cryptol program. Doing this would require us to attempt to find some termination argument for the recursive function sufficient to satisfy Rocq; for now, no attempt is made to do so. if you attempt to extract a recursive function, SAW will produce an error about a "malformed term" with `Prelude.fix` as the head symbol.

Certain limited kinds of recursion are available via the `foldl` Cryptol primitive operation, which is translated directly into a fold operation in Rocq. This is sufficient for many basic iterative algorithms.

15.4.2 Type coercions

Another limitation of the translation system is that Cryptol uses SMT solvers during its typechecking process and uses the results of solver proofs to justify some of its typing judgments. When extracting these terms to Rocq, type equality coercions must be generated. Currently, we do not have a good way to transport the reasoning done inside Cryptol's typechecker into Rocq, so we just supply a very simple `Ltac` tactic to discharge these coercions (see `solveUnsafeAssert` in `CryptolPrimitivesForSAWCoreExtra.v`). This tactic is able to discover simple coercions, but for anything nontrivial it may fail. The result will be a typechecking failure when compiling the generated code in Rocq when the tactic fails. If you encounter this problem, it may be possible to enhance the `solveUnsafeAssert` tactic to cover your use case.

15.4.3 Error terms

A final caveat that is worth mentioning is that Cryptol can sometimes produce runtime errors. These can arise from explicit calls to the `error` primitive, or from partially defined operations (e.g., division by zero or sequence access out of bounds). Such instances are translated to occurrences of an unrealized Rocq axiom named `error`. In order to avoid introducing an inconsistent environment, the `error` axiom is restricted to apply only to inhabited types. All the types arising from Cryptol programs are inhabited, so this is no problem in principle. However, collecting and passing this information around on the Rocq side is a little tricky.

The main technical device we use here is the `Inhabited` type class; it simply asserts that a type has some distinguished inhabitant. We provide instances for the base types and type constructors arising from Cryptol, so the necessary instances ought to be automatically constructed when needed. However, polymorphic Cryptol functions add some complications, as type arguments must also come together with evidence that they are inhabited. The translation process takes care to add the necessary `Inhabited` arguments, so everything ought to work out. However, if Rocq typechecking of generated code fails with errors about `Inhabited` class instances, it likely represents some problem with this aspect of the translation.

CHAPTER 16

Bisimulation Prover

SAW contains a bisimulation prover to prove that two terms simulate each other. This prover allows users to prove that two terms executing in lockstep satisfy some relations over the state of each circuit and their outputs. This type of proof is useful in demonstrating the eventual equivalence of two circuits, or of a circuit and a functional specification. SAW enables these proofs with the experimental `prove_bisim` command:

```
prove_bisim : ProofScript () -> [BisimTheorem] -> Term -> Term -> Term -
↪> Term -> TopLevel BisimTheorem
```

When invoking `prove_bisim strat theorems srel orel lhs rhs`, the arguments represent the following:

1. `strat`: A proof strategy to use during verification.
2. `theorems`: A list of already proven bisimulation theorems.
3. `srel`: A state relation between `lhs` and `rhs`. This relation must have the type `lhsState -> rhsState -> Bit`. The relation's first argument is `lhs`'s state prior to execution. The relation's second argument is `rhs`'s state prior to execution. `srel` then returns a `Bit` indicating whether the two arguments satisfy the bisimulation's state relation.
4. `orel`: An output relation between `lhs` and `rhs`. This relation must have the type `(lhsState, output) -> (rhsState, output) -> Bit`. The relation's first argument is a pair consisting of `lhs`'s state and output following execution. The relation's second argument is a pair consisting of `rhs`'s state and output following execution. `orel` then returns a `Bit` indicating whether the two arguments satisfy the bisimulation's output relation.
5. `lhs`: A term that simulates `rhs`. `lhs` must have the type `(lhsState, input) ->`

`(lhsState, output)`. The first argument to `lhs` is a tuple containing the internal state of `lhs`, as well as the input to `lhs`. `lhs` returns a tuple containing its internal state after execution, as well as its output.

6. `rhs`: A term that simulates `lhs`. `rhs` must have the type `(rhsState, input) -> (rhsState, output)`. The first argument to `rhs` is a tuple containing the internal state of `rhs`, as well as the input to `rhs`. `rhs` returns a tuple containing its internal state after execution, as well as its output.

On success, `prove_bisim` returns a `BisimTheorem` that can be used in future bisimulation proofs to enable compositional bisimulation proofs. On failure, `prove_bisim` will abort.

16.1 Bisimulation Example

This section walks through an example proving that the Cryptol implementation of an AND gate that makes use of internal state and takes two cycles to complete is equivalent to a pure function that computes the logical AND of its inputs in one cycle. First, we define the implementation's state type:

```
type andState = { loaded : Bit, origX : Bit, origY : Bit }
```

`andState` is a record type with three fields:

1. `loaded`: A `Bit` indicating whether the input to the AND gate has been loaded into the state record.
2. `origX`: A `Bit` storing the first input to the AND gate.
3. `origY`: A `Bit` storing the second input to the AND gate.

Now, we define the AND gate's implementation:

```
andImp : (andState, (Bit, Bit)) -> (andState, (Bit, Bit))
andImp (s, (x, y)) =
  if s.loaded /\ x == s.origX /\ y == s.origY
  then (s, (True, s.origX && s.origY))
  else ({ loaded = True, origX = x, origY = y }, (False, 0))
```

`andImp` takes a tuple as input where the first field is an `andState` holding the gate's internal state, and second field is a tuple containing the inputs to the AND gate. `andImp` returns a tuple consisting of the updated `andState` and the gate's output. The output is a tuple where the first field is a ready bit that is `1` when the second field is ready to be read, and the second field is the result of gate's computation.

`andImp` takes two cycles to complete:

1. The first cycle loads the inputs into its state's `origX` and `origY` fields and sets `loaded` to `True`. It sets both of its output bits to `0`.

2. The second cycle uses the stored input values to compute the logical AND. It sets its ready bit to `1` and its second output to the logical AND result.

So long as the inputs remain fixed after the second cycle, `andImp`'s output remains unchanged. If the inputs change, then `andImp` restarts the computation (even if the inputs change between the first and second cycles).

Next, we define the pure function we'd like to prove `andImp` bisimilar to:

```
andSpec : ((), (Bit, Bit)) -> ((), (Bit, Bit))
andSpec (_, (x, y)) = ((), (True, x && y))
```

`andSpec` takes a tuple as input where the first field is `()`, indicating that `andSpec` is a pure function without internal state, and the second field is a tuple containing the inputs to the AND function. `andSpec` returns a tuple consisting of `()` (again, because `andSpec` is stateless) and the function's output. Like `andImp`, the output is a tuple where the first field is a ready bit that is `1` when the second field is ready to be read, and the second field is the result of the function's computation.

`andSpec` completes in a single cycle, and as such its ready bit is always `1`. It computes the logical AND directly on the function's inputs using Cryptol's `(&&)` operator.

Next, we define a state relation over `andImp` and `andSpec`:

```
andStateRel : andState -> () -> Bit
andStateRel _ () = True
```

`andStateRel` takes two arguments:

1. An `andState` for `andImp`.
2. An empty state (`()`) for `andSpec`.

`andStateRel` returns a `Bit` indicating whether the relation is satisified. In this case, `andStateRel` always returns `True` because `andSpec` is stateless and therefore the state relation permits `andImp` to accept any state.

Lastly, we define a relation over `andImp` and `andSpec`:

```
andOutputRel : (andState, (Bit, Bit)) -> ((), (Bit, Bit)) -> Bit
andOutputRel (s, (impReady, impO)) ((), (_, specO)) =
  if impReady then impO == specO else True
```

`andOutputRel` takes two arguments:

1. A return value from `andImp`. Specifically, a pair consisting of an `andState` and a pair containing a ready bit and result of the logical AND.
2. A return value from `andSpec`. Specifically, a pair consisting of an empty state `()` and a pair containing a ready bit and result of the logical AND.

`andOutputRel` returns a `Bit` indicating whether the relation is satisfied. It considers the relation satisfied in two ways:

1. If `andImp`'s ready bit is set, the relation is satisfied if the output values `impO` and `specO` from `andImp` and `andSpec` respectively are equivalent.
2. If `andImp`'s ready bit is not set, the relation is satisfied.

Put another way, the relation is satisfied if the end result of `andImp` and `andSpec` are equivalent. The relation permits intermediate outputs to differ.

We can verify that this relation is always satisfied–and therefore the two terms are bisimilar–by using `prove_bisim`:

```
import "And.cry";
enable_experimental;

and_bisim <- prove_bisim z3 [] {{ andStateRel }} {{ andOutputRel }} {{␣
↪andImp }} {{ andSpec }};
```

Upon running this script, SAW prints:

```
Successfully proved bisimulation between andImp and andSpec
```

16.1.1 Building a NAND gate

We can make the example more interesting by reusing components to build a NAND gate. We first define a state type for the NAND gate implementation that contains `andImp`'s state. This NAND gate will not need any additional state, so we will define a type `nandState` that is equal to `andState`:

```
type nandState = andState
```

Now, we define an implementation `nandImp` that calls `andImp` and negates the result:

```
nandImp : (nandState, (Bit, Bit)) -> (nandState, (Bit, Bit))
nandImp x = (s, (andReady, ~andRes))
  where
    (s, (andReady, andRes)) = andImp x
```

Note that `nandImp` is careful to preserve the ready status of `andImp`. Because `nandImp` relies on `andImp`, it also takes two cycles to compute the logical NAND of its inputs.

Next, we define a specification `nandSpec` in terms of `andSpec`:

```
nandSpec : ((), (Bit, Bit)) -> ((), (Bit, Bit))
nandSpec (_, (x, y)) = ((), (True, ~ (andSpec ((), (x, y))).1.1))
```

As with `andSpec`, `nandSpec` is pure and computes its result in a single cycle.

Next, we define a state relation over `nandImp` and `nandSpec`:

```
nandStateRel : andState -> () -> Bit
nandStateRel _ () = True
```

As with `andStateRel`, this state relation is always `True` because `nandSpec` is stateless.

Lastly, we define an output relation indicating that `nandImp` and `nandSpec` produce equivalent results once `nandImp`'s ready bit is `1`:

```
nandOutputRel : (nandState, (Bit, Bit)) -> ((), (Bit, Bit)) -> Bit
nandOutputRel (s, (impReady, impO)) ((), (_, specO)) =
  if impReady then impO == specO else True
```

To prove that `nandImp` and `nandSpec` are bisimilar, we again use `prove_bisim`. This time however, we can reuse the bisimulation proof for the AND gate by including it in the `theorems` paramter for `prove_bisim`:

```
prove_bisim z3 [and_bisim] {{ nandStateRel }} {{ nandOutputRel }} {{␣
↪nandImp }} {{ nandSpec }};
```

Upon running this script, SAW prints:

```
Successfully proved bisimulation between nandImp and nandSpec
```

16.2 Understanding the proof goals

While not necessary for simple proofs, more advanced proofs may require inspecting proof goals. `prove_bisim` generates and attempts to solve the following proof goals:

```
OUTPUT RELATION THEOREM:
  forall s1 s2 in.
    srel s1 s2 -> orel (lhs (s1, in)) (rhs (s2, in))

STATE RELATION THEOREM:
  forall s1 s2 out1 out2.
    orel (s1, out1) (s2, out2) -> srel s1 s2
```

where the variables in the `forall`s are:

- `s1`: Initial state for `lhs`
- `s2`: Initial state for `rhs`
- `in`: Input value to `lhs` and `rhs`
- `out1`: Initial output value for `lhs`

- `out2`: Initial output value for `rhs`

The `STATE RELATION THEOREM` verifies that the output relation properly captures the guarantees of the state relation. The `OUTPUT RELATION THEOREM` verifies that if `lhs` and `rhs` are executed with related states, then the result of that execution is also related. These two theorems together guarantee that the terms simulate each other.

When using composition, `prove_bisim` also generates and attempts to solve the proof goal below for any successfully applied `BisimTheorem` in the `theorems` list:

```
COMPOSITION SIDE CONDITION:
  forall g_lhs_s g_rhs_s.
    g_srel g_lhs_s g_rhs_s -> f_srel f_lhs_s f_rhs_s
    where
      f_lhs_s = extract_inner_state g_lhs g_lhs_s f_lhs
      f_rhs_s = extract_inner_state g_rhs g_rhs_s f_rhs
```

where `g_lhs` is an outer term containing a call to an inner term `f_lhs` represented by a `BisimTheorem` and `g_rhs` is an outer term containing a call to an inner term `f_rhs` represented by the same `BisimTheorem`. The variables in `COMPOSITION SIDE CONDITION` are:

- `extract_inner_state x x_s y`: A helper function that takes an outer term `x`, an outer state `x_s`, and an inner term `y`, and returns the inner state of `x_s` that `x` passes to `y`.
- `g_lhs_s`: The state for `g_lhs`
- `g_rhs_s`: The state for `g_rhs`
- `g_srel`: The state relation for `g_lhs` and `g_rhs`
- `f_srel`: The state relation for `f_lhs` and `f_rhs`
- `f_lhs_s`: The state for `f_lhs`, as represented in `g_lhs_s` (extracted using `extract_inner_state`).
- `f_rhs_s`: The state for `f_rhs`, as represented in `g_rhs_s` (extracted using `extract_inner_state`).

The `COMPOSITION SIDE CONDITION` exists to verify that the terms in the bisimulation relation properly set up valid states for subterms they contain.

16.3 Limitations

For now, the `prove_bisim` command has a couple limitations:

- `lhs` and `rhs` must be named functions. This is because `prove_bisim` uses these names to perform substitution when making use of compositionality.
- Each subterm present in the list of bisimulation theorems already proven may be invoked at most once in `lhs` or `rhs`. That is, if some function `g_lhs` calls `f_lhs`, and `prove_bisim`

is invoked with a `BisimTheorem` proving that `f_lhs` is bisimilar to `f_rhs`, then `g_lhs` may call `f_lhs` at most once.

CHAPTER 17

Command Line Reference

This chapter documents the command-line interfaces of the `saw` and `saw-remote-api` executables in full detail.

17.1 `saw` Command Line

There are three ways to run the `saw` executable:

- `saw` [*options*] : runs the interactive REPL.
- `saw` [*options*] *filename*`.saw` : loads and runs the SAWScript proof logic in the given filename.
- `saw` [*options*] `-B` *filename*`.isaw` : loads and runs the filename as a series of REPL commands.

The `-I` option forces REPL mode, even if a filename is given. Any such filename is ignored. There is, for the time being at least, no easy way to first load a file and then enter the REPL. (The best way to do that is to start the REPL and then load the file manually with `include`. Another way is to add the experimental command `subshell` to the end of the file.)

The `-I` and `-B` options cannot be used together.

Files loaded with the `-B` option can contain REPL `:`-commands. However, because the file is fed into the system one line at a time, as if each line were typed separately, each line must be syntactically complete. The adjustments and affordances in the REPL meant for interactive use are also engaged, which can in some cases produce unexpected results. In general the `-B` option is mostly useful for testing SAW.

The `-B` option can also be spelled `--batch` and the `-I` option can also be spelled `--interactive`.

A number of the following options take lists of filenames and/or directories. Per convention, on Unix (including MacOS) these lists are delimited by colons; on Windows, use semicolons.

The following additional options are available:

-b *list*, --java-bin-dirs=*list*
: Specify a list of directories to search for a Java executable. If this option is not given, the `PATH` environment variable will be used instead.

-c *list*, --classpath=*list*
: Specify a list of directories to search for Java classes. If multiple `-c` options are found, the lists given are concatenated in the order seen on the command line. The `CLASSPATH` environment variable is also used; its contents go on the search path last. (That is, directories given with `-c` are searched before directories in `CLASSPATH`.)

--clean-mismatched-versions-solver-cache[=*dir*]
: Run the `clean_mismatched_versions_solver_cache` command on the solver cache in the given directory. If no directory is given, the `SAW_SOLVER_CACHE_PATH` environment variable is used to find the solver cache. After cleaning out the cache, exit. See *Caching Solver Results* for a description of the `clean_mismatched_versions_solver_cache` command and the solver caching feature in general.

-d *num*, --sim-verbose=*num*
: Set the verbosity level of the LLVM/Java/MIR symbolic simulator.

--detect-vacuity
: Check for contradictory assumptions. This is not the default because it can be expensive and is rarely needed.

-f *format*, --summary-format=*format*
: Set the output format for the verification summary generated with `-s`. The recognized formats are `json` and `pretty`; the default is `pretty`. `pretty` is intended to be human-readable; `json` is more useful for automated processing.

-h, --help, -?
: Print a usage message. This lists all the available options.

-i *list*, --import-path=*list*
: Specify a list of directories to search for SAWScript includes. If multiple `-i` options are found, the lists given are concatenated in the order seen on the command line. The `SAW_IMPORT_PATH` environment variable is also used; its contents go on the search path last. (That is, directories given with `-i` are searched before directories in `SAW_IMPORT_PATH`.)

-j *list*, --jars=*list*
: Specify a list of paths to `.jar` files to search for Java classes. The search ordering of `.jar` files given with `-j` relative to `.jar` files given with `-c` or in the `CLASSPATH` environment variable is not specified.

--no-color
: Disable terminal colors and also Unicode character output.

--output-locations
: Print the current SAWScript source location with every output message. This feature is eventually intended to be replaced with a more helpful tracing facility.

-s *filename*, **--summary=***filename*
: Write a verification summary to the given file after all proofs are done.

-t, **--extra-type-checking**
: Perform extra type checking of intermediate values. This option no longer does anything and will be removed eventually.

-T, **--timestamping**
: Add a timestamp to messages printed during execution.

-v *num*, **--verbose=***num*
: Set the verbosity level of the SAWScript interpreter. Recognized verbosity levels range from 0 to 5, and the default is 4. This option is eventually intended to be replaced with a more directed scheme for controlling the output of different operations and subsystems.

-V, **--version**
: Show the version of the SAWScript interpreter.

17.2 `saw` Environment Variables

The following environment variables also affect `saw`.

Some of these hold lists of filenames and/or directories. Per convention, on Unix (including MacOS) these lists are delimited by colons; on Windows, use semicolons.

CLASSPATH
: Specify a list of directories to search for Java classes. Note that paths can also be specified using the `-c` (aka `--classpath`) command-line option. Paths given with `-c` take priority.

CRYPTOLPATH
: Specify a list of directories to search for Cryptol imports (including the Cryptol prelude).

PATH
: If the `--java-bin-dirs` option is not given, then the `PATH` will be searched to find a Java executable.

SAW_IMPORT_PATH
: Specify a list of directory paths to search for imports. Note that paths can also be specified using the `-i` (aka `--import-path`) command-line option. Paths given with `-i` take priority.

SAW_JDK_JAR
: Specify the path of the `.jar` file containing the core Java libraries. This is only necessary if a Java executable is not found on the `PATH` or via the `--java-bin-dirs` option.

SAW_SOLVER_CACHE_PATH
: Specify a path at which to keep a cache of solver results obtained during calls to certain tactics.

A cache is not created at this path until it is needed. See *Caching Solver Results* for further information.

17.3 `saw-remote-api` Command Line

The general form of the `saw-remote-api` command line is one of:

- `saw-remote-api -h` : Print the command-line usage text and exit. `-h` can also be spelled `--help` and `-?` is also accepted.
- `saw-remote-api -v` : Print the SAW version and exit. `-v` can also be spelled `--version`.

`saw-remote-api` doc
Dump out the API documentation text.

`saw-remote-api` *mode* `-h`
Print the usage text for the given *mode*. `-h` can also be spelled `--help`.

`saw-remote-api` [*overall-options*] *mode* [*mode-options*] [*mode-args*]
Run the remote API server in the given *mode*.

The available modes are `stdio`, `socket`, and `http`.

17.3.1 Overall Options

The following options are accepted in any mode:

`--log` *destination*
Enables fairly verbose connection logs. The *destination* should be either a filename or the magic string `stderr` to print to `saw-remote-api`'s standard error.

`--read-only`
Avoids generating any output files. Useful if running on a read-only file system. By default `saw-remote-api` saves state to disk so server crashes don't lose client context.

`--max-occupancy` *num*
Sets the maximum number of sessions allowed at once. The default is 1.

`--no-evict`
Don't evict old sessions if someone connects when the server is full.

17.3.2 `stdio` mode

`saw-remote-api stdio` communicates over `stdin` and `stdout`.

The only *mode-option* for `stdio` mode is `-h`. There are no *mode-args*.

17.3.3 `socket` mode

In socket mode `saw-remote-api` communicates over a socket using a simple "netstrings" protocol wrapping the basic JSON packets.

The Python bindings will run `saw-remote-api` in socket mode by default if not pointed to another location.

The *mode-options* for socket mode (besides `-h`) are:

`--session` *session-name*
: Serve as the named session *session-name*. Fails if there's already a server for that session.

`--host` *hostname*
: Set the listen hostname or interface address. On multihomed machines, listening on specific addresses allows accepting connections from some networks and not others. To listen for connections from anywhere, use `0.0.0.0` or `::`. To restrict connections to the same machine, use `127.0.0.1` or `::1`. The default is `::1`.

`--port` *port*
: Set the listen port number. If no explicit port is selected `saw-remote-api` lets the OS pick one. In any case the port number is printed to standard output during startup.

Socket mode uses no *mode-args*.

17.3.4 `http` mode

In HTTP mode `saw-remote-api` communicates over a socket using both HTTP and "netstrings" to wrap the basic JSON packets.

The *mode-options* for HTTP mode (besides `-h`) are:

`--tls`
: Enable TLS (transport layer security, aka more-modern SSL) and run over HTTPS. Setting the environment variable `TLS_ENABLE` has the same effect.

`--session` *session-name*
: This option is recognized but not supported for HTTP.

`--host` *hostname*
: Set the listen hostname or interface address. On multihomed machines, listening on specific addresses allows accepting connections from some networks and not others. To listen for connections from anywhere, use `0.0.0.0` or `::`. To restrict connections to the same machine, use `127.0.0.1` or `::1`. The default is `0.0.0.0`. Note that this is different from socket mode.

`--port` *port*
: Set the listen port number. If no explicit port is selected the default is 8080.

In HTTP mode one *mode-arg* must be provided: the name of the HTTP endpoint to serve the protocol on. For example, `saw-remote-api http --host 127.0.0.1 /foo` will respond to requests made to `http://127.0.0.1:8080/foo`.

17.4 `saw-remote-api` Environment Variables

> **Warning**
>
> This section is under construction!

CHAPTER 18

SAWScript Language Reference

> **Warning**
>
> This section is under construction!

18.1 Using Cryptol in SAWScript

The primary use of Cryptol within SAWScript is to construct values of type `Term`. Although `Term` values can be constructed from various sources, inline Cryptol expressions are the most direct and convenient way to create them.

Specifically, a Cryptol expression can be placed inside double curly braces (`{{` and `}}`), resulting in a value of type `Term`. As a very simple example, there is no built-in integer addition operation in SAWScript. However, we can use Cryptol's built-in integer addition operator within SAWScript as follows:

```
sawscript> let t = {{ 0x22 + 0x33 }}
sawscript> print t
85
sawscript> :type t
Term
```

Although it printed out in the same way as an `Int`, it is important to note that `t` actually has type `Term`. We can see how this term is represented internally, before being evaluated, with the `print_term` function.

```
sawscript> print_term t
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.TCNum 8
      x@3 = Cryptol.PLiteralSeqBool x@2
    }
 in Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool x@2)
      (Cryptol.ecNumber (Cryptol.TCNum 34) x@1 x@3)
      (Cryptol.ecNumber (Cryptol.TCNum 51) x@1 x@3)
```

For the moment, it's not important to understand what this output means. We show it only to clarify that `Term` values have their own internal structure that goes beyond what exists in SAWScript. The internal representation of `Term` values is in a language called SAWCore. The full semantics of SAWCore are beyond the scope of this manual.

The text constructed by `print_term` can also be accessed programmatically (instead of printing to the screen) using the `show_term` function, which returns a `String`. The `show_term` function is not a command, so it executes directly and does not need `<-` to bind its result. Therefore, the following will have the same result as the `print_term` command above:

```
sawscript> let s = show_term t
sawscript> :type s
String
sawscript> print s
<same as above>
```

Numbers are printed in decimal notation by default when printing terms, but the following two commands can change that behavior.

- `set_ascii : Bool -> TopLevel ()`, when passed `true`, makes subsequent `print_term` or `show_term` commands print sequences of bytes as ASCII strings (and doesn't affect printing of anything else).
- `set_base : Int -> TopLevel ()` prints all bit vectors in the given base, which can be between 2 and 36 (inclusive).

A `Term` that represents an integer (any bit vector, as affected by `set_base`) can be translated into a SAWScript `Int` using the `eval_int : Term -> Int` function. This function returns an `Int` if the `Term` can be represented as one, and fails at runtime otherwise.

```
sawscript> print (eval_int t)
85
sawscript> print (eval_int {{ True }})

"eval_int" (<stdin>:1:1):
eval_int: argument is not a finite bitvector
sawscript> print (eval_int {{ [True] }})
```

(continues on next page)

(continued from previous page)

```
1
```

Similarly, values of type `Bit` in Cryptol can be translated into values of type `Bool` in SAWScript using the `eval_bool : Term -> Bool` function:

```
sawscript> let b = {{ True }}
sawscript> print_term b
Prelude.True
sawscript> print (eval_bool b)
true
```

Anything with sequence type in Cryptol can be translated into a list of `Term` values in SAWScript using the `eval_list : Term -> [Term]` function.

```
sawscript> let l = {{ [0x01, 0x02, 0x03] }}
sawscript> print_term l
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.PLiteralSeqBool (Cryptol.TCNum 8)
    }
 in [Cryptol.ecNumber (Cryptol.TCNum 1) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 2) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 3) x@1 x@2]
sawscript> print (eval_list l)
[Cryptol.ecNumber (Cryptol.TCNum 1) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
,Cryptol.ecNumber (Cryptol.TCNum 2) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
,Cryptol.ecNumber (Cryptol.TCNum 3) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))]
```

Finally, a list of `Term` values in SAWScript can be collapsed into a single `Term` with sequence type using the `list_term : [Term] -> Term` function, which is the inverse of `eval_list`.

```
sawscript> let ts = eval_list l
sawscript> let l = list_term ts
sawscript> print_term l
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.PLiteralSeqBool (Cryptol.TCNum 8)
    }
 in [Cryptol.ecNumber (Cryptol.TCNum 1) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 2) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 3) x@1 x@2]
```

In addition to being able to extract integer and Boolean values from Cryptol expressions, `Term` values

can be injected into Cryptol expressions. When SAWScript evaluates a Cryptol expression between `{{` and `}}` delimiters, it does so with several extra bindings in scope:

- Any variable in scope that has SAWScript type `Bool` is visible in Cryptol expressions as a value of type `Bit`.
- Any variable in scope that has SAWScript type `Int` is visible in Cryptol expressions as a *type variable*. Type variables can be demoted to numeric bit vector values using the backtick (`` ` ``) operator.
- Any variable in scope that has SAWScript type `Term` is visible in Cryptol expressions as a value with the Cryptol type corresponding to the internal type of the term. The power of this conversion is that the `Term` does not need to have originally been derived from a Cryptol expression.

In addition to these rules, bindings created at the Cryptol level, either from imported files or inside Cryptol quoting brackets, are visible only to later Cryptol expressions, and not as SAWScript variables.

To make these rules more concrete, consider the following examples. If we bind a SAWScript `Int`, we can use it as a Cryptol type variable. If we create a `Term` variable that internally has function type, we can apply it to an argument within a Cryptol expression, but not at the SAWScript level:

```
sawscript> let n = 8
sawscript> :type n
Int
sawscript> let {{ f (x : [n]) = x + 1 }}
sawscript> :type {{ f }}
Term
sawscript> :type f

<stdin>:1:1-1:2: unbound variable: "f" (<stdin>:1:1-1:2)
sawscript> print {{ f 2 }}
3
```

If `f` was a binding of a SAWScript variable to a `Term` of function type, we would get a different error:

```
sawscript> let f = {{ \(x : [n]) -> x + 1 }}
sawscript> :type {{ f }}
Term
sawscript> :type f
Term
sawscript> print {{ f 2 }}
3
sawscript> print (f 2)

type mismatch: Int -> t.0 and Term
 at "_" (REPL)
```

(continues on next page)

(continued from previous page)

```
mismatched type constructors: (->) and Term
```

One subtlety of dealing with `Term`s constructed from Cryptol is that because the Cryptol expressions themselves are type checked by the Cryptol type checker, and because they may make use of other `Term` values already in scope, they are not type checked until the Cryptol brackets are evaluated. So type errors at the Cryptol level may occur at runtime from the SAWScript perspective (though they occur before the Cryptol expressions are run).

So far, we have talked about using Cryptol *value* expressions. However, SAWScript can also work with Cryptol *types*. The most direct way to refer to a Cryptol type is to use type brackets: `{|` and `|}`. Any Cryptol type written between these brackets becomes a `Type` value in SAWScript. Some types in Cryptol are *numeric* (also known as *size*) types, and correspond to non-negative integers. These can be translated into SAWScript integers with the `eval_size` function. For example:

```
sawscript> let {{ type n = 16 }}
sawscript> eval_size {| n |}
16
sawscript> eval_size {| 16 |}
16
```

For non-numeric types, `eval_size` fails at runtime:

```
sawscript> eval_size {| [16] |}

"eval_size" (<stdin>:1:1):
eval_size: not a numeric type
```

In addition to the use of brackets to write Cryptol expressions inline, several built-in functions can extract `Term` values from Cryptol files in other ways. The `import` command at the top level imports all top-level definitions from a Cryptol file or module and places them in scope within later bracketed expressions. This includes Cryptol `foreign` declarations (https://galoisinc.github.io/cryptol/master/FFI.html). If a Cryptol implementation of a foreign function (https://galoisinc.github.io/cryptol/master/FFI.html#cryptol-implementation-of-foreign-functions) is present, then it will be used as the definition when reasoning about the function. Otherwise, the function will be imported as an opaque constant with no definition.

The `cryptol_load` command behaves similarly, but returns a `CryptolModule` instead. If any `CryptolModule` is in scope, its contents are available qualified with the name of the `CryptolModule` variable. A specific definition can be explicitly extracted from a `CryptolModule` using the `cryptol_extract` command:

- `cryptol_extract : CryptolModule -> String -> TopLevel Term`

CHAPTER 19

SAWScript Commands Reference

Warning

This section is under construction!

CHAPTER 20

SAW REPL Reference

The primary mechanism for interacting with SAW is through the `saw` executable included as part of the standard binary distribution. With no arguments, `saw` starts a read-evaluate-print loop (REPL) that allows the user to interactively evaluate commands in the SAWScript language.

20.1 REPL Commands

There are a number of REPL-specific commands that can be accessed by typing a colon and the command name, and perhaps some arguments.

- `:?` prints the list of `:`-commands with brief descriptions. If given an argument, it instead looks up a SAWScript builtin and prints its type and help text.
- `:cd` changes the REPL's current directory.
- `:env` displays the values and types of all currently bound variables, including built-in functions and commands.
- `:help` or `:h` is the same as `:?`.
- `:llvmdis` disassembles an LLVM module. See separate subsection below.
- `:pwd` prints the REPL's current directory.
- `:quit` or `:q` exits the REPL. (If the REPL was started as a subshell, it only exits that subshell, not the whole of SAW.)
- `:search` with one or more types (complex types go in parentheses) prints matching currently bound variables. See separate subsection below.

- `:tenv` displays the values and types of all currently bound types and type aliases, including many (but currently not all) built-in types.
- `:type` or `:t` checks and prints the type of an arbitrary SAWScript expression:

```
sawscript> :t show
{a.0} a.0 -> String
```

20.1.1 Using `:search`

The REPL `:search` command takes one or more type patterns as arguments, and searches the current value namespace (including functions and builtins) for objects that mention types matching all the patterns given. In practice this is mostly useful for searching for builtins.

Type patterns are type names extended with _ as a wildcard. Thus for example `[_]` matches any list type. You may also forall-bind type variables before the patterns using the `{a}` syntax. The scope of such bindings is the whole search.

Complex patterns should be written in parentheses; otherwise they become syntactically ambiguous. Note in particular that parentheses are required to treat a type constructor application to another type as a single search term. Without parentheses, `:search` would treat the unapplied type constructor and the other type as two separate search terms. For instance, `:search ProofScript Int` will search for objects mentioning both `ProofScript` (applied to anything) and `Int`. To search for objects that mention the type `ProofScript Int`, write `:search (ProofScript Int)`.

Type variables in the search patterns are matched as follows:

- Already-bound type variables (typedef names, certain builtin types) must match exactly.
- Free type variables match any type, but all occurrences must match the same type. Thus for example `[a] -> a` matches `sum : [Int] -> Int` and `head : {t} [t] -> t` but not `length : {t} [t] -> Int`.

  This is true across all patterns in the same search; searching for `[a]`, `[b]`, and `a -> b` will match `map : {a, b} (a -> b) -> [a] -> [b]`, as well as `pam : {a, b} [a] -> (a -> b) -> [b]` if you define such a thing. But it won't match `mapFirst : {a, b, c} (a -> b) [(a, c)] -> [(b, c)]`.

  Perhaps unfortunately, it *will* match `intNth : [Int] -> Int -> Int`. The search logic does not require distinct patterns to match distinct parts of the target type, nor is there a way to prevent it from picking the same substitution for both `a` and `b`. (Neither of these behaviors is entirely desirable and might be improved in the future.)
- Forall-bound type variables in the pattern are matched in the same way as free type variables, but will *only* match forall-bound type variables in the search space. Thus, `:search {a} (a -> a)` will match `id : {a} a -> a` but not `inc: Int -> Int`, while `:search (a -> a)` will match both. This is helpful to search specifically for polymorphic functions.

Because SAWScript functions are curried, searching for `Int -> Int` will match `String -> Int`

`-> Int`. However, it will not match `Int -> String -> Int`. The best way to search for a function that takes `Int` in *any* argument position and also returns `Int` is by searching for both `Int -> _` and `_ -> Int`: `:search (Int -> _) (_ -> Int)`.

There is, however, no good way yet to search for a function that takes two `Int`s in arbitrary argument positions. Searching for `Int -> Int -> _` will only find functions where the two arguments are adjacent, `Int -> _ -> Int -> _` will only find functions where one other argument is between them, and searching for `Int -> _` twice falls afoul of the limitation where two patterns can match the same thing.

20.1.2 Using `:llvmdis`

`:llvmdis` disassembles LLVM bitcode or prints other metadata from an `LLVMModule`. The first argument is the name of a SAWScript-level `LLVMModule` that has been loaded.

The second argument selects an object to disassemble. If it is a `!` followed by a number *N*, the _N_th unnamed metadata reference is extracted and printed. If it contains a `:`, it is treated as a filename and line number and the code or data structure associated with that location is extracted and printed. Otherwise it is treated as a function name to disassemble.

The output is comparable to LLVM's `llvm-dis` tool but is based on SAW's internal representation and knowledge of the LLVM module.

Examples:

```
sawscript> bc <- llvm_load_module "intTests/testmulti/foo.bc"
sawscript> :llvmdis bc foo.c:2
  ... shows just line 2 of foo.c in LLVM textual format
sawscript> :llvmdis bc !10
  ... shows the metadata at index 10
sawscript> :llvmdis bc foo
  ... shows the foo function in LLVM textual format
```

CHAPTER 21

Python API Reference

Warning

This section is under construction!

CHAPTER 22

SAWCore Language Reference

Warning

This section is under construction!

CHAPTER 23

Mapping Cryptol to SAWCore

Warning

This section is under construction!

CHAPTER 24

Appendices

24.1 Formal Deprecation Process

SAW primitives, and thus their associated SAWScript built-ins, sometimes become obsolete or are found inadequate and replaced. The process by which that happens has three steps, as follows:

1. The decision is made to deprecate and eventually remove the objects in question. This can happen at the level of individual built-in elements (for example, when replacing a function with an awkward interface or unfortunate name) or at the level of internal units of functionality with possibly multiple built-ins affected. At this step the built-ins in question are marked for a deprecation warning. They remain available by default, but referring to them will trigger a warning.
2. The objects in question are made invisible by default. Now, referring to the affected built-ins will fail unless the `enable_deprecated` command is used. In that case referring to them will still produce a warning.
3. The objects in question are removed entirely and are no longer available.

In general any object or group of objects will move only one step per release; that is, something first marked deprecated (so it warns) in SAW 1.2 will not disappear by default before SAW 1.3 and not be removed entirely before SAW 1.4. The time frame may be longer depending on the needs of downstream users, the complexity of migration, and the cost/impact of keeping the deprecated code in the system.

We may move faster if circumstances dictate, but hope not to need to.

Objects that have never appeared in a release, or that have never moved past experimental, may be removed without first being deprecated. However, we aim to avoid this in cases where the objects in

question have gotten substantial use despite their formal status.

24.2 Deprecated Items

This appendix attempts to document the things in SAW that have been deprecated and removed over recent releases, in more detail than will fit in the change log and release notes.

24.2.1 Warning When Used

The following elements are currently deprecated and warn when used; this has happened since SAW 1.5 was released, so they will be/are that way in SAW 1.6. They are expected to be hidden by default in SAW 1.7.

- The `prove_extcore` builtin; you can just use `prove_print` instead now.
- The `add_prelude_eqs` and `add_cryptol_eqs` builtins; use `add_core_thms` instead.
- The `assume_unsat` builtin, which was informally deprecated years ago; use `admit` instead. `admit` takes an extra string argument that's supposed to state why you're admitting the theorem without proof.
- The type `SetupValue` (an old name for `LLVMValue`)
- The type `CrucibleMethodSpec` (an old name for `LLVMSpec`)
- The type `CrucibleSetup` (an old name for `LLVMSetup`)

The following elements were deprecated as of SAW 1.5, and currently warn. They will be hidden by default when we remove Boolector from what4-solvers. This will happen at some point after SAW 1.6; the time frame depends mostly on the maintenance burden.

- The commands related to the Boolector solver, which has been replaced upstream with Bitwuzla. All `boolector` commands have equivalent `bitwuzla` commands.

24.2.2 Hidden by Default

The following elements were previously experimental and will be deprecated and hidden by default as of SAW 1.6:

- The experimental builtin `crucible_llvm_verify_x86` (an old name for `llvm_verify_x86`), which was left in place in SAW 1.5 due to some downstream uses that needed to be migrated first.

There are not yet any (non-experimental) elements that were first deprecated as of SAW 1.6 and are now hidden by default. (Such elements would continue to be hidden by default in SAW 1.7 and would be removed in SAW 1.8.)

The following elements were deprecated as of SAW 1.5 (or earlier), and are hidden by default as of SAW 1.6. They are expected to be removed in SAW 1.7.

- The commands related to the CVC4 solver; proofs should be migrated to CVC5. Each `cvc4` builtin has a corresponding `cvc5` builtin. If you have proofs that work with `cvc4` but time out in `cvc5`, and you cannot find another solver that can handle them instead, please file an issue (with us or CVC5 upstream as seems best).
- The `external_aig_solver` builtin, which was renamed to `arbitrary_aig`.
- The `external_cnf_solver` builtin, which was renamed to `arbitrary_cnf`.
- The `w4_offline_smtlib2` builtin, which was renamed to `offline_w4_smtlib2` for consistency.
- The `llvm_struct` builtin; the current name for it is `llvm_alias`, because it looks up typedef names. If you are trying to create an LLVM struct type from its contents, as many historical callers were found to be, you want `llvm_struct_type` instead.
- The old `coq` names of the Rocq backend; change `coq` to `rocq`.
- The many old `crucible` names of LLVM backend functions; e.g. `crucible_null` is now `llvm_null`. Most of these names have been outdated for years.
- Similarly, the `crucible_java_extract` builtin, which is an old name for `jvm_extract`.

24.2.3 Removed

The following elements have been removed entirely:

- The `env` builtin (use the `:env` REPL command instead)
- Certain old `crucible` names for LLVM backend functions that never made it past "experimental":
  - `crucible_fresh_cryptol_var`
  - `crucible_alloc_with_size`
  - `crucible_points_to_array_prefix`
  - `crucible_llvm_compositional_extract`
  - `crucible_llvm_array_size_profile`
- The `addsimp'` and `addsimps'` builtins, which added unproven rewrite rules to a `Simpset`. Don't do that; it's borrowing trouble. If you can't prove your rewrite rules, for whatever, reason, use `admit` explicitly and then use the ordinary `addsimp` and/or `addsimps` bulitins to add them to `Simpset`s.
- The `crucible_setup_val_to_term` builtin. This was a partial inverse for `llvm_term`; that is, it turned an `LLVMValue` value back to a `Term` if it was one that could be represented. In general LLVM specs should be written so they lift values from `Term` to `LLVMValue` and not the other way around.
- The unused type `JavaSetup`; any stray references to it should be changed to `JVMSetup`.

- The `extract_uninterp` builtin. This was a way to rewrite a `Term` to make functions uninterpreted (generalize it over all functions that are functions in place of specific functions with definitions) and also extract the uses of those functions for further processing. This depended on problematic internal functionality, and also relied on being able to identify which applications of a given function are actually the same, which is difficult in general and not the right approach.
- The Heapster, MRSolver, and Monadify features, which proved unmaintainable and inconsistent with efforts to build SAW an assurance case.
- The `sbv_uninterpreted` and `read_sbv` builtins and their `Uninterp` type. This was code for importing from an obsolete file format.
- The `callcc` builtin, whose behavior was erratic, whose internal implementation was insupportable, and which had only one known use that was itself better done some other way.

Also, the following SAWScript language-level behavior has been removed:

- It used to be possible to update top-level variable bindings by just binding the same variable again. This would update some uses and not others depending on the then-very-dodgy evaluation semantics of SAWScript. Now if you bind the same variable again, it just shadows the old variable in the namespace and does not replace it.

  For uses that really need the mutability behavior, you can now, at the top level only, do `let rebindable ...`, which lets you bind a variable that you can update with another `let rebindable ...`, and the semantics of the update are predictable.

24.3 Glossary

Warning

This section is under construction!